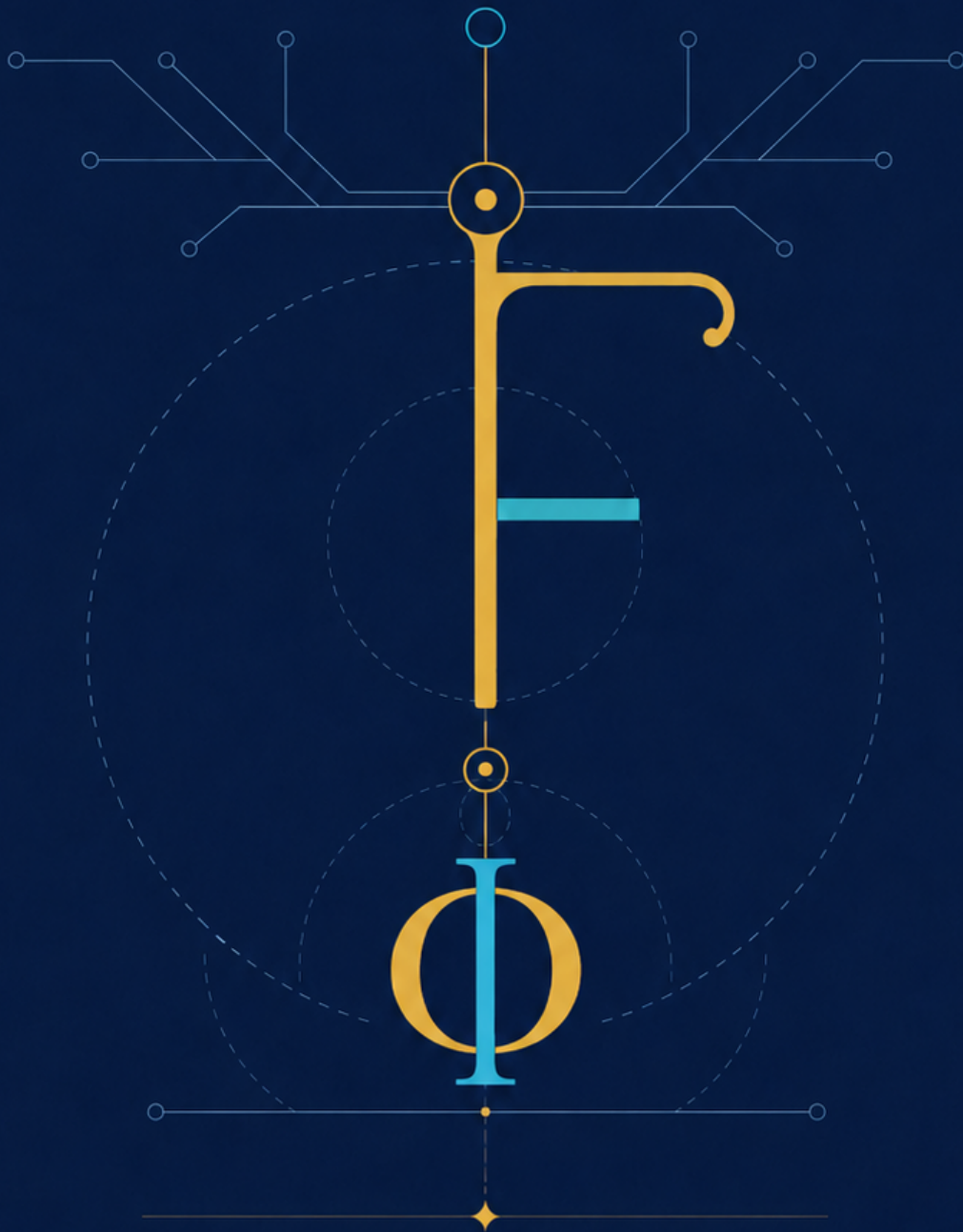


FO λ ANG

LANGUAGE REFERENCE MANUAL



Syntax, Semantics, Types, and Core Language Rules

Kameswara Rao Mithipati



Folang is a general-purpose programming language designed to be **expressive, consistent, and extensible**, merging functional fluency with object-centric abstractions.

Normative Status

This document is the authoritative and normative definition of the FoLang programming language.

All FoLang syntax, semantics, type-system rules, name-resolution rules, execution behavior, and other language requirements are defined by this document.

Every addition, correction, clarification, modification, or extension to the FoLang language must be recorded in this document before it becomes part of the language.

A compiler or other language implementation may use any internal architecture, parser, intermediate representation, runtime, or backend. However, to be identified as a conforming FoLang implementation, its externally observable behavior must adhere to the requirements defined in this document.

When an implementation conflicts with this specification, the specification governs unless the discrepancy is formally accepted and incorporated as a specification revision.

Implementation-specific extensions must be clearly identified as extensions and must not be represented as standard FoLang features unless they have been incorporated into this specification.

Audience and Scope

This document is the normative FoLang language and implementation reference. Its primary audience is frontend/compiler implementers, backend/code-generator implementers, tooling implementers, runtime/library implementers where language contracts apply, and conformance-test authors.

The document defines accepted source forms, semantic relationships, validation requirements, diagnostics, frontend obligations, backend-observable contracts, conformance requirements, and areas intentionally left implementation-defined. It is not a getting-started guide or a tutorial for learning FoLang. Examples are retained when they establish, distinguish, or test normative behavior; pedagogical progression and programmer-oriented introductory material are outside the purpose of this reference.

A backend or tool may use any internal representation or optimization permitted by this specification, but implementation techniques do not redefine FoLang syntax, type relationships, member relationships, or observable semantics.

Grammar, Semantics, and Examples

The normative lexical and syntactic requirements stated throughout this specification define the FoLang source forms accepted by the current language profile. Normative semantic rules define the meaning of those forms and any additional validity constraints that are checked after parsing.

Examples are **illustrative only**. The presence or absence of an example does not enable, disable, reserve, or otherwise modify a grammar production, token, declaration form, operator, or semantic rule. Likewise, an inventory or explanatory table does not create new source syntax unless this specification explicitly defines that syntax or identifies the spelling as an explicitly reserved future form.

For parser behavior and generation of derived grammar documentation, the governing classification is therefore deterministic:

```
active normative syntax and lexical requirements
```

```
-> accepted and parsed according to the applicable requirements
```

```
explicitly identified reserved/future form
```

```
-> recognized as language-owned where its reserved spelling is defined
```

```
-> rejected with an unsupported-feature diagnostic in the current  
profile
```

```
all other source text
```

```
-> ordinary lexical or syntax error when it does not match the active  
requirements
```

Language Evolution and Compatibility

The alpha period permits experimentation, consolidation, implementation, renaming, syntax changes, and removal. Before version 1.0, a proposed structural feature becomes part of the active language only when a specification revision explicitly incorporates it into the normative syntax requirements and defines its applicable normative semantics. A proposed or future feature that is mentioned only descriptively does not become active syntax.

Nothing is carried into version 1.0 as implicitly reserved or unsupported merely because it appeared in an example, table, inventory, or design discussion. A spelling is reserved while unsupported only when this specification explicitly identifies that spelling or form as reserved/future syntax.

At version 1.0, the **structural FoLang language surface** closes permanently. After version 1.0, no later major or minor release introduces new core grammar forms, hard/contextual keywords, declaration kinds, operator spellings or fixities, or built-in `@co.*` metadata-form names. Existing structural constructs retain the externally observable semantics defined by the 1.0 specification, except for corrections that restore already-stated intent.

The standard-package rule is narrower and separate. After version 1.0, the `co.*` **package/subpackage hierarchy** is frozen, while ordinary declarations inside those existing packages may evolve through later language-provided `.fo1enc` artifacts that expose the standard package contexts. Adding or updating an ordinary type, unit-level function, method, module, algorithm, data structure, or other declaration inside an existing package is package-API evolution and does not create new grammar or a new package path.

FoLang remains extensible after 1.0 through extension mechanisms already defined by the 1.0 language, including third-party libraries, user-defined annotations and decorators, macros, custom operators, native integration, including FFI, dynamic-runtime facilities, backend integration points, and other explicitly defined extension mechanisms. Frontend, backend, runtime, optimizer, intermediate representation, diagnostics, compilation strategy, memory management, code generation, performance, and hardware support may evolve provided externally observable FoLang semantics remain conforming.

A post-1.0 correction may fix an implementation/specification discrepancy or remove an internal contradiction when doing so restores the already-stated intent of an existing feature. A correction does not introduce a new grammar form, keyword, declaration kind, operator grammar, metadata-form name, or previously unavailable structural capability.

Lexical Profile and Statement Termination

The normative lexical and syntactic requirements are defined by this reference. The rules below state source-level constraints needed by parser/frontend implementations and conformance tests. Appendix A provides a derived EBNF aid; it is not an independent or authoritative definition of FoLang syntax.

Source Encoding and Identifiers

FoLang source text is UTF-8. A U+FEFF byte-order mark is permitted only as the first code point of a source file; U+FEFF anywhere else is an error.

Ordinary FoLang identifiers are ASCII-only. An identifier begins with an ASCII letter, may continue with ASCII letters or decimal digits, and may contain `_` only between two non-empty alphanumeric segments. An identifier cannot begin or end with `_`, cannot contain consecutive underscores, and cannot be the single spelling `_`. The single `_` is a contextual language token whose meanings are defined by the applicable wildcard/discard, filename-derived declaration, parameterized-type-placeholder, and refinement-predicate rules.

After its character sequence is recognized, an ordinary identifier is checked against the reserved-word table. Hard-reserved words are emitted as reserved tokens rather than identifiers.

Examples:

name	valid
myVar2	valid
v1_hr	valid
a_b_c	valid
123hr	invalid: identifier cannot begin with a digit
_x	invalid: identifier cannot begin with underscore

<code>_1</code>	invalid: identifier cannot begin with underscore
<code>a_</code>	invalid: trailing underscore
<code>a__b</code>	invalid: consecutive underscores
<code>-</code>	contextual token, not an ordinary identifier

Control labels use a separate apostrophe-prefixed lexical form such as `'outer'`. They are not ordinary identifiers. `'outer:'` declares a structured control label and `'outer'` references it; see [Labels and Named Blocks](#). A character literal remains distinct because it has a closing apostrophe, for example `'c'`.

Numeric Literals

FoLang supports the integer and floating literal families defined by the normative lexical requirements in this reference. Numeric digit separators are not part of the current profile, so forms such as `1'000`, `0x1'a`, and `0b1011'0010` are invalid.

A numeric sign is not part of an ordinary numeric literal token. In expressions, unary `+` or `-` is parsed as a prefix operator. Pattern syntax separately permits a leading `+` or `-` before an integer or floating literal.

Integer literals support binary, octal, decimal, and hexadecimal forms together with the suffixes admitted by the normative syntax requirements. Floating literals support decimal and hexadecimal forms. A radix-point form requires at least one digit on each side of the point: `1.0` and `0.10` are valid, while `1.` and `.10` are invalid. Scientific notation without a radix point, such as `1e5`, remains valid. A backend-conditional floating suffix is accepted only when the selected backend/compiler contract supports the corresponding representation.

Comments, Whitespace, and Line Breaks

FoLang supports `//` line comments and non-nesting `/* ... */` block comments. A line comment ends before the next CR or LF terminator. A block comment ends at the first following `*/`; an inner `/*` does not nest.

Comments are recognized before ordinary symbolic-run scanning. Spaces, horizontal tabs, form-feed characters, line breaks, and comments are token separators and are discarded between tokens. A line break never terminates a FoLang statement; FoLang has no automatic semicolon insertion.

Statement and Expression Termination

FoLang uses explicit termination:

simple statement or expression statement	-> terminated by ;
direct block/body	-> terminated by its closing }
braced expression/literal	-> } closes the expression,
then ; closes its statement	

A semicolon is required after simple declarations, assignments, compound assignments, calls used as statements, `$=>` callable-result statements, `$->>` iteration-advance statements, `$->|` structured-exit statements, object/collection construction expressions used in a statement, type-alias declarations containing generic instantiations, literal expression statements, forward declarations, and other simple declaration forms.

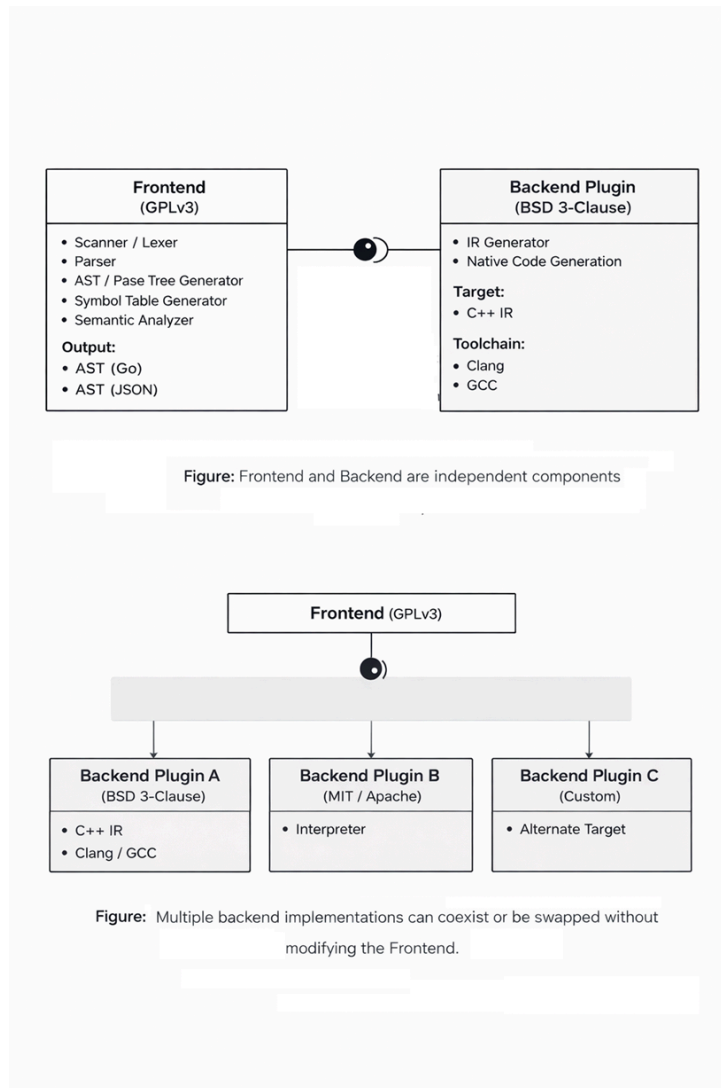
A direct declaration body or function/method body terminates at its closing `}` and must not be followed by `;`.

A braced **expression** is different from a direct block/body. Object construction and typed map/collection values still require the enclosing statement's semicolon:

```
emp := Employee{id: 1, name: "Rao"};
$=> Employee{id: 1};
StringIntMap co.type =
    co.Map(co.string, co.int);
cfg := StringIntMap{"a": 1, "b": 2};
```

Built-in directives, annotations, pragmas, and decorators are independently delimited metadata applications and do not acquire a trailing semicolon merely because they appear on their own source line.

Design Overview



FoLang follows a deliberately different approach from conventional programming language designs. The system is structured to ensure **clear separation of concerns**, **license isolation**, and **extensibility** through well-defined integration boundaries.

Compiler and Backend

1. Frontend

The Frontend is responsible for source-level analysis and semantic processing of FoLang programs.

Frontend Components

- Scanner / Lexer
- Parser

- AST / Parse Tree Generator
- Symbol Table Generator
- Installed Standard-Package Loader
- `.fo1enc` Artifact and Symbol Loader
- Semantic Analyzer

Implementation

- Implemented in **Go**
- Uses Go structures internally for AST, symbol-table, and semantic processing.
- The externally consumable frontend output is serialized according to the selected backend's interchange contract.
- The frontend writes that artifact beneath the reserved project-root `build/` domain.
- The backend-supplied contract identifies the supported FoLang/plugin protocol version, HIR schema version, and wire format.
- The backend installs/provides this interchange-contract file in the same installation directory as the FoLang compiler executable.
- The frontend reads that installed backend contract to determine the interchange representation required by the selected backend.
- Canonical `build/` presence, ownership, and source-discovery rules are defined in [Project Layout](#).

Named Diagnostic Registry

Named diagnostics use stable, case-sensitive identifiers shared by the compiler, IDE tooling, conformance tests, and serialized diagnostic output. Implementations may add explanatory text, source locations, and internal numeric identifiers, but they must preserve the registered name.

The registry groups related normative rejection rules under reusable names. The compiler emits the most specific applicable diagnostic and places the particular declaration kind, symbol, metadata form, type, relationship, or effect in its structured details. It must not create a different public name merely to restate the same rule for another declaration kind. Secondary errors caused solely by one primary failure should be suppressed or reported as related notes rather than

independent cascades. All entries currently have severity `Error` and prevent successful compilation. They are frontend diagnostics, not runtime `co.error` values or effects.

Diagnostic name	Severity	Phase	Normative use
<code>Unclassified</code>	Error	Diagnostic migration	A legacy diagnostic has not yet been assigned a more specific stable name
<code>UnsupportedFeature</code>	Error	Profile validation	Source uses a recognized feature or reserved future form that the active language profile does not support
<code>UnsupportedBackendFeature</code>	Error	Backend compatibility	Source requires a representation or facility not supported by the selected backend contract
<code>InvalidIdentifier</code>	Error	Lexing	An identifier violates FoLang spelling, underscore, reserved-word, or encoding rules
<code>InvalidLiteral</code>	Error	Lexing/parsing	A scalar literal token violates its normative lexical form
<code>UnexpectedToken</code>	Error	Parsing	A token cannot occur at the current grammar position
<code>ExpectedToken</code>	Error	Parsing	A required delimiter, binder, separator, operand, or other token is absent
<code>InvalidSyntax</code>	Error	Parsing	Source violates a grammar rule for which no more specific registered

			parser diagnostic applies
<code>UnknownOperator</code>	Error	Operator-aware scanning	A complete symbolic run is neither built in nor registered in the active operator table
<code>InvalidProjectLayout</code>	Error	Project validation	A required root, component tree, fixed surface, filename role, or project-kind layout rule is violated
<code>InvalidSourcePlacement</code>	Error	Source-structure validation	A file, declaration, unit, block form, directive, pragma, or component surface appears in a forbidden source context
<code>InvalidDeclarationForm</code>	Error	Declaration validation	A declaration has an unsupported head, nesting, body, filename-derived name, classifier combination, or structural shape
<code>ReservedPackageShadowing</code>	Error	Project validation	Project or dependency content attempts to shadow the installed <code>co.*</code> package identity
<code>InvalidImport</code>	Error	Import resolution	An import has an invalid form, target, alias, visibility, package/component/library role, or source context
<code>InvalidDependency</code>	Error	Dependency analysis	A dependency crosses a forbidden project, component,

			capability, or packaged-library direction
DependencyCycle	Error	Dependency analysis	Effective package, component-surface, or standalone-library dependencies form a cycle
CapabilityViolation	Error	Capability analysis	Source accesses or enables a facility outside its permitted application, native, dynamic-runtime, or other capability domain
ExportBoundaryViolation	Error	Boundary analysis	A public/projected/packaged surface exposes a forbidden, inaccessible, private, internal, native, or representation-reachable type
DuplicateDeclaration	Error	Symbol analysis	Two variables, types, members, aliases, fields, variants, metadata keys, or other non-callable declarations normalize to one symbol in the same namespace
DuplicateCallableSignature	Error	Symbol/overload analysis	Two callables normalize to the same owner, receiver category, name, and parameter signature, including declarations differing only by result type

UnresolvedSymbol	Error	Name resolution	A required package, type, value, callable, label, declaration reference, or other symbol cannot be resolved
AmbiguousSymbol	Error	Name resolution	More than one symbol remains after the applicable qualification and resolution rules
InaccessibleSymbol	Error	Access analysis	A resolved declaration exists but is not accessible from the requesting context
InvalidReceiver	Error	Receiver analysis	A value/type/companion/extension receiver is absent, has the wrong category, or does not match its required owner
UnusedImport	Error	Usage analysis	An import contributes no live resolved use under the strict import-liveness rules
UnusedSymbol	Error	Usage analysis	A declaration subject to strict liveness is not reachable or used as required by its declaration category
TypeMismatch	Error	Type checking	Actual and required canonical types, result positions, receiver types, or callable shapes are incompatible

<code>ConstraintViolation</code>	Error	Constraint checking	A value or type violates a refinement predicate, subtype/supertype set, bound, variance, capability, or other declared constraint
<code>InvalidAssignment</code>	Error	Type/assignment analysis	An assignment target, source, binding operator, mutability rule, or value-transfer relationship is invalid
<code>Uninitialized</code>	Error	Flow analysis	A refinement/dependent value is read where a valid assignment is not proven on every reachable path
<code>InvalidNoneUse</code>	Error	Type/flow analysis	<code>co.const.none</code> , <code>isNone()</code> , or missing-result none substitution is used with a refinement/dependent position that does not admit none
<code>InvalidDependentIndex</code>	Error	Dependent-type analysis	A dependent index has an invalid form, resolution, evaluability, sign, or permitted source
<code>DependentTypeMismatch</code>	Error	Dependent-type analysis	Dependent constructors or corresponding normalized indices do not match where equality is required
<code>InvalidConstruction</code>	Error	Construction analysis	A struct/class/object construction or initialization path

			cannot produce a valid readable value
<code>MutationNotAllowed</code>	Error	Mutability analysis	Source attempts a statically detectable mutation through an immutable, constant, literal, or otherwise non-mutable root
<code>OverloadNotAllowed</code>	Error	Callable validation	A declaration category or callable identity is not permitted to introduce the requested overload
<code>NoApplicableOverload</code>	Error	Overload resolution	No visible overload accepts the resolved receiver and argument types
<code>AmbiguousOverload</code>	Error	Overload resolution	Multiple incomparable most-specific overload candidates remain applicable
<code>InvalidReturn</code>	Error	Callable validation	A return count, type, or required result production is invalid
<code>MissingImplementation</code>	Error	Implementation analysis	A required callable body, interface method, abstract/virtual slot, runtime binding, or other implementation is absent
<code>InvalidForwardDeclaration</code>	Error	Declaration validation	A bodyless declaration has invalid classification, mapping, signature, or forward-declaration metadata

InvalidGenericDeclaration	Error	Generic analysis	Generic markers, rank, placement, declaration-head form, classifier combination, or parameter structure is invalid
InvalidGenericMapping	Error	Generic analysis	Generic mapping rows are absent where required, conflicting, cyclic, structurally incompatible, or leave a required marker unresolved
GenericResolutionFailure	Error	Generic analysis	Generic arguments or result markers cannot be resolved to one consistent permitted assignment
SignatureConformanceFailure	Error	Contract conformance	A module, instance, handler, interface implementation, or other declaration fails its required structural signature
InvalidAssociatedTypeBinding	Error	Contract conformance	An associated-type binding is missing, unknown, fixed incompatibly, or violates generic arity, bounds, variance, or referenced signature requirements
InvalidRelationship	Error	OOP validation	An interface/class/mixin/trait relationship has an invalid kind, target, count, selector,

			accessibility, or canonical identity
<code>InheritedMemberConflict</code>	Error	OOP validation	Inherited state or behavior conflicts and the normative merge/selection rules cannot resolve it automatically
<code>InvalidOverride</code>	Error	OOP validation	An override target, source category, accessibility, sealing rule, or exact signature does not satisfy override rules
<code>InvalidImplementation</code>	Error	OOP/contract validation	An explicit implementation mapping selects the wrong source, target, method, accessibility, or signature
<code>InvalidLifecycleDeclaration</code>	Error	Lifecycle validation	<code>@@new</code> , <code>@@init</code> , <code>::</code> invocation, lifecycle metadata, placement, accessibility, or overload/override use violates lifecycle rules
<code>InvalidMemberPolicy</code>	Error	Member-policy validation	A field or member carries a forbidden constant, immutable, shared, locking, <code>CopyOnWrite</code> , static/class, or related policy
<code>UnresolvedMetadataForm</code>	Error	Metadata resolution	A user-defined annotation/decorator name cannot be resolved to a valid metadata declaration

<code>InvalidMetadataPlacement</code>	Error	Metadata validation	An annotation, decorator, directive, or pragma appears in a source location where that metadata category is forbidden
<code>InvalidMetadataTarget</code>	Error	Metadata validation	A resolved metadata form does not permit the declaration or expression kind to which it is attached
<code>MissingMetadataField</code>	Error	Metadata validation	A required field of a recognized metadata form is absent
<code>InvalidMetadataField</code>	Error	Metadata validation	A metadata field is unknown, duplicated, uses an invalid binder, or is forbidden for the selected form/context
<code>InvalidMetadataValue</code>	Error	Metadata validation	A metadata value has the wrong type, cardinality, range, enumeration value, declaration category, or compile-time status
<code>ConflictingMetadata</code>	Error	Metadata/classification validation	Two or more individually valid metadata forms cannot legally classify or govern the same declaration together
<code>InvalidOperatorDeclaration</code>	Error	Operator registration	A custom operator declaration has invalid spelling, fixity, precedence, associativity, arity, attributes, ownership, or component placement

<code>InvalidOperatorOverload</code>	Error	Operator overload validation	An operator implementation violates operand ownership, receiver, signature, arity/fixity, genericity, accessibility, or result-independent identity rules
<code>InvalidEffectDeclaration</code>	Error	Effect analysis	<code>@co.dap.effects</code> , emitted error types, or effect metadata placement violates the effect declaration rules
<code>InvalidEffectPolicy</code>	Error	Effect analysis	A call-site <code>@co.dap.onEffect</code> record has an invalid effect key, structure, placement, or ordinary/execution-model policy shape
<code>InvalidEffectHandler</code>	Error	Effect analysis	A handler list is empty, duplicated, inaccessible, non-module, signature-incompatible, or illegally contains local <code>@co.dap.onEffect</code> handling
<code>InvalidEffectResolution</code>	Error	Effect analysis	A resolution is missing, duplicated, unknown, contextually forbidden, or incompatible with the enclosing callable results
<code>InvalidRetryPolicy</code>	Error	Effect analysis	Retry configuration has an invalid attempt

			count, exhaustion action, placement, or structure
<code>InvalidExecutionModel</code>	Error	Execution-model validation	An execution-model declaration or invocation violates its result, effects, submission, boundary, or call-site handling rules

The registry name is the stable machine-readable identifier. A diagnostic instance additionally carries at least its severity, compiler phase, human message, primary source span, zero or more related spans, and structured details. Parser recovery may emit `UnexpectedToken`, `ExpectedToken`, or the fallback `InvalidSyntax`; once a valid AST construct exists, semantic analysis should prefer its more specific registered diagnostic.

Examples of the stable name plus specific message are:

```
error[DuplicateDeclaration]: variable 'count' is already declared in this
scope
error[OverloadNotAllowed]: this callable category does not permit
overloading
error[InvalidMetadataPlacement]: @co.ddap.import is valid only in a file
preamble
```

License

- GNU General Public License v3 (GPLv3)

2. Backend

The Backend is responsible for transforming validated frontend output into executable artifacts.

The default backend must be downloaded or built separately. It is not bundled with the frontend binary.

Backend Components

- Intermediate Representation (IR) Generator
- Native Binary Executable Generation

Implementation

A backend may be implemented in any language. It consumes the validated frontend artifact from the project-root `build/` directory. The artifact encoding is selected by the backend-supplied interchange contract.

During backend installation, that interchange-contract file is placed in the same installation directory as the FoLang compiler executable. The frontend reads the installed contract to determine the compatible FoLang/plugin protocol version, HIR schema version, and wire format required by the selected backend. The exact artifact basename and format-specific schema/version may therefore vary by backend contract, but the artifact location is always beneath `<project-root>/build/`.

Runtime-operation handlers

Backend-neutral HIR may contain a resolved runtime-operation identifier originating from an exported standard declaration marked with `@co.dap.implementation`. The identifier states the required semantic operation, not its target-language spelling or implementation. Each backend owns an internal mapping from supported `co.runtime.operation.*` identifiers to backend handlers.

```
co.runtime.operation.out.println
  -> reference C++ HIR backend handler
  -> generated call to the reference runtime implementation
```

The reference C++ HIR backend may generate a call to a function declared in a separately maintained C++ header and implemented in a separately compiled C++ source/runtime library. Other backends may lower the same operation to different runtime calls, VM instructions, imports, or target-specific code. Backend headers, implementation filenames, source fragments, and native symbol spellings do not appear in the backend-independent FoLang declaration.

The backend interchange contract must identify the runtime-operation contract version or otherwise provide an equivalent compatibility check. Before code generation, the selected backend must reject any required operation for which it has no compatible handler. A backend handler is selected by the resolved operation ID carried in HIR, not by appending arbitrary target-language text obtained from source annotations.

Default / Reference Backend

The default FoLang backend is also the **reference backend implementation**. Its purpose is to provide an executable, inspectable example of how the FoLang specification can be implemented and to give backend implementers a concrete behavioral baseline for conformance testing.

The written FoLang specification remains normative. The reference backend demonstrates the required externally observable semantics, but its internal algorithms, allocation strategy, memory-management choices, data structures, optimization level, and performance characteristics are not themselves language requirements unless this specification explicitly says otherwise. If an implementation defect in the reference backend conflicts with the written specification, the written specification takes precedence.

A third-party backend may use a completely different runtime architecture or memory model and may optimize semantics differently, provided that FoLang programs observe behavior conforming to this specification. Backend implementations may validate their behavior against the reference backend and the FoLang conformance tests where applicable.

- Backend orchestration is implemented in **Go**
- Code generation target is **C++**
- Uses **Clang** or **GCC** to generate native binaries from generated C++ IR

License

Third-party backends may use their own licensing terms and implementation choices. The default backend uses the following license. **Default backend is not part of the complete compiler binary and is separate;** it must be downloaded or built separately.

- BSD 3-Clause License ***

Frontend Output Contract

Installed Backend Interchange Contract

The selected backend may supply this contract to tell the frontend which FoLang/plugin protocol, HIR schema, and wire format it accepts. During backend installation, the contract file is placed in the same installation directory as the FoLang compiler executable, and the frontend reads it from that installation location. Every frontend build also embeds a complete default backend contract. If `backend-conf.json` is absent, the frontend uses protobuf and that frontend release's embedded current protocol, HIR-schema, and runtime-operations versions; these values are fixed for that binary, not resolved dynamically. A present contract replaces those defaults and must be complete, well-formed, and supported, or compilation fails.

```
{
  "protocol":          "folang-plugin/1.0",
  "hir_schema":        "folang-hir/1",
  "wire":              "protobuf",
  "runtime_operations": "folang-runtime-operations/1"
}
```

The frontend has one fixed **location** contract and a backend-selected **encoding** contract:

```
<project-root>/build/
  └─ <frontend-artifact>    encoding/version selected by backend
contract
```

Rules:

- the frontend/backend interchange artifact is written beneath the reserved root-level `build/` domain;
- each frontend build embeds default protocol, HIR-schema, wire-format, and runtime-operations versions; an absent installed contract selects those fixed defaults, while a present contract must supply every field and be well-formed and supported;

- that contract file resides in the same installation directory as the FoLang compiler executable;
- `wire="protobuf"` in the example above requests Protocol Buffers; the current protobuf representation is a provisional, language-neutral `Value / Struct / List` artifact tree carrying the logical `FoLang-hir/1` model, not yet a typed protobuf definition of every HIR node and interface, and a typed schema with node messages and interface `oneof` declarations is required before treating it as the stable third-party-backend schema; another supported contract may request a different compatible wire format/version;
- `runtime_operations` identifies the backend-neutral runtime-operation contract understood by the selected backend;
- backends consume the validated frontend artifact from `build/`;
- canonical filesystem rules for `build/` are defined only in [Project Layout](#).

Licensing Summary

Layer	Responsibility	Implementation	License
Frontend	Parsing and semantic analysis	Go	GPLv3
Backend (default)	IR processing and native code generation	Go (orchestration) + C++ (target)	BSD 3-Clause

The copyrightable material in the [FoLang Language Definition and Documentation](#), including its syntax, grammar, and semantic-rule descriptions, is licensed separately under [CC BY 4.0](#). ***

3. Capability Security Model

FoLang’s compiler ships with all language features compiled in but **native capabilities are disabled by default**. The compiler has no hardcoded keys — capability configuration happens entirely at install time. This moves authorization from source code (developer-controlled) to the compiler installation (organization-controlled).

Capability Domains and Install-Time Gates

Domain	Features	Default State
<code>application</code>	Ordinary FoLang language/application capabilities, including macros, templates, concurrency/control abstractions, <code>co.http</code> , core declarations such as <code>co.List</code> , <code>co.encoding</code> , <code>co.crypto</code> , etc.	✓ enabled
<code>dynamicvmrt</code>	application capabilities plus the defined dynamic-runtime / <code>co.meta</code> facilities	explicit source capability domain; toolchain policy may restrict availability
<code>native</code>	Raw pointers/references/addresses, pointer arithmetic, <code>@co.dap.native</code> , <code>co.native</code> , <code>co.sys.unsafe</code> , MMIO, heap allocators, low-level platform/runtime implementation, <code>co.sys.ffi</code> , extern declarations/types, foreign symbols, calling conventions, linkage, C/native ABI work, and ABI-compatible <code>cstruct</code> values	🔒 disabled by default — requires install-time configuration

`packaged` is an exposure model using application capabilities, not a separate privileged capability domain.

Variable Syntax

A FoLang variable declaration places the identifier first and the declared type second:

```
name co.string = "SomeName";
count co.int;
```

The general source shape is:

```
identifier type [= initializer] ;
```

`identifier` must satisfy the FoLang identifier rules. `type` may be a built-in type, a user-defined type, or a named specialization/derived type. An initializer, when present, must be assignable to the declared type.

Types are fundamental to every programming language. A type defines the kind of value a program can represent and the operations that are valid for that value.

Variables and other bindings associate names with values, and each value participates in the type system according to its declared, inferred, or runtime type.

FoLang likewise has a rich type system, described in the following sections.

Types

FoLang types are grouped into four broad families:

1. built-in data types;
2. built-in collection/protocol types;
3. user-defined types;
4. derived and specialized types.

These categories organize the standard surface; they do not introduce separate runtime type systems.

Built-in Types

Type	Purpose
<code>co.string</code>	Built-in string type.
<code>co.int</code>	Built-in integer type.
<code>co.bit</code>	Built-in bit type.
<code>co.double</code>	Built-in double-precision floating type.

<code>co.float</code>	Built-in floating type.
<code>co.long</code>	Built-in long integer type.
<code>co.byte</code>	Built-in byte type.
<code>co.char</code>	Built-in character type.
<code>co.any</code>	General language-provided value supertype where permitted.
<code>co.bool</code>	Built-in Boolean type.
<code>co.void</code>	Built-in no-value/void type where the applicable construct permits it.
<code>co.value</code>	Language/runtime value representation used by value/snapshot-oriented operations.
<code>co.untyped</code>	Explicitly untyped value representation used by facilities such as macros and custom matchers.
<code>co.word</code>	Built-in machine-word-oriented type family root.
<code>co.MatchBindings</code>	Binding container returned by custom matcher protocol implementations.
<code>co.number</code>	Built-in numeric type/category used by numeric constraints and APIs.
<code>co.uninit</code>	Lifecycle-only uninitialized-object state used by class construction.
<code>co.error</code>	Standard recoverable-error interface and first-error result contract. Every emitted effect object is a class instance satisfying this interface.
<code>co.AbstractError</code>	Standard mixin supplying common <code>co.error</code> state and behavior.
<code>co.literal</code>	Language/runtime representation of literal objects.
<code>co.operator</code>	Operator-declaration type/kind valid only in the project operator-component context.
<code>co.delegate</code>	Delegate declaration/type facility for one or more registered callable targets.

<code>co.condition</code>	Language condition object used by condition, loop, and value-selection machinery; it is not merely an alias for <code>co.bool</code> .
---------------------------	--

Built-in Collection and Protocol Types

Name	Purpose
<code>co.List</code>	Ordered list collection.
<code>co.Set</code>	Set collection.
<code>co.Map</code>	Key/value collection.
<code>co.Tree</code>	Tree collection.
<code>co.Trie</code>	Trie collection.
<code>co.Array</code>	Array collection abstraction.
<code>co.Tuple</code>	Tuple value type.
<code>co.Comparable</code>	Standard comparison protocol/type.
<code>co.Stack</code>	Stack collection.
<code>co.Queue</code>	Queue collection.
<code>co.StructObject</code>	Runtime/type descriptor surface for struct objects where provided by the standard package.
<code>co.ClassObject</code>	Runtime/type descriptor surface for class objects where provided by the standard package.
<code>co.ModuleObject</code>	Runtime/type descriptor surface for module objects where provided by the standard package.
<code>co.InstanceObject</code>	Runtime/type descriptor surface for instance objects where provided by the standard package.
<code>co.ObjectObject</code>	Runtime/type descriptor surface for <code>co.object</code> objects where provided by the standard package.
<code>co.Matrix</code>	Matrix collection/type abstraction.

User-Defined Types

User-defined types are introduced through:

1. built-in declaration kinds such as `co.struct`, `co.class`, `co.interface`, `co.module`, and related kinds; or
2. named `co.type` declarations and the type constructors/specializations described below.

A source declaration is not considered a distinct user-defined type merely because it introduces a variable or value.

Derived Types

FoLang provides named derived types built from existing types. The current families include:

1. pointers;
 - ordinary pointers;
 - fat pointers;
2. arrays;
 - single-dimensional;
 - multidimensional;
 - jagged;
 - zero-dimensional;
 - zero-length;
 - variable-length;
 - initializer-sized forms where defined;
3. thunks;
4. slices;
5. ranges;
6. references;
 - ordinary references;
 - lvalue references;
 - heap references;

- 7. addresses;
- 8. word-representation specializations.

Derived types are named through `co.type` declarations before ordinary variable use.

Built-in Kinds

Kind	Purpose
<code>co.struct</code>	Value-oriented aggregate declaration with fields and associated behavior under the struct rules.
<code>co.cstruct</code>	ABI-oriented value aggregate whose layout and boundary rules are defined by the native/ABI contract.
<code>co.class</code>	Concrete reference-semantic object type supporting instance state, inheritance/composition, encapsulation, overriding, lifecycle operations, and dynamic dispatch.
<code>co.interface</code>	Pure behavioral contract; interface methods are abstract requirements.
<code>co.union</code>	Union declaration kind.
<code>co.object</code>	One named singleton object that may own constants, immutable/global/shared state, locks, annotations, or support state associated with other declarations.
<code>co.instance</code>	Typeclass/contract instance declaration kind.
<code>co.matcher</code>	Custom matcher declaration kind used by the pattern-matching protocol.
<code>co.loader</code>	Loader declaration kind where the applicable runtime/package rules permit it.
<code>co.trait</code>	Reusable abstract composition contract that may provide concrete/default behavior and abstract/virtual requirements according to trait rules.

<code>co.mixin</code>	Reusable abstract composition unit that may provide state/behavior and abstract/virtual requirements according to mixin rules.
<code>co.extension</code>	Reusable implemented behavior composed with supported target types without class inheritance.
<code>co.typeclass</code>	Typeclass contract declaration kind.
<code>co.module</code>	Named singleton module satisfying its declared module/signature contracts.
<code>co.unit</code>	Stateless source grouping. Ordinary units merge into the package namespace; <code>*.comp.unit.fol</code> contributes associated behavior to its target struct. A unit does not own a source-visible context object.
<code>co.block</code>	Language block kind where block-kind values/AST classification are required.
<code>co.kind</code>	Kind-level declaration/value facility, primarily for metaprogramming and AST/kind classification.
<code>co.signature</code>	Structural module/signature contract declaration kind.
<code>co.function</code>	Function declaration kind used by the callable model.
<code>co.callable</code>	callable declaration kind used by the callable model.
<code>co.boundcallable</code>	callable produced after <code>bind</code> with parameter values on function.
<code>co.enum</code>	Closed tagged ADT declaration. Parameterized states are compiler-provided state functions returning the enclosing enum type; zero-parameter states are state values.
<code>co.symbol</code>	Language/compiler symbol representation used by AST/compiler-facing facilities.
<code>co.component</code>	Structural component/surface declaration valid in component surface files and standardized component locations.

Type Specializations and Type Constructors

Type form	Purpose
<code>co.variants(...)</code>	Defines a closed parameterized variant family. Entries may reference declaration type parameters.
<code>co.tag(...)</code>	Runtime type/value package used where a value must carry its concrete runtime type descriptor.
<code>co.hokrlt</code>	Type-valued holder used where a type itself is carried as a value/object.
<code>co.newtype</code>	Introduces a type identity distinct from its representation/base type.
<code>co.opaquetype</code>	Introduces an opaque wrapper identity with the one-way representation/assignment rules defined below.
<code>co.subtype</code>	Type-level set/range selecting strict subtypes of the referenced type.
<code>co.supertype</code>	Type-level set/range selecting strict supertypes of the referenced type.
<code>co.dependentType(...)</code>	RHS type-expression constructor used by a <code>co.type</code> declaration to define a value-indexed dependent type family. It is not a declaration kind or callable result kind.
<code>co.polymorphic(...)</code>	RHS type-expression constructor defining a named polymorphic type from a binder set and a type body.
<code>co.refinementType</code>	Base type restricted by a predicate over candidate values.
<code>co.associatedType</code>	Associated type placeholder/binding used by signature/module contracts.
<code>co.predicateType</code>	Type-level predicate/filter over candidate types.
<code>co.data(...)</code>	Concrete closed ADT constructor whose payload types are already concrete/resolved. It is not parameterized by declaration-head type parameters.

<code>co.type</code>	General named type declaration kind.
<code>co.generic</code>	Reserved/future generic-type constructor; it is not part of the active generic declaration model unless this reference explicitly enables it.
<code>co.shape</code>	Shape/type-expression facility for callable/type shapes where defined by the syntax and semantic rules in this reference.

`co.variants(...)`, `co.data(...)`, `co.dependentType(...)`, and `co.polymorphic(...)` are not interchangeable:

<code>co.variants(...)</code>	parameterized closed variant family
<code>co.data(...)</code>	concrete closed ADT
<code>co.dependentType(...)</code>	value-indexed dependent type family
<code>co.polymorphic(...)</code>	explicitly bound polymorphic type

FoLang Reserved Words and Context Sigils

The hard-reserved words are:

```
co
this
fΦλ
```

`$` is the compiler-owned current-context sigil. It is not an identifier and cannot be declared or redefined by source.

`fΦλ` (`f` = U+0066, `Φ` = U+03A6, `λ` = U+03BB) is the permanently reserved private standard-package root used by privileged standard-library/bootstrap source. Ordinary project source consumes the projected public `co.*` surface instead.

`this` denotes the nearest applicable non-callable object context according to the context rules.

FoLang Operators and Language Tokens

Arithmetic Operators

+ - * / % **

Logical Operators

&& || !

Bitwise Operators

& | ^

Comparison and Type-Relation Operators

== != < > <= >=

Assignment and Declaration Operators

= ?= := ::= += -= *= /= %= **=

Range, Function, Dispatch, and Structural Tokens

.. ... <.. ..< <..<
=>> => ->
<- ->> ->|
:: @@
@ # ~ \$
() { } []
_ ` ?
: ; , .
" ' \
_

The exact lexical/syntactic role of each spelling is defined by the normative syntax requirements and corresponding semantic sections in this reference.

`$=>`, `$->>`, and `$->|` are contextual control productions formed from the `$` context sigil and the applicable control token. The former `this =>`, `this ^=>`, `this ->>`, and `this ->|` control spellings are not part of the current model.

Pre-declared Operator Glyphs

U n

Project-specific custom operators are declared only through the operator-component mechanism.

Built-In Operator Parse Table

Larger precedence numbers bind more tightly. Precedence and associativity determine the parse tree; FoLang's left-to-right operand evaluation rule independently determines evaluation order within that tree.

Precedence	Operator/form	Fixity	Associativity	Arity / parse role
700	call (...), index [...], member ., lifecycle call ::name(...), postfix !	postfix	left	call/index/member/lifecycle-call syntax; postfix ! unary
650	**	infix	right	binary
600	+, -, !	prefix	right	unary
550	*, /, %	infix	left	binary
500	U, n	infix	left	binary
450	+, -	infix	left	binary
400	.., <.., .. <, <.. <..<	infix/range	none	range form; a bound may be omitted where the range syntax permits
350	<, <=, >, >=, :>, <:	infix	none	binary
300	==, !=	infix	none	binary
250	&	infix	left	binary
200	^	infix	left	binary
150		infix	left	binary

100	&&	infix	left	binary; short-circuit
50		infix	left	binary; short-circuit
10	=, +=, -=, *=, /=, %=, **=, &=, ^=, =	infix assignment	right	binary assignment

The definition spellings `:=`, `::=`, and `?=` are statement-level definition operators, not general expression operators, so they do not receive an expression-precedence level. Structural spellings such as `::`, `=>`, `=>>`, `->`, `<-`, `->>`, and `<->` are likewise not ordinary expression operators merely because they contain symbol characters.

Standard Operator Examples

```

left  co.int = 10;
right co.int = 3;

notEqual co.bool = left != right;

bitsAnd co.int = 6 & 3;
bitsOr  co.int = 6 | 3;
bitsXor co.int = 6 ^ 3;

mulAssign co.int = 6;
mulAssign *= 3;

divAssign co.int = 18;
divAssign /= 3;

modAssign co.int = 17;
modAssign %= 5;

powAssign co.int = 2;
powAssign **= 3;

andAssign co.int = 6;
andAssign &= 3;

```

```
xorAssign co.int = 6;  
xorAssign ^= 3;  
  
orAssign co.int = 6;  
orAssign |= 3;
```

Large-Scale Applications, Libraries, and Components

FoLang supports both small single-source applications and large projects split across packages, libraries, and source components.

The principal organization forms are:

1. application source packages;
2. standalone libraries (`.folenc`);
3. source components;
4. the installed standard library projected as `co.*`.

These forms affect source placement, dependency direction, capability boundaries, and API exposure. They do not define separate language semantics.

Application Packages

Application packages are directories beneath:

```
<project-root>/src/
```

For example:

```
payroll/  
└─ src/  
    └─ app1.fo1  
        └─ hr/  
            └─ emp/
```

`hr` is a top-level application package and `hr.emp` is a subpackage. Package identity follows the directory/package rules; no additional package declaration syntax is required merely to create the directory hierarchy.

`src/app1.fo1` remains the application entry source.

Packages have following access specifiers

1. @co.dap.local
2. @co.dap.public (is default)

Libraries

A standalone library has source/package organization analogous to an application but exposes its public surface through `src/component.fol` rather than `src/app1.fol`.

The surface file defines what clients may consume. Internal library packages remain hidden unless projected by the library surface rules.

FoLang capability domains relevant to library/project boundaries are:

- `application`;
- `dynamicvmrt`;
- `native`.

`packaged` is an exposure/projection model, not a separate privileged capability domain.

A built standalone library is represented by a `.folenc` artifact and is normally placed beneath:

```
<project-root>/lib/
```

for consumption by a project.

Packaged Exports

Packaged export surfaces project selected internal APIs directly through the export/projection mechanism rather than through the ordinary API-surface model.

The FoLang standard library uses controlled projection from its private canonical namespace into the public `co.*` namespace.

Components

Components provide library-like isolation while remaining in source form inside the project rather than being prebuilt `.folenc` libraries.

Components live beneath:

```
<project-root>/components/
```

Their exact standardized subdirectories and surface files are defined by the project-layout requirements in this reference.

Component rules include:

1. a standardized component kind has its designated project location;
2. component exports are checked for use/liveness according to the component rules;
3. peer components do not directly import/reference each other; interaction is mediated through the application/package/library surfaces permitted by the dependency model.

Operator Components

Operator components are project-specific components used to declare custom operators.

They are not general-purpose libraries and are not exported as ordinary reusable component/library APIs. Operator declarations are valid only in the dedicated operator-component source context.

Importing Packages, Components, and Libraries

FoLang uses the built-in directive:

```
@co.ddap.import(...)
```

to import application packages, libraries, and components.

Importing a Package

```
@co.ddap.import(package="hr.employee", as="emp")
```

When `as=` is present, source accesses the imported package through that alias:

```
emp.EmployeeService.find(1001);
```

When `as=` is omitted, the complete imported package path is used. Omission of `as=` does not implicitly invent a short alias.

Importing a Library

```
@co.ddap.import(library="hrlib", as="hr")
```

`hrlib` resolves to the corresponding library artifact, such as:

```
<project-root>/lib/hrlib.folenc
```

subject to the project/library resolution rules.

Importing a Component

```
@co.ddap.import(component="native", as="native")
```

A component import resolves the project component surface identified by the component kind/name defined by the project-layout rules.

Symbol Lookup in FoLang

Unqualified lexical lookup follows the active symbol-table scope first and then crosses named-definition context boundaries only after the current lexical scopes are exhausted.

Conceptually:

1. Search the `SymbolTable` active at the use site.
2. Follow same-scope declaration-order `ParentId` links as required.
3. Follow `LexicalParentId` to enclosing lexical scopes and repeat.
4. When the current `Context`'s lexical scopes are exhausted:
 - a. move to `Context.ParentId`;
 - b. resume from `Context.ParentCtxSymbolTableId`,
the exact enclosing visibility point at which the child `Context` branched.
5. Repeat until the permitted lexical/context chain is exhausted.
6. Apply imports/aliases and any other lookup domain allowed by the current `ResolutionPolicy`.
7. If no valid declaration resolves, report `UnresolvedSymbol`.

Qualified lookup does not perform arbitrary fallback:

- a package/library/component qualifier resolves within that imported/projected context;
- another explicit context qualifier resolves within that context according to its access rules;
- failure to find a valid member in the selected context is an error.

See [Contexts, Symbol Tables, and Symbols](#).

Types in Detail

Most ordinary built-in scalar and collection types behave as their declared standard-package contracts specify. The following types need additional language-level explanation.

HOKRLT Type

```
x co.hokrlt = co.int;
```

`co.hokrlt` carries a type as a value/object in contexts that require type-valued data.

It is not the declaration kind used to define a new source type; named source types continue to use `co.type` and the applicable type constructors.

Value Type

`co.value` is the language/runtime value representation used by value/snapshot-oriented facilities where data is carried independently of its original object identity.

It should not be treated as a synonym for every ordinary source type.

Literal Type

`co.literal` represents literal values/objects in compiler/runtime facilities that need the literal as data rather than merely its evaluated source value.

Uninitialized Type

`co.uninit` is used by the class lifecycle model. Allocation/lifecycle operations may temporarily produce an uninitialized class object before a successful `@@init` establishes a readable instance.

It is not the universal representation of an ordinary uninitialized local variable; ordinary none/uninitialized rules are defined separately.

Untyped Type

`co.untyped` explicitly denotes data for which the ordinary static value type is not carried by the source-level contract.

Its principal uses include metaprogramming and the custom matcher protocol, where a pattern or AST-like value may be inspected dynamically.

`co.MatchBindings`

`co.MatchBindings` is the binding container returned by the custom matcher protocol.

Conceptually it contains zero or more named match bindings produced by a successful custom match. A binding name is unique within one result. Bound values may have unrelated concrete types, so implementations may internally represent each entry with its value together with the value's concrete type descriptor.

Bindings become visible only for the selected successful match case. The exact construction/manipulation API is provided by the standard package and matcher contract.

Condition Type

`co.condition` is the language condition object used by condition, loop, and value-selection machinery.

It is richer than treating every control construct as syntactic sugar around a raw `co.bool`; the standard condition operations define the applicable `then`, `when`, `default`, and `loop` behavior.

Operator Type

`co.operator` is the declaration facility for project-defined operators.

Its use is restricted to the dedicated operator component. A `co.operator` declaration in any other source context is rejected.

```
⊗ co.operator = {
  fixity = co.operator.fixity.infix,
  precedence = 60,
  associativity = co.operator.associativity.left,
  arity = co.operator.arity.binary,
  commutative = co.const.false,
  idempotent = co.const.false,
  identity = co.const.none,
  foldable = co.const.false,
  vectorizable = co.const.false,
  distributes_over = [],
  desugar = "intrinsic:tensor_product"
};

+- co.operator = {
  fixity = co.operator.fixity.infix,
  precedence = 60,
  associativity = co.operator.associativity.left,
  arity = co.operator.arity.binary
};
```

Error and AbstractError

`co.error` is the standard recoverable-error interface used by FoLang effects.

`co.AbstractError` is the standard reusable mixin that provides common `co.error` state/behavior. User-defined recoverable error classes normally satisfy `co.error` and may compose `co.AbstractError`.

Type Specializations in Detail

Type Alias

```
SomeInt co.type = co.int;

EmpId co.type = co.string;
DeptId co.type = co.string;
```

A `co.type` alias denotes the same underlying type identity when the declaration is an alias form.

Aliases are useful for shortening long qualified type names and for giving domain-significant names without introducing a new type identity.

```
empId EmpId = "E-100";  
deptId DeptId = empId;      // valid: same aliased underlying type  
  
someString co.string = deptId; // valid
```

An alias is therefore not a protection against accidental interchange. Use a distinct/opaque type form when that distinction is required.

Opaque Types

```
EmpId co.opaquetype = co.int;  
DeptId co.opaquetype = co.int;  
  
someInt co.int = 20;  
empId EmpId = 10;  
deptId DeptId;  
  
deptId = empId;    // invalid: different opaque identities  
deptId = 20;       // valid from permitted representation/base input  
deptId = someInt;  // valid where the opaque-type input rule permits the  
base type  
someInt = deptId;  // invalid: opaque value does not implicitly expose its  
base type
```

An opaque type has its own identity while permitting only the explicitly defined representation/base-direction conversions. Two opaque types with the same representation are not interchangeable.

Opaque types are useful when source should accept validated/approved base values but must prevent accidental interchange of domain identities.

New Types

```
SomeType co.newtype = co.int;  
  
x SomeType;  
y co.int = 20;
```

```
x = y; // invalid without an explicitly permitted conversion
y = x; // invalid without an explicitly permitted conversion
```

`co.newtype` creates a type identity distinct from its base/representation type. Ordinary assignment does not treat the new type and base type as interchangeable.

Type Unions

```
NumberOrText co.type = co.int | co.string;
```

This is a type-level union/predicate-style type expression: the resulting type position admits one of the selected types.

It is **not** by itself a value-level tagged ADT and does not manufacture variant tags. Closed value alternatives with explicit states use `co.variants(...)`, `co.data(...)`, or an enum as applicable.

Supertypes

```
EmployeeSupers co.supertype = some.ContractEmployee;
```

`co.supertype` denotes the strict supertypes of the referenced type according to FoLang's type-relation rules. The referenced base type itself is excluded unless it is explicitly reintroduced through another type expression.

Subtypes

```
EmployeeSubs co.subtype = some.Employee;
```

`co.subtype` denotes the strict subtypes of the referenced type. The referenced base type itself is excluded unless explicitly reintroduced.

To admit both the base and the strict relation set, combine them explicitly:

```
EmployeeOrSubtype co.type = EmployeeSubs | some.Employee;
```

Refinement Types

```
Percentage co.refinementType =  
    (co.int).where(_ >= 0 && _ <= 100);  
  
p Percentage = 200; // compile-time rejection when the value is statically  
known
```

A refinement type restricts the values admitted by a base type using a Boolean predicate over the candidate value `_`.

Refinement-type storage is non-`none` and follows the definite-initialization rules defined for refinement/dependent values.

Dependent Types

```
Vector(n) co.type =  
    co.dependentType(  
        co.int->([n])  
    );  
  
v3 Vector(3) = Vector(3){1, 2, 3};  
v4 Vector(4) = Vector(4){1, 2, 3, 4};
```

A dependent type includes value indices in type identity. Therefore:

```
Vector(3) != Vector(4)
```

even though both families ultimately contain `co.int` elements.

A refinement type restricts which **values** of a base type are valid. A dependent type makes one or more compile-time/index values part of the **type identity** itself.

Path-dependent result types may refer to a value/type path when the applicable syntax and type-resolution rules in this reference permit it:

```
PathIdentity co.type =  
    (x co.int)->(x.type);  
  
identity PathIdentity(x) = x;  
  
m co.int = 10;  
n co.int = 20;
```

```

p := identity(m);
q := identity(n);

type of p = m.type
type of q = n.type

m.type != n.type

```

Compile time type evaluation

```
someIntVar co.int ; someVar co.hokrlt.type.decltype(someIntVar) = 200; ####
```

Predicate Types

```

SomeType co.predicateType =
  co.type.where(
    candidate =>
      candidate == co.int ||
      candidate == co.string
  );

```

A predicate type filters candidate **types**, not candidate runtime values.

Predicate types are especially useful in generic bounds and other type positions that need semantic constraints richer than one nominal base type. They are not limited exclusively to generics.

Polymorphic Types

```

SomeFArg co.type =
  co.polymorphic({T}, (T, T)->(T));

```

`T` is a type binder owned by the polymorphic type declaration. Each valid instantiation binds `T` to one type for that instantiation; the binder is not supplied by an unrelated `co.generic` declaration.

When the polymorphic body is callable, an ordinary named function may implement that named callable contract directly:

```
identity SomeFArg(value, other) = value;
```

subject to the callable shape and generic-resolution rules.

Parameterized Variant Types

```
Option(T) co.type =  
  co.variants(  
    Some(T),  
    None  
  );
```

`T` is a **type parameter** of the type family.

`Some(T)` is a parameterized state whose payload type depends on `T`; `None` is a zero-payload state. Concrete applications such as `Option(co.int)` are ordinary type applications.

Terminology note: FoLang calls this a **parameterized type**. In type-theory literature, the same `Type -> Type` behavior is often described as a *type constructor*. FoLang does not introduce a separate constructor declaration category for it.

Associated Types

In a signature:

```
T co.associatedType;
```

and in a module satisfying that signature:

```
T co.associatedType = co.int;
```

An associated type is a type member required by a signature and supplied by the implementing/satisfying module.

Associated types provide type-level variation for module/signature contracts without making the module itself a generic declaration.

Data Types

```
SelectedValue co.type =  
  co.data(  
    StringValue(co.string),
```

```
    BoolValue(co.bool)
);
```

`co.data(...)` defines a **concrete closed ADT**. Its state payload types are already concrete/resolved at the declaration site.

Parameterized closed families belong to `co.variants(...)`; a declaration-head parameterized `co.data(...)` form is not the intended model.

Tag Values

```
co.tag(co.string, "Hello")
```

`co.tag` packages a runtime value together with the concrete type descriptor associated with that value.

It is an open runtime tagging mechanism, not a closed ADT declaration facility. It is useful where heterogeneous runtime values must retain their concrete type information, including matcher/runtime-introspection scenarios.

Kind Types

```
blockOrMacro co.kind = block | macro;
```

`co.kind` represents alternatives among language/compiler declaration/AST kinds rather than ordinary user value types.

It is primarily useful in macros and other metaprogramming facilities that inspect or transform program structure.

Generic Type Constructor

```
SomeType co.type = co.generic(T);
```

`co.generic(...)` is reserved/future syntax unless the active language profile explicitly enables it.

Current FoLang generic declarations use the generic declaration/binder mechanisms defined by the generic rules rather than relying on this constructor.

Function Types

```
IntBinary co.type =  
  (co.int, co.int)->(co.int);
```

A callable type may be used for variables, parameters, and results that hold/reference named function objects:

```
add IntBinary(a, b) = a + b;  
  
operation IntBinary = add;  
result := operation(10, 20);
```

FoLang does not require or introduce a general inline function-expression syntax for parameter/result positions. Named functions are first-class callable objects; restricted lambdas remain the separate callback-expression mechanism where explicitly allowed.

Delegate Types

```
SomeDelegate co.delegate =  
  (a co.int, b co.int)->(co.int, co.int);
```

A delegate is a callable registration/invoke facility rather than merely an alias for a function type.

```
someDelegate = myFunc;  
someDelegate(10, 20);  
  
someDelegate += mySecondFun;  
someDelegate(10, 20);
```

The exact invocation/result combination rules are defined by the delegate contract. Delegates are distinct from ordinary function chaining (`=>>`).

Derived Types

```
IntRef      co.type = co.int->(&);  
IntLValueRef co.type = co.int->(&&);  
IntHeapRef  co.type = co.int->(~);  
IntAddress  co.type = co.int->(@);  
IntThunk    co.type = co.int->(^);
```

```

IntSlice      co.type = co.int->(:);

ThreeInts     co.type = co.int->([3]);
InferredInts  co.type = co.int->([]);
InferredGrid  co.type = co.int->([,] );
ZeroLengthArray co.type = co.int->([0]);
ZeroDimArray  co.type = co.int->([.]);
JaggedArray   co.type = co.int->([][]);
VariableLenArray co.type = co.int->([...]);
VariableRankarray co.type = co.int->([]*); //variable rank + variable
extents

IntPtr        co.type = co.int->(*);
IntPtr        co.type = co.int->(**);
IntPtr        co.type = co.int->(****);

IntRange co.type = co.int->(..);

```

Variable Rank + Variable Extents Arrays and their use

```

Tensor co.type = co.float->([]*);

MatrixOrTensor co.refinementType =
    (Tensor).where(
        _.rank >= 2 && _.rank <= 4
    );

```

The arrow-tail syntax is part of the type expression. A variable subsequently uses the named derived type rather than repeating the derivation inline.

Specialized Types

```

//Generic types must be aliased as derived types
IntList      co.type = co.List(co.int);
IntSet       co.type = co.Set(co.int);
StringIntMap co.type =
    co.Map(co.string, co.int);

```

When Generics are initialized with concrete type, Folang doesn't allow them to be inline instead they are needed to be declared as a type

User-Defined Types

FoLang supports a rich set of user-defined type forms for different modeling requirements.

FoLang does not permit ordinary inner, nested, or anonymous named type declarations directly inside another type definition except through the explicit mechanisms defined by the language.

CStructs

```
// Rect.fol

_ co.cstruct = {
    origin Point;
    width  co.int;
    height co.int;
}
```

`co.cstruct` is a C-like value type: it is passed by value, has a simple memory layout, and is safe to cross supported ABI boundaries.

Worth noting: as package names are derived from directory structure, a top-level type declared with `_` derives its name from the source filename. The resulting type name is canonical and case-insensitive according to the filename-derived declaration rules.

`_` is the filename-derived declaration placeholder.

Structs

```
// Employee.fol

_ co.struct = {
    id      co.int;
    name    co.string;
    address Address;
}
```

FoLang structs are pure data types. They do not carry class-style initialization or field policies such as constant, final, lazy, or similar object-state modifiers unless a separate struct rule explicitly permits them.

Struct bodies do not define ordinary instance methods. Struct-associated behavior is supplied through companion units and associated functions.

Structs do not participate in class inheritance or the class OOP relationship model.

Structs support composition/embedding according to the struct rules.

Struct Embedding

Embedding promotes fields of an embedded struct directly into the outer struct — they act as the outer struct's own fields at construction and access sites. This is distinct from composition where the embedded struct is a named field.

```
// E.fol
_ co.struct = {
  id    co.int;
  name  co.string;
}

// ✅ No conflict – id and name promoted as B's own fields
// B.fol
_ co.struct = {
  age  co.float;
  E;           // embedded – id and name promoted
}

b := B{ age: 25.0, id: 1, name: "Rao" }; // all fields at same level
b.id    // direct – no b.E.id needed
b.name  // direct – no b.E.name needed
b.age   // direct
```

```
// ❌ Compiler error – name conflict between B.name and E.name
// B.fol
_ co.struct = {
  name co.string; // conflicts with E.name
  E;
  age  co.float;
}

// Fix 1 – rename B's conflicting field
// Fix 2 – use explicit composition instead: e E;
```

```
// Explicit composition – no promotion, always qualified access
// B.fol
```

```

_ co.struct = {
  name co.string;
  e    E;           // named field – no conflict, no promotion
  age  co.float;
}

b.name ;    // B's own name
b.e.id ;    // E's id – always explicit
b.e.name;   // E's name – always explicit

```

Embedding Rules

Situation	Behavior
Embedded field, no conflict	Promoted — acts as the outer struct's own field
Embedded field, name conflict with outer	✗ Compiler error — rename or use composition
Multiple embeds, no conflict between them	All fields promoted
Multiple embeds, conflict between embedded structs	✗ Compiler error
Explicit composition (<code>e E</code>)	No promotion — always accessed via <code>b.e.field</code>

FoLang does **not** silently shadow conflicting fields. Any such name conflict is a compile-time error; the conflicting declaration must be renamed or the relationship expressed through an allowed explicit-composition mechanism.

For member-style embedding, an accessible public instance-associated function with an explicit value receiver follows the same promotion and conflict rules as a field. Receiverless companion functions, type-associated functions, operator functions, and companion-associated type declarations remain in their defined companion or operator lookup domains and are not promoted as instance members merely because their owner struct is embedded.

Embedding never widens member access. A promoted field or instance-associated function retains its original declaration owner, symbol identity, and access classifier. A private member remains accessible only inside the embedded struct's ownership boundary, including that struct's companion unit; it is not made accessible to the

embedding struct. Conflict checking considers the accessible members eligible for promotion and reports a compiler error when no unique promoted member can be selected.

Interfaces

```
// IEmployee.fol
_ co.interface = {
    storeEmployee(emp Employee)->(Employee);
}
```

FoLang interfaces are pure behavioral specifications. They contain no instance state and no partially implemented concrete behavior; their methods are abstract requirements.

Classes

```
@co.dap.oops(
    interfaces=[someInterface1, someInterface2, someInterface3],
    classes=[someClass1, someClass2],
    mixins=[someMixin1, someMixin2, someMixin3],
    traits=[someTrait1, someTrait2, someTrait3],
)
// test.fol
_ co.class = {
    getTest(id co.int)->(test) ={}
}
```

co.dap.oops annotation is optional

Classes with Operators

```
// Employee.fol
_ co.class = {
    @co.dap.operator(symbol="+")
    add(other Employee)->(Employee) = {
        // implicit instance method: operands are this and other
    }

    @co.dap.operator(symbol=="")
    @co.dap.static
    equals(
```

```

        left Employee,
        right Employee
    )->(co.bool) = {
        // static operator implementation
    }

    @co.dap.operator(symbol=">")
    @co.dap.class
    greater(
        left Employee,
        right Employee
    )->(co.bool) = {
        // class-associated operator implementation
    }
}

```

FoLang classes may contain instance fields and methods.

Class members may use the following access specifiers:

1. @co.dap.public
2. @co.dap.private (default)
3. @co.dap.internal
4. @co.dap.protected
5. @co.dap.package

Mixins

```

_ co.mixin={

    someNum co.int;

    someFun1()->()={
        co.out.println(someNum);
    }

    @co.dap.abstract
    someotherFun()->();

    @co.dap.virtual
    someVirtFun()->()={

```

```
    ...  
}  
  
}
```

A mixin is the dedicated abstract-class-like composition form; it avoids declaring an ordinary class merely to mark it abstract.

A mixin may contain state, abstract methods, fully implemented methods, and virtual methods where the mixin rules permit them.

A class incorporates a mixin by listing it in the `mixins=[...]` field of `@co.dap.oops`, implements its abstract methods, and may override its virtual methods.

Traits

// EmployeeTrait.fo1

```
_ co.trait={  
  
  @co.dap.abstract  
  someFunction()->();  
  
  somerealFun()->()={  
    co.out.println("Doing real work" );  
  }  
  
}
```

A trait is interface-like but may provide default function implementations. A trait carries no instance state.

A class participating in the `@co.dap.oops` model includes a trait by listing it in the annotation's `traits=[...]` field.

A consuming class must implement every abstract function that remains unsatisfied.

Extensions

```
// EmployeeExtension.fol

_ co.extension->(fortype=somePkg.Employee) = {

    someFun()->>() = {
        co.out.println(this.someName);
    }

    @co.dap.class
    someOtherFun()->>() = {
        co.out.println(this.clsVariable);
    }
}
```

A `co.extension` is a reusable collection of fully implemented methods that adds behavior to one explicitly selected class **without creating a subclass or changing the target class's nominal type identity or inheritance hierarchy**. The extension chooses its target through the mandatory `fortype` argument; the target class does not adopt the extension through `@co.dap.oops`.

FoLang provides two distinct ways to extend class behavior:

1. Inheritance
2. Extensions

Inheritance changes the class hierarchy and is therefore not the preferred mechanism when the requirement is only to add reusable behavior.

For those cases, FoLang provides extensions without changing the target class's nominal inheritance relationship.

Signatures

```
// StackSignature.fol

_ co.signature = {
    T      co.associatedType;
```

```

Stack co.type;

empty(value T)->(Stack);
push(value T, stack Stack)->(Stack);
pop(stack Stack)->(T, Stack);
}

```

A signature defines a structural contract that may include callable requirements and associated declarations/state required by that contract. It is intended for cases where multiple module implementations must satisfy the same shape without introducing a class hierarchy.

Modules

```

// ListStackModule.fol
_ co.module->(matches=StackSignature) = {
  T co.associatedType = co.int;
  Stack co.type = co.List(T);

  empty(value T)->(Stack) = { ... }
  push(value T, stack Stack)->(Stack) = { ... }
  pop(stack Stack)->(T, Stack) = { ... }
}

```

A module is a named singleton implementation unit. It is non-inheritable and is commonly used where a single implementation should satisfy a signature without creating class instances.

Type Classes

In FoLang, a typeclass defines a type-directed behavioral contract for ad-hoc polymorphism. An instance supplies the implementation of that contract for a specific type or valid combination of types.

Functors

```

//Functor.fol
@co.dap.typeclass(
  kind=Functor,
  shape=(F(_)),
  aliases=[

```

```

        {name=MapFunction,    type=(A)->(B)},
        {name=InputContainer, type=F(A)},
        {name=ResultContainer,type=F(B)}
    ]
)
_ co.typeclass = {
    map(value InputContainer, f MapFunction) -> (ResultContainer);
}

```

Applicatives

```

//Applicative.fol
@co.dap.typeclass(
    kind=Applicative,
    shape=(F(_)),
    aliases=[
        {name=MapFunction,    type=(A)->(B)},
        {name=FunctionContainer,type=F(MapFunction)},
        {name=InputContainer,  type=F(A)},
        {name=ResultContainer, type=F(B)}
    ]
)
_ co.typeclass = {
    pure(x A) -> (InputContainer);
    apply(fab FunctionContainer, fa InputContainer) -> (ResultContainer);
}

```

Monads

```

//Monad.fol
@co.dap.typeclass(
    kind=Monad,
    shape=(F(_)),
    aliases=[
        {name=InputContainer, type=F(A)},
        {name=ResultContainer,type=F(B)},
        {name=FlatMapFunction,type=(A)->(ResultContainer)}
    ]
)
_ co.typeclass = {
    pure(x A) -> (InputContainer);
    flatMap(fa InputContainer, f FlatMapFunction) -> (ResultContainer);
}

```

Monoids

```
//Monoid.fol
@co.dap.typeclass(kind=Monoid, shape=(T))
_ co.typeclass = {
  empty() -> (T);
  combine(a T, b T) -> (T);
}
```

Transformers

```
//Transformer.fol
@co.dap.typeclass(
  kind=Transformer,
  shape=(F(_), G(_)),
  aliases=[
    {name=MapFunction, type=(A)->(B)},
    {name=InputContainer, type=F(A)},
    {name=ResultContainer, type=G(B)}
  ]
)
_ co.typeclass = {
  map(value InputContainer, f MapFunction) -> (ResultContainer);
}
```

Instances

An instance provides an implementation of a typeclass for the type or type combination named by the instance declaration.

```
// ListFunctor.fol
_ co.instance->(for=Functor, type=co.List) = {
  map(value InputContainer, f MapFunction)->(ResultContainer) = {
    result := ResultContainer{};
    value.each(_, item, { result.append(f(item)) });
    $=> result;
  }
}
```

```
// ListToSetTransformer.fol
_ co.instance->(for=Transformer, types=[co.List, co.Set]) = {
  map(value InputContainer, f MapFunction)->(ResultContainer) = {
```

```

    result := ResultContainer{};
    value.each(_, item, { result.insert(f(item)) });
    $=> result;
  }
}

```

Using an Instance

An instance is selected **by name**. There is no implicit search.

```

@co.ddap.import(package="abc.tc", as="tc")

IntList co.type = co.List(co.int);
xs IntList = IntList{1, 2, 3};
double(x co.int)->(co.int) = { $=> x * 2; }

ys := tc.ListFunctor.map(xs, double);

```

Activating Instance Methods

An instance may also be **activated**, which makes its functions callable as methods on the receiver. Activation uses the directive `@co.dap.use`

```

@co.ddap.import(package="abc.tc", as="tc")
@co.ddap.use(from="tc.ListFunctor", methods=[map, reduce])

ys := xs.map(double);           // resolves to tc.ListFunctor.map(xs, double)

```

Custom Matcher

```

// PositiveEvenMatcher.fol
@co.dap.matcher
_ co.matcher->(type=co.int) = {
  matchCase(
    value co.int,
    pattern co.untyped
  )->(co.int, co.MatchBindings) = {
    // user logic
    // 0 = no match, >0 = match
  }
}

```

Object Types

Annotations

```
// Annotation implementation – named singleton object, can carry data

// myAnnotation.fol

@co.dap.annotation
_ co.object = {
  value    co.string;
  enabled  co.bool;
}
```

FoLang allows annotations to be defined as named objects and inspected through the reflection facilities where annotation-driven runtime or compile-time decisions are required.

Associated Objects

```
// AuditAnnotation.fol
_ co.object->(for=someAnnotation) = {
  ...
}
```

FoLang does not place global/shared state directly on classes. Class-associated constant, immutable, global, shared, or lock-protected state is owned by an associated `co.object`, which is a named singleton and may be associated with one or more types.

```
// ProducerConsumerShared.fol
_ co.object->(
  for=[Producer, Consumer]
) = {
  @co.dap.const capacity co.int = 100;
  @co.dap.final configuration QueueConfiguration = loadConfiguration();

  queue Queue;
  queueLock co.lock;

  enqueue(value Item)->() = {
    lock(queueLock) {
```

```
        queue.add(value);
    }
}

dequeue()->(Item) = {
    lock(queueLock) {
        $=> queue.remove();
    }
}
}
```

Unions

`co.union` declares an untagged ADT. Its body lists the alternative members defined by the union.

```
// myUnion.fol
_ co.union={
    intValue co.int;
    strValue co.string;
}
```

Blocks

Named Blocks

```
labelBlock co.block = {
}

labelBlock.expand();
```

A named block is a reusable named block value that can be expanded according to the `co.block` rules.

Anonymous Blocks

An ordinary anonymous block is a lexical execution scope. It can access bindings visible in its enclosing lexical scope and introduces a child lexical scope for declarations made within it:

```
{  
    // statements  
}
```

Label blocks

```
'outer: {  
    // statements  
  
    (someCondition).then({  
        $->| 'outer;  
    });  
}
```

Enums

`co.enum` is FoLang's closed tagged algebraic-data-type declaration. The body of an enum contains only **state declarations**. States are owned by the enclosing enum and form its complete set of alternatives.

```
// Shape.fo1  
_ co.enum = {  
    Circle(radius co.float),  
    Square(side co.float),  
    Rectangle(width co.float, height co.float),  
    Point  
}
```

Enum States and State Functions

Every enum member is a **state**. A state is not an independent type and does not introduce a struct, class, object, or constructor declaration.

A state with no parameters is a complete value of the enclosing enum type. It is referenced directly, without `()`:

```
origin Shape = Shape.Point;
```

`Shape.Point()` is invalid because `Point` declares no parameters and is not a zero-argument ordinary function call.

A state with one or more parameters defines a compiler-provided **state function**. Every state-function parameter is intrinsically named. The declaration therefore gives each payload component both a name and a type:

Calling a state function produces a value of the enclosing enum type. A call must bind every supplied payload value explicitly by parameter name using `name=value`; a positional state-function call is invalid:

```
circle Shape = Shape.Circle(radius=5.0);
square Shape = Shape.Square(side=4.0);
rect  Shape = Shape.Rectangle(width=4.0, height=6.0);
rect2  Shape = Shape.Rectangle(height=6.0, width=4.0); // order is
irrelevant

// invalid: positional payloads are never accepted for enum state functions
// Shape.Circle(5.0);
// Shape.Rectangle(4.0, 6.0);
```

Conceptually:

value payload	static type	state	named
Shape.Circle(radius=5.0)	Shape	Circle	{radius=5.0}
Shape.Circle(radius=10.0) {radius=10.0}	Shape	Circle	
Shape.Square(side=5.0)	Shape	Square	{side=5.0}
Shape.Point	Shape	Point	{}

Variable Declaration and Initialization

```
intVar co.int; //isNone() true  by default it is initialized to
co.const.none if a value not provided

intVar1 co.int = 10; // initialized with 10

intVar2 := 10 ; //defines and initialized with 10 inferred to `co.int`

intVar3 ?= 20; // defines and initializes with 20, inferred as co.int

intVar3 ?=40; // sets the variable to 40 as it exists

intVar4 ::= 50; // defines a dynamic variable backed by co.any; current
```

```

value is co.int

intVar4 = "abc"; // valid

intVar3 = "abc"; // compiler error
intVar2 = "abc"; // compiler error
intVar1 = "abc"; // compiler error

intVar2 := 30; // compiler error as intVar2 already defined in scope

intVar4 ::= 'c'; // valid for a dynamic variable; current value is
character 'c'

```

A variable declared with `::=` is a dynamic variable represented by `co.any`. It may hold values of different concrete types during its lifetime while ordinary lexical scope and name-resolution rules still apply.

Lazy calls

```

@co.dap.lazy
x := someFun(10);

```

What lazy does is, it will not invoke `someFun` immediately, it will invoke only on first time accessing the variable `x`.

BoundCallables

```

add(a co.int, b co.int)->(co.int)={
    $=> a + b;
}

x co.boundcallable = add.bind(10,30);

z := x.invoke(); //actual call happens here

```

Symbols

FoLang exposes compiler/language symbol values through `co.symbol` where symbol objects are required by AST or metaprogramming facilities.

Function-Shaped Declarations

Functions and Methods

```
add ( a co.int, b co.int )->(co.int)={  
    $=> a + b;  
}
```

Multiple Return

```
returnPair( a co.int, b co.int)->(co.int, co.int)={  
    $=> b, a;  
}
```

Inner function

```
someFun( a co.int, b co.int)->(co.int) = {  
  
    someInner(b co.int)->(co.int)= {  
        $=> b * 2;  
    }  
  
    $=> someInner( a ) + b;  
  
}
```

Anonymous Inner Functions

Anonymous functions are not standalone top-level declarations.

```
someFun( a co.int, b co.int )->(co.int)={  
  
    $=> (a co.int)->(co.int){  
        $=> a * 4;  
    }(b) + a;  
  
}
```

An anonymous function must be used in a context that consumes the resulting function object, such as assignment to a variable or immediate invocation.

```

someFun( a co.int, b co.int )->(co.int)={

    temp co.function = (a co.int)->(co.int){
        $=> a * 4;
    }

    temp( b) + a;

}

```

Curried Functions

```

add ( a co.int)(b co.int)(c co.int)->(co.int)={
    $=> a + b + c;
}

someVar co.function = add(1);
someOtherVar co.function = someVar(2);

result co.int = someOtherVar(3);

```

Named Parameter Functions

```

// someNamedParam.unit.fol
fun1(~k co.int, ~v co.int)->()={

}

Usage:
fun1(v=10,k=20); // valid
fun1(10,20); //valid here k =10 and v=20

```

Optional Parameter Functions

//someOptional.unit.fol

```

fun1(k? co.int)->()={
    k.omitted.then({

    }).default({

```

```
});  
  
}
```

Default Parameter Functions

// somefununit.unit.fol

```
fun1(k co.int, b co.char = 'A')->(co.int, co.char)={  
}  
  
usage:  
  fun1(10, 'B'); // k = 10 and b = 'B'  
  fun1(10);      // k = 10 and b = 'A', the declared default value
```

Variadic Functions

//someCurried.unit.fol

```
fun1 (k co.int, ...b co.char)->(co.int, co.char)={  
}
```

Closures

//someClosure.unit.fol

```
IntAdder co.type = (co.int)->(co.int);  
  
adder()->(IntAdder) = {  
  sum co.int = 0;  
  $=> (x co.int)->(co.int){  
    sum += x;  
    $=> sum;  
  };  
}
```

When a function returns an inner function that captures bindings from the enclosing lexical scope, the returned function is a closure. The captured bindings remain part of that closure's state according to the closure rules.

Named Callable-Type Implementation

An ordinary named function may implement a named `co.type` whose resolved underlying type is callable. This form reuses the callable contract instead of restating parameter and result types.

Canonical forms:

```
name CallableType(parameterNames) = expression; //expression bodied
functions

name CallableType(parameterNames) = {
    ...
}
```

Example with an ordinary function type:

```
funType co.type = (co.int)->(co.int);
x co.int = 10;
SomeFun funType(x) = x * 10;
```

```
name CallableType(parameterNames) = expression;
```

Function Patterns

FoLang expresses function-pattern behavior through ordinary functions whose expression body performs pattern matching.

```
classify(n co.int)->(co.string) =
    n.match()
        .case(v: v > 0 => "positive")
        .case(v: v < 0 => "negative")
        .default("zero");
```

Expression-Bodied Functions with Polymorphic Callable Types

```
PolyId co.type = co.polymorphic({U}, (U)->(U));

identity PolyId(value) = value;
```

Local and Nested Types

Local Types

FoLang provides `@co.dap.local` for eligible top-level definitions, including functions and types, to restrict a declaration to one or more designated owners without physically nesting the declaration in source.

```
// EmployeeAddress.fol
@co.dap.local(for=[hr.employee.Employee,hr.manager.Manager])
_ co.struct = {
    street co.string;
    city   co.string;
}
```

```
// Employee.fol
_ co.struct = {
    id      co.int;
    name    co.string;
    address EmployeeAddress; // composition
}
```

When a local declaration applies to one owner, `for=hr.employee.Employee` is sufficient. When it applies to multiple owners, `for=[...]` uses the list form.

Nested Types

```
// EmployeeAddress.fol
@co.dap.nested(target=hr.employee.Employee)
_ co.struct = {
    street co.string;
    city   co.string;
}
```

```
// Employee.fol
_ co.struct = {
    id      co.int;
    name    co.string;
    address EmployeeAddress; // composition
}
```

`target` is always single

```

@co.dap.local(for=addFunc(co.int, co.int)->(co.int))
someFunc(a co.int)->(co.int)={
    $=> a *100;
}

addFunc(a co.int , b co.int)->(co.int)={
    $=> someFunc(b) + a;
}

subFunc( a co.int, b co.int)->( co.int)={
    $=> someFunc(a) -b; // compiler error as someFunc is for only addFunc
}

```

Nested vs Local

Nested declarations model true inner declarations: they may resolve the enclosing declaration's visible state, variables, parameters, and members according to normal accessibility and lexical-capture rules.

```

@co.dap.local(for=addFunc(co.int, co.int)->(co.int))
someFunc()->(co.int)={
    $=> a *100; // compiler error as a is undefined
}

addFunc(a co.int , b co.int)->(co.int)={
    $=> someFunc(b) + a;
}

@co.dap.nested(target=addFunc(co.int, co.int)->(co.int))
someFunc()->(co.int)={
    $=> a *100; //works fine as addFunc defined the variable as parameter
}

addFunc(a co.int , b co.int)->(co.int)={
    $=> someFunc(b) + a;
}

```

Inline Functions

```

@co.dap.inline
add(a co.int, b co.int)->(co.int) ={
    $=> a + b;
}

```

`@co.dap.inline` requests inline expansion of the function at eligible call sites according to the compiler/backend rules.

Deferred Functions

```
someErrorFun(a co.int)->>() = {
    resource Resource = acquireResource();

    @co.dap.defer
    cleanup(resource Resource, originalArgument co.int)->>() = {
        releaseResource(resource);
        logArgument(originalArgument);
    }(resource, a);

    performOperation(resource);
}
```

FoLang deferred functions are registered for execution when the enclosing function completes, including completion through an effect path, subject to the defer ordering and effect-handling rules.

Function Scopes

FoLang uses lexical scope for function name resolution. Functions and nested callable forms resolve enclosing bindings from their definition site; FoLang does not define a dynamic-scope mode.

Explicit Calling Context

`@co.dap.with(context)` on a callable invocation explicitly exposes the caller's current Context to that invocation under `$.ctx`. Inside the called function, `$.ctx.name` accesses caller bindings; the callee's own `$` Context remains unchanged and ordinary unqualified lookup remains lexical.

```
doSome(a co.int, b co.int)->()={
    $.ctx.b = a + b ; //throw error as the ctx under $ is not available if
    no annotation at call site.
}

someFun (a co.int)->()={
```

```

b := 10;

@co.dap.with(context)
doSome(10, 20);

co.out.println( b ); // will print 30
}

```

Function Chaining

FoLang function chaining passes the result positions of one function invocation to the next chained invocation.

```

addFunc(a co.int, b co.int)->(co.int)={
    $=> a + b;
}

subFunc( a co.int, b co.int)->(co.int)={
    $=> a - b;
}

myFunc(a co.int, b co.int)->(co.int,co.int)=>>addFunc($1,$2);
someOtherFunc()->(co.int, co.int)=>>subFunc($1,$2);

```

`$1` through `$N` are bind variables for the immediately preceding chained call's result positions.

`$1` through `$N` correspond to declared return order.

Decorators

FoLang decorators add behavior around functions and methods according to the decorator's declared scope and execution phase.

```

@co.dap.decorator
myDecorator(target co.function)->(co.function) = { }

```

`@co.dap.decorator` supports attributes such as:

```

scope= runtime, compiletime
when= before, after, around , afterEffect

```

Macros

```
// b. Escape assign
@co.dap.macro
yes_esc_assign()->(co.untyped)={
  $=> co.macro.quote({
    co.macro.esc(y) = 42;
    co.out.println("Inside macro: y = ", y);
  });
}
```

```
@co.dap.macro
debug(expr)->(co.untyped)={
  tmp := co.macro.gensym(co.var, "tmp");
  $=> co.macro.quote({
    tmp = co.macro.esc(expr);
    co.out.println("Result: ", tmp);
    tmp;
  });
}
```

if else macro

```
@co.dap.macro(
  group={items=["if","else"], chain=true},
  sugarform={forms=["if expr block"]},
  bind={vars=["x"]},
  isolate={vars=["temp", "index"]},
  gensym={prefix="tmp_"},
  hygienic=true,
  argtransform={param="body", wrap="lambda", whentype="block"},
  desugar={exprs=["if($cond) { $block }" => "if($cond,$block)"]},
  mode="inject"
)
if(condition expr, body block)->()={}
```

blockormacro co.kind = block | macro

```
@co.dap.macro(
  group={items=["if","else"], chain=true},
  sugarform={forms=["else block","else if"]},
  chainswith={macro="if", position="immediate", required=true},
  argtransform={param="body", wrap="lambda", whentype="block"},
  standalone=false,
```

```

    desugar={exprs=[
        "else if($cond) { $block }" => "else(if($cond, $block))",
        "else { $elseblock }" => "else($elseblock)"
    ]},
)
else(body blockormacro)->()={}

```

FoLang macros define compile-time source/AST transformations. They may construct or rewrite FoLang syntax according to the macro metadata and hygiene rules.

Extension Methods

```

@co.dap.extension(fortype=co.string, what=extends)
upperCase()->(co.string) = {
    $=> this.upper();
}

@co.dap.extension(fortype=[co.string], what=overrides)
equals(str co.string)->(co.bool) = {
    $=> this == str;
}

```

FoLang extension methods add behavior to supported structs, modules, or other permitted targets without modifying the original declaration. For classes, reusable behavior should use the class extension mechanisms defined by the language rather than introducing inheritance solely for convenience.

Activating Extension Methods

Activation uses `@co.dap.use`. For declarations in the current package, omit `from`:

```

@co.ddap.use(methods=[equals, upperCase])
k.upperCase();

```

For another package, use its alias or complete package path:

```

@co.ddap.import(package="text.util", as="tu")

@co.ddap.use(from="tu", methods=[upperCase])
@co.ddap.use(from="text.util", methods=[upperCase])

```

Templates

```
@co.dap.template
add(a co.int, b co.int)->(co.int) ={
    $=> a + b;
}
```

```
@co.dap.template
add(a, b)->(co.untyped) ={
    $=> a + b;
}
```

Templates are compile-time substitution/expansion mechanisms. Typed templates preserve the declared callable contract and may be expanded similarly to inline function calls according to the template rules.

Indexers

```
@co.dap.indexer(symbol="[]")
(g MyList) get(index co.int)->(co.int) ={
    $=> g.eles[index];
}

@co.dap.indexer(symbol="[]=")
(g MyList) set(index co.int, value co.int)->() ={
    g.eles[index] = value;
}

lst MyList;
co.out.println(lst[0]);
lst[1] = 22;
```

Indexers provide array-like `[]` and `[]=` access for user-defined types.

Execution Model

FoLang executes ordinary code **sequentially by default**. A normal function or method declaration therefore requires no execution-model decorator merely to be called sequentially.

The built-in decorator `@co.dap.executionmodel(...)` is used only when a declaration requires non-default execution semantics that must remain observable across conforming FoLang implementations. The language exposes an execution-

model choice only when that choice changes required FoLang behavior. A distinction that changes only a backend’s internal implementation strategy is not a separate FoLang execution model.

```
@co.dap.executionmodel(type=concurrent, kind=task)
  someConcurrent(a co.int)->(co.int,co.error) = {
    ...
  }

co.cpcsa.submit(someConcurrent, params=SubmitParams{10},
results=SubmitResults{val, errors});
(errors.isEmpty).then(
  co.out.println(val)
).default(
  co.out.println(errors)
);
```

for scheduling

```
@co.dap.executionmodel(
  type=concurrent,
  kind=task,
  scheduling=cooperative
)
```

Execution-Model Summary

Type	Source-level kind/model	Optional semantic dimension	Package responsible for operations
default sequential	—	—	ordinary function/method call
concurrent	task, thread	scheduling=cooperative preemptive; structural policies such as fork-join when separately defined	co.cpcsa
parallel	—	parallel-specific policy only when separately specified	co.cpcsa

<code>async</code>	—	<code>completion=future</code> <code> callback</code>	<code>co.cpa</code>
<code>continuation</code>	<code>full</code> , <code>delimited</code>	<code>delimited</code> <code>control="shift-</code> <code>reset" "prompt-</code> <code>control" "spawn-</code> <code>yield"</code>	<code>co.control</code>

Native Methods

```
@co.dap.native
nativeMethod(a co.int, b co.int)->(co.int) = {
    // native implementation
}
```

FoLang provides native-only constructs that require the applicable `@co.dap.native` classification, including declarations that use `co.native` facilities. See [Built-in Packages](#).

Native methods must be declared within a native component or a library whose capability domain is `native`.

Associated Functions

```
@co.dap.operator(symbol="+")
(emp Employee) add(other Employee)->(Employee) = {
    ...
}

@co.dap.operator(symbol=="")
equals(
    left Employee,
    right Employee
)->(co.bool) = {
    ...
}

@co.dap.operator(symbol">")
(Employee) greater(
    left Employee,
    right Employee
)->(co.bool) = {
```

```
    ...  
}
```

Associated functions for a struct must be declared in that struct's companion unit, using the `<StructName>.comp.unit.fol` naming convention.

Comprehensions

A FoLang comprehension is a source-driven transformation. Its canonical form is:

```
result := (pattern <- source).yield(resultExpression);
```

Permitted Comprehension Sources

FoLang comprehensions intentionally accept only the following source categories:

1. **Any FoLang iterable**, including standard iterable forms such as arrays, lists, sets, maps/dictionaries, and ranges.
2. `Some(T)`.
3. `Future(T)`.

For example:

```
(x <- IntList{1,2,3}).yield(x * 2); // valid: iterable  
(x <- 1 .. 10).yield(x * 2);        // valid: iterable range  
((k, v) <- valuesMap).yield(k, v); // valid: iterable map/dictionary  
(x <- Some(5)).yield(x * 2);        // valid: permitted non-iterable source  
(x <- someFuture).yield(f(x));      // valid: permitted non-iterable source
```

Lambda Expressions

```
k := (1 .. 10)  
  .filter(|x| => x % 2 == 0)  
  .map(|x| => x * x);
```

Only allowed as an inline callback argument to receiver-qualified collection operations (e.g. `each`, `map`, `filter`, `reduce`, `forEach`, `sortBy`, `groupBy`).

The lambda must be a direct argument of the allowed collection call. That call may itself be nested, for example `consume(nums.map(|x| => x*x))`; the enclosing call does not make the lambda an argument of `consume`.

Lambda expressions use lexical scope. They may reference bindings visible at their definition site, which are captured when required by the lambda's lifetime.

Conditions, Loops, and Ternary Operations

Conditions

`then` is the one-shot conditional branch verb. Its argument may be a block or an ordinary value/expression. `when(condition)` introduces each additional Boolean condition and must be followed by `.then(...)`. `default(result)` is the optional terminal fallback and may likewise receive a block or value. The same `then / when(condition) / default` chain is used for statement-style conditional execution and value-producing ternary selection.

A block supplied to a conditional branch follows the ordinary anonymous-block lexical-scope rules.

```
(boolean truth).then({
}).when(boolean truth).then({
}).default({
});
```

```
x co.bool =co.const.true;
(x).then({ // parentheses optional

}).default({

});
```

Loop

```
(boolean truth).loop({
});

x := co.const.true;
```

```
x.loop({  
  
});
```

Ternary

A value-producing multi-branch conditional therefore has the canonical shape:

```
s = (boolean truth).then(some var/val).when(boolean truth).then(some  
var/val).default(some var/val);
```

```
(n > 0)  
  .then("positive")  
  .when(n < 0)  
  .then("negative")  
  .default("zero");
```

Chain head	Parentheses	Why
<code>x</code>	optional	an identifier is already a complete postfix expression
<code>arr[0]</code> , <code>f()</code>	optional	index and call suffixes are postfix too
<code>co.const.true</code> , <code>myPkg.flag</code>	required	a qualified name would absorb <code>.then</code> as a further segment, giving <code>co.const.true.then</code>
<code>x > y</code>	required	<code>.then</code> binds tighter than <code>></code> , so <code>x > y.then({...})</code> groups as <code>x > (y.then({...}))</code>

A qualified name needs parentheses whether it names a literal or an identifier, so `myPkg.flag` is the same case as `co.const.true`.

Pattern Matching

```
x co.int = 10;  
  
x.match().case(n: n > 10 => { n = n+100; "GT" }).case( n: n < 10 =>
```

```

"LT").default("EQ");
x.match().case(n: n > 10 => { n = n+100; "GT" }).case( n: n < 10 =>
"LT").case(_=>"EQ");
x.match(co.pattern.Type).case(co.int => ...).case(co.float => ...);
x.match(co.pattern.Value).case(0 => ...).case(1 => ...);
x.match(co.pattern.Instance).case(xx.CAT => ...).case(xx.DOG =>
...).default("Animal");
x.match(co.pattern.Object).case(xx.Ball => "Ball").case(xx.CAT =>
"CAT").default("Unknown");
x.match(co.pattern.Shape).case(Point{x, y} => ...).default(...);

x.match(co.pattern.Any).case(co.int => ...).case(co.float => ...).case(0 =>
...).default( ...);

x.match(PositiveEvenMatcher).case(0 => "Neither even nor odd").case(2 =>
"First Even Prime").default(...);

```

FoLang provides pattern matching with both built-in matcher categories and user-defined custom matchers.

`match()` uses the default value matcher. FoLang also provides the following built-in matcher categories:

1. `co.pattern.Type`
2. `co.patter.Value` -> (default)
3. `co.pattern.Instance`
4. `co.pattern.Object`
5. `co.pattern.Shape`
6. `co.pattern.Kind`
7. `co.pattern.Any`

Units

A unit is a stateless source container. A package may contain any number of ordinary unit files, and all their members are consolidated directly into the package namespace.

Companion Units

Companion units name start with struct name `structname.comp.unit.fol` unlike normal units name which can be anything `xxxx.unit.fol`. Compiler will throw error if the `structname.fol` not available in that package and it is not having top level declaration with `co.struct`.

Normal Units

The constructs under Normal units are addressed with package name as they are merged into the same package and any conflicts will be compiler error. Name of normal unit fols can be anything as name of unit doesn't matter at the end they belong to `package`.

Generics and Specializations

Generics

```
@co.dap.generic(  
  at=callsite,  
  types=[  
    {name=U, variance=invariant, bound=Number, inference=param},  
    {name=T, variance=invariant, bound=Number}  
  ],  
  mapping=[  
    {U=co.int, T=co.int},  
    {U=co.float, T=co.float}  
  ],  
  resolution=compiletime  
)  
add(a U, b U)->(T) = { $=> a + b; }
```

Generic annotation fields:

Attribute	Values
types	list of generic-parameter records such as <code>[{name=T}, {name=R}]</code>
aliases	declaration-local derived type aliases such as <code>[{name=Result, type=F(B)}]</code> ; alias expressions may reference markers declared by <code>types=</code>

requires	
mapping	compile-time relationships among declared generic parameters; valid only for compile-time generic resolution
resolution	<code>runtime</code> , <code>compiletime</code>
reified	<code>true</code> or <code>false</code>
at	<code>usesite</code> or <code>callsite</code>
specializable	<code>true</code> or <code>false</code>
lifecycle	<code>true</code> or <code>false</code> ; interpreted only when the generic declaration target is <code>co.class</code> ; ignored for lifecycle semantics on generic structs/functions/methods

types attributes

Attribute	Values
name	
constraints	
upper-bound	
lower-bound	
bound	not needed when <code>upper-bound</code> and/or <code>lower-bound</code> is specified
default	
variance	<code>covariant</code> , <code>invariant</code> , <code>contravariant</code>
nullable	
inference	<code>param</code> , <code>arg</code> , <code>var</code>
capabilities	
isAKind	an interface, class, or trait; e.g. <code>SomeInterface</code>
typekind	<code>type</code> , <code>class</code> , <code>function</code> , <code>struct</code> , <code>typecon</code> <code>structor</code>
inclusive	

The entries above are fields of each generic-parameter record inside the `types=[...]` list.

Example: `@co.dap.generic(types=[{name=T, variance=..., bound=...}])`

These are not independent attributes; they describe each declared generic type entry.

`lifecycle` is different from the per-type records above: it is a field of the outer `@co.dap.generic(...)` application. Every class already has compiler-owned inherited lifecycle implementations. For a generic class, `lifecycle=true` grants the developer permission to override or overload `@@new` and `@@init`; absent or `false` forbids developer lifecycle customization. The field does not itself expose a lifecycle call. A developer-defined lifecycle implementation participates in `::new(...)` or `::init(...)` lookup according to its own accessibility. On a generic struct, function, or method, the field is accepted but not considered for lifecycle semantics.

Generic Marker Classification

Names listed by the immediately associated `@co.dap.generic(types=[...])` are **generic markers** for that declaration. In applicable type positions the parser/frontend records those names as generic-marker references; they are not looked up as ordinary type symbols.

A type-position identifier that is **not** listed in that declaration's `types=[...]` list is an ordinary type reference. It is resolved through the normal symbol/type environment and is a compiler error when no such type exists.

```
@co.dap.generic(types=[{name=U}])  
f(x U, y T)->(U) = { ... }
```

Here `U` is a generic marker. `T` is not declared as a marker by the annotation, so `T` must resolve as an actual type symbol. If no type named `T` is visible, the declaration is invalid.

By contrast:

```
@co.dap.generic(types=[{name=U}, {name=T}])  
f(x U, y T)->(U) = { ... }
```

both `U` and `T` are generic markers.

Generic-marker spelling does not manufacture a distinct generic signature. These two declarations have the same generic parameter structure and the same parameter signature, so they conflict as duplicates in one callable identity:

```
@co.dap.generic(types=[{name=T}])  
f(x T)->(T) = { ... }  
  
@co.dap.generic(types=[{name=U}])  
f(x U)->(U) = { ... } // compiler error: duplicate generic signature
```

The compiler compares declared generic-marker roles/positions rather than treating `T` and `U` as ordinary type names. This normalization applies **only to names already classified as generic markers**. An undeclared type-position name is never converted into a generic marker merely because another declaration uses a marker with a similar role.

For example, if an actual type named `T` exists, these parameter structures are distinct:

```
@co.dap.generic(types=[{name=U}])  
f(x U, y T)->(U) = { ... } // second parameter is actual type T  
  
@co.dap.generic(types=[{name=U}])  
f(x U, y U)->(U) = { ... } // both parameters use generic marker U
```

Return types still do not participate in ordinary overload selection; the example uses the parameter signature deliberately because that is where the structural distinction matters.

Generic-Context Derived Aliases

The built-in `aliases=` field of `@co.dap.generic` declares derived type names owned by that one generic declaration. Each record has exactly two fields: `name`, the local alias name, and `type`, the type expression derived from the markers

declared by the same annotation's `types=` list.

```
@co.dap.generic(  
  types=[{name=F}, {name=A}, {name=B}],  
  aliases=[  
    {name=FunctorOf,      type=Functor(F)},  
    {name=MapFunction,    type=(A)->(B)},  
    {name=InputContainer, type=F(A)},  
    {name=ResultContainer, type=F(B)}  
  ]  
)  
mapAll(  
  inst FunctorOf,  
  value InputContainer,  
  fn MapFunction  
)->(ResultContainer) = {  
  $=> inst.map(value, fn);  
}
```

These aliases are not independently quantified polymorphic types and do not introduce another generic parameter list, a new type, nominal identity, or a separate specialization mechanism. The `aliases=` field only gives another name to a type representation within the annotated generic declaration. Each alias resolves exactly like a `co.type` alias, using the generic parameters introduced by the same annotation. Ordinary generic substitution applies to the alias's underlying representation. For a substitution with `F=co.List`, `A=co.int`, and `B=co.string`, the effective representations are:

```
FunctorOf      = Functor(co.List)  
MapFunction    = (co.int)->(co.string)  
InputContainer = co.List(co.int)  
ResultContainer = co.List(co.string)
```

All names referenced by an alias expression must be either declared generic markers or ordinary visible type names. Alias names are available only in the associated declaration's signature and body; they are not package members and cannot be imported or used by another declaration. An alias name must not duplicate a generic marker, parameter, sibling alias, or another name in the declaration's signature scope. References between aliases use the ordinary `co.type` alias-resolution and cycle-detection rules; `aliases=` does not introduce separate ordering, expansion, or cycle semantics.

The `type=` member in this field is a deliberate built-in-metadata exception to the ordinary annotation-value restriction. The parser reads it with the type expression grammar because `@co.dap.generic` is compiler-defined and already classifies the following declaration. Custom annotations cannot define an equivalent field, introduce aliases, or affect parsing decisions.

Frontend Handling of Generic Metadata

The frontend must parse and preserve the complete `@co.dap.generic` metadata application. Fields needed to establish frontend syntax, symbol identity, generic-marker classification, type resolution, callable selection, or a concrete result contract are interpreted where required. In particular, `types=` establishes generic markers, `aliases=` establishes declaration-local derived type names after those markers are known, and `mapping=` participates in the compile-time result-resolution rules defined below.

Other generic fields and attributes may be backend- or later-stage-oriented. The frontend records and serializes them in the Final AST/backend interchange and does **not** fail frontend generation merely because it has no semantic handler for such a field. A later compiler/backend stage may interpret, validate, specialize, reify, or reject those preserved values according to the applicable feature contract. Malformed metadata syntax remains a parser error.

When `mapping=` is present, the mapping relation itself must be resolved by the frontend wherever it is necessary to produce a concrete callable/result contract. A backend-oriented field such as `resolution`, `reified`, `at`, or `specializable` does not by itself block frontend artifact generation merely because the frontend does not otherwise act on it.

Type classification is never supplied by a redundant generic-metadata flag. After name and type resolution, the resolved FoLang type is the authoritative source for whether a value or callable is concrete, polymorphic, dependent, variant-based, refinement-based, or otherwise type-classified. Metadata cannot override or restate a classification already determined by the resolved type.

Generic Mapping, Result Resolution, and Class-Inheritance Augmentation

`mapping=` defines a finite compile-time relationship among the generic parameters declared by the same `@co.dap.generic` annotation. It is **not required merely because a generic parameter appears in the return signature**. If every generic needed by the return signature is already resolvable from callable parameter-position generic information (or from explicit generic arguments), the return signature uses those already-resolved generic values directly and `mapping=` is unnecessary. `mapping=` is required only when a generic needed by the result contract would otherwise remain unresolved and is intended to be derived from already-resolved generic inputs.

```
@co.dap.generic(  
  types=[  
    {name=U, inference=param},  
    {name=T}  
  ],  
  mapping=[  
    {U=co.int,   T=co.int},  
    {U=co.float, T=co.float}  
  ]  
)  
f(x U, y U)->(T) = {  
  ...  
}
```

For `f(10, 20)`, ordinary parameter analysis first establishes `U=co.int`; the mapping then establishes `T=co.int`. For arguments statically typed `co.float`, the second row establishes `T=co.float`. The return type did not select the callable and the expected destination type was not consulted.

When `mapping=` Is Not Required

A generic result needs no mapping when every generic variable occurring in the return signature is already determined directly from the callable's parameter types or explicit generic arguments.

```
@co.dap.generic(types=[{name=T}])  
identity(x T)->(T) = {  
  $=> x;  
}
```

```
@co.dap.generic(types=[{name=T}])
choose(x T, fallback T)->(T) = {
    ...
}
```

For these declarations, parameter-position inference establishes all generic values needed by the result contract before result typing begins. For example, passing `co.int` values to `identity` or `choose` establishes `T=co.int`, so the result type is `co.int` without a mapping row.

By contrast, this declaration is incomplete without another explicit resolution source for `T`:

```
@co.dap.generic(types=[{name=U}, {name=T}])
f(x U, y U)->(T) = {
    ...
}
```

A call such as `f(10, 20)` can establish `U=co.int`, but nothing in the parameter signature establishes `T`. FoLang does not infer `T` from the assignment target, expected destination type, or another return context. Therefore `T` must be supplied explicitly, resolved by an applicable `mapping=` row, or resolved by another mechanism explicitly defined by this specification.

The generic-result rule is therefore:

```
infer generic values from parameter positions and explicit generic
arguments
-> if every generic needed by the return signature is resolved
    mapping is unnecessary
-> otherwise
    resolve the remaining generic through mapping= or another
explicit mechanism
-> expected/destination return type never participates
```

Generic candidate preparation and resolution proceed without consulting return context:

```
resolve canonical callable identity and candidate declarations
-> for each generic candidate, infer generic values available from
    ordinary call-parameter positions and explicit generic arguments
-> instantiate enough of each candidate's parameter signature to test
```

```
ordinary static overload applicability/specificity
-> select the unique parameter overload
-> apply mapping rows to the selected generic's already-resolved inputs
-> resolve any remaining mapped generic parameters
-> complete generic instantiation
-> type-check the concrete return/result contract
```

A `mapping=` row is therefore never used to make a generic function win ordinary overload selection. It resolves mapped generic parameters only after the callable has been selected from parameter information.

A generic parameter that occurs only in the return signature is **never inferred from the expected destination or return context**. It must be resolved by an explicit generic argument, by an applicable `mapping=` row, or by another generic-resolution mechanism explicitly defined by this specification. Result-context generic inference is not part of FoLang.

When `mapping=` is present, its rows define the permitted mapped relationships for the generic parameters they mention. Mapping matches use canonical type identity; overload-style subtype widening is not performed while choosing a mapping row. Once the determining generic values are known, the mapping must yield one consistent assignment for every unresolved mapped generic. No applicable row, multiple conflicting applicable rows, a cyclic unresolved dependency, or a mapping that leaves a required generic unresolved is a compile-time error.

Mapping does not create sibling function overloads. The declaration above remains one generic callable with one body and one declared return-signature structure `-> (T)` even though different valid instantiations may make `T` concrete as different types. This is generic instantiation, not return-type overloading.

Generic Mapping Augmentation Through Class Inheritance

Generic mapping augmentation is intentionally restricted to **class inheritance**. Package-level/free functions do not need cross-package augmentation because package ownership is part of callable identity: two same-named functions in different packages are different callables rather than one overload family. An application therefore cannot reopen another package merely to add `mapping=` rows to a package function.

When a class inherits a visible generic method, additional mapping entries may be associated with that **already inherited generic method** in the derived-class context. The thing being augmented is the inherited method's effective **mapping set**. The augmentation does not declare another callable, does not provide another body, and is not a bodyless/forward generic method declaration.

A bodyless generic method remains an ordinary forward declaration and **cannot contain `mapping=`**. Mapping metadata intended as an inheritance augmentation must resolve to exactly one inherited generic method and must match that method's callable/receiver category, declared generic-marker structure, ordinary parameter-signature structure, and declared return-signature structure. If no such inherited generic exists, the mapping contribution is a compiler error.

This can apply to classes defined by the application itself or classes from package contexts explicitly exported by a packaged component or standalone packaged library and therefore present in the executable application's open graph. It does not penetrate projected `application`, `native`, or `dynamicvmrt` boundaries because their internal classes and methods remain hidden behind surface APIs.

Conceptually:

```
BaseProcessor.convert
  generic markers: U, T
  implementation body: owned by BaseProcessor
  mapping set:
    {U=co.int, T=co.int}

DerivedProcessor inherits BaseProcessor.convert
  augmentation mapping contribution:
    {U=abc.Employee, T=abc.SuperEmployee}

DerivedProcessor effective inherited convert mapping set:
  {U=co.int, T=co.int}
  {U=abc.Employee, T=abc.SuperEmployee}
```

The compiler merges inherited and derived-context mapping entries as a set:

```
identical row + identical row
  -> one logical row

same already-resolved input assignment -> same derived assignment
```

-> duplicate; one logical row

same already-resolved input assignment -> different derived assignment

-> conflict; compiler error

different input assignment

-> additional valid mapping row

The augmentation affects the effective inherited generic in that derived-class context; it does not rewrite the base declaration globally and does not alter sibling derived classes. Generic mapping remains a compile-time frontend mechanism where it is required to establish the concrete callable/result contract, and it is not re-evaluated from runtime argument types by `@co.ddap.dynamicdispatch(true)`.

Generic Functions — Parameters and Return Values

Rank-1: Outer function is generic; parameter uses the same type variable

`T` is fixed at the call site before the function parameter is used. The passed function is already monomorphic inside the body.

Invalid inline signature

The inline form is a compiler error because `(T, T)->(T)` is a derived type used directly in a parameter. It must be named by the generic declaration.

Required named type alias

//somGen2.unit.fol

```
_ co.unit = {  
  @co.dap.generic(  
    types=[{name=T}],  
    aliases=[{name=SomeFArg, type=(T, T)->(T)}]  
  )  
  someFunction(f SomeFArg, a T, b T)->(T) = {}  
}
```

`SomeFArg` belongs to the generic declaration context established by `@co.dap.generic`; it is not a free-standing parameterized alias.

Rank-2: The function parameter is itself polymorphic (higher-rank)

The passed function stays generic **inside the callee**. The binder list of `co.polymorphic(...)` belongs to a named `co.type`; the consuming function does not own those binders and is not made generic merely by accepting the named polymorphic type.

Named polymorphic type //someGen4.unit.fol

```
_ co.unit = {  
  SomeFArg co.type = co.polymorphic({T}, (T, T)->(T));  
  
  // T belongs to SomeFArg; this consumer is not itself generic.  
  someFunction(f SomeFArg)->(co.int) = {}  
}
```

Returning Generic Functions

Rank-1 return

//someGen6.unit.fol

```
_ co.unit = {  
  @co.dap.generic(  
    types=[{name=T}],  
    aliases=[{name=Adder, type=(T)->(T)}]  
  )  
  makeAdder(a T)->(Adder) = {  
    adder Adder = (b T)->(T) {  
      $=> a + b;  
    };  
    $=> adder;  
  }  
}
```

Rank-2 return — returning a polymorphic function

//somGen7.unit.fol

```
_ co.unit = {  
  PolyIdentity co.type = co.polymorphic({T}, (T)->(T));  
  
  identity PolyIdentity(x) = x;
```

```

    makeIdentity()->(PolyIdentity) = {
        $=> identity;
    }
}

```

`PolyIdentity` owns the polymorphic binder and callable shape. `identity` implements that named callable type without a separate `@co.dap.generic` declaration. `makeIdentity` merely returns the already-polymorphic named callable. FoLang has no general anonymous-function literal that can introduce an independent polymorphic binder set.

Rank-3: A Parameter is Itself a Rank-2 Function

Rank-3 uses named `co.type` layers. Each `co.polymorphic(...)` binder list remains inside the type declaration that owns it; consuming and returning functions use the resulting named types.

Named type layers //someGen9.unit.fol

```

_ co.unit = {
    Rank2FnType co.type = co.polymorphic({T}, (T, T)->(T));
    Rank3ArgType co.type = (Rank2FnType)->(co.int);

    applyRank2(f Rank3ArgType, value Rank2FnType)->(co.int) = {
        $=> f(value);
    }
}

```

Rank-3 return

//somGen10.unit.fol

```

_ co.unit = {
    Rank2FnType co.type = co.polymorphic({T}, (T)->(T));
    Rank3ConsumerType co.type = (Rank2FnType)->(co.int);

    consumeRank2(f Rank2FnType)->(co.int) = {
        $=> f(42);
    }

    makeRank2Consumer()->(Rank3ConsumerType) = {

```

```
    $=> consumeRank2;
  }
}
```

Passing a Polymorphic Type Object to a Generic Function

FoLang makes no value/object distinction that excludes types: every type is an object. A marker introduced by `@co.dap.generic(types=[{name=T}])` may therefore resolve to `co.type`, and the corresponding parameter value may be a type object such as `co.int`, `Employee`, or `PolyId`.

In this subsection, `Box(T)` denotes a parameterized `co.type` constructor; `Box` is not an annotation-declared generic struct or class. The polymorphic shape is first named by the required `co.type` declaration, and that named type object is passed through the ordinary call syntax:

```
//somGen11.unit.fol
```

```
_ co.unit = {
  Box(T) co.type = co.variants(Boxed(T));
  PolyId co.type = co.polymorphic({U}, (U)->(U));

  @co.dap.generic(
    types=[{name=T}],
    aliases=[{name=BoxOfT, type=Box(T)}]
  )
  box(x T)->(BoxOfT) = {}

  someFun()->() = {
    result := box(PolyId);
    // T resolves to co.type; x contains the PolyId type object.
  }
}
```

The call above is ordinary generic inference over a type object. It does not bind `T` to the polymorphic callable type represented by `PolyId`; the argument value is the type object `PolyId`, so `T` resolves to `co.type`. No special generic metadata is involved.

Generic Binding to a Polymorphic Callable Type

When the argument is instead a callable value whose resolved type is a named polymorphic callable type, generic inference binds the generic marker directly to that resolved type. No opt-in flag is required because the callable’s type already carries the complete classification.

```
_ co.unit = {
  Box(T) co.type = co.variants(Boxed(T));
  PolyId co.type = co.polymorphic({U}, (U)->(U));

  identity PolyId(value) = value;

  @co.dap.generic(
    types=[{name=T}],
    aliases=[{name=BoxOfT, type=Box(T)}]
  )
  box(x T)->(BoxOfT) = {}

  someFun()->() = {
    result := box(identity); // T = PolyId
  }
}
```

Here the frontend resolves `identity` as a named function whose callable type is `PolyId`. Because `PolyId` resolves to `co.polymorphic({U}, (U)->(U))`, `T` is bound to `PolyId` directly. FoLang does not require or permit a second metadata switch to repeat that fact. In type-theory terminology this permits a generic variable to range over a polymorphic type, but in FoLang the decision is entirely type-driven.

The named-type rule still applies. A complex polymorphic type expression must be named before it participates in ordinary type use or generic substitution. The inline spelling below therefore remains invalid:

```
result := box(co.polymorphic({U}, (U)->(U))); // compiler error: use PolyId
```

Generic Function Rank Support Matrix

Scenario	Allow?	Notes
Rank-1 generic parameter	 Yes	Declare markers and derived aliases in <code>@co.dap.generic</code>

Rank-1 generic return	✓ Yes	Parameters and results use named aliases when their type is derived
Rank-2 param via named <code>co.type</code>	✓ Yes	<code>co.polymorphic(...)</code> owns the binders inside the named type declaration; the consumer uses that type
Rank-2 param via a <code>co.function</code> value declaration	✗ Compiler error	Function objects are concrete values; define a named <code>co.type</code> with <code>co.polymorphic(...)</code> instead
Rank-2 return via named <code>co.type</code>	✓ Yes	Return a named callable that implements the polymorphic callable type
Implement a named polymorphic callable type	✓ Yes	<code>identity PolyId(value) = value;</code> inherits parameter/result types and polymorphic binders from <code>PolyId</code>
Rank-3 via named <code>co.type</code> layers	✓ Yes	Higher-rank structure is expressed by composing named types
Rank-3 return	✓ Yes	Return a named callable matching the named Rank-3 type
Rank-3 via a <code>co.function</code> value declaration	✗ Compiler error	Same rule as Rank-2; function objects are concrete
Pass a named polymorphic type object to a generic parameter	✓ Yes	<code>T</code> resolves to <code>co.type</code> ; the parameter value is the named type object
Pass a value whose type is a named polymorphic type	✓ Yes	<code>T</code> resolves directly to that named polymorphic type from the value's resolved callable type; no classification flag is required
Inline <code>co.polymorphic(...)</code> call	✗ Compiler error	Name the polymorphic type with <code>co.type</code> and pass that

argument	name
----------	------

`@co.dap.generic(types=[...])` remains the generic-marker mechanism for declarations that define their own generic signature. `co.polymorphic(...)` separately introduces binders owned by the named `co.type` value it constructs. An ordinary named function that implements such a callable type inherits that type's binder and callable contract and therefore does not redeclare the same binders with `@co.dap.generic`. Functions express higher-rank parameters and returns by using these named polymorphic callable types. See [Polymorphic Types](#), [Named Callable-Type Implementation](#), and [Generic Declarations and Parameterized Types](#).

Generics Inheritances and Types

This is in conceptual stage not supported.

- A) Abstract vs concrete type members
- B) Path-dependent types
 1. Type-level projection
 2. Path-dependent In FoLang how it would be

Polymorphic Types

`co.polymorphic(...)` is the built-in RHS type-expression constructor for a named polymorphic type. It is not a keyword and does not introduce a separate declaration category. The enclosing declaration remains an ordinary `co.type` declaration.

Canonical form:

```
PolyId co.type = co.polymorphic({T}, (T)->(T));
```

The first argument, `{...}`, introduces the polymorphic type binders owned by that one type expression. The second argument is the polymorphic type body and may reference those binders. Binder scope begins with the binder set of that constructor and ends with the enclosing `co.polymorphic(...)` expression. The binders do not become declarations in the surrounding unit, package, function, class, or other lexical scope.

The resolved type declaration is authoritative for polymorphic classification. A symbol whose resolved callable type is a named `co.polymorphic(...)` type is polymorphic by virtue of that type; a symbol whose resolved callable type is concrete is concrete. FoLang does not use a separate source flag to assert, enable, disable, or override this classification. For example, after `identity PolyId(value) = value;` resolves, the compiler knows that `identity` is polymorphic because its declared callable type is `PolyId`.

`co.polymorphic(...)` is valid only as the complete RHS type expression of a `co.type` declaration. A polymorphic type must therefore be named before it is used in a field, variable, parameter, receiver, function result, annotation value, or ordinary call argument.

```
SomeFArg co.type = co.polymorphic({T}, (T, T)->(T));
PolyId    co.type = co.polymorphic({U}, (U)->(U));

// Rank-2 parameter: the consumer is not itself generic.
someFunction(f SomeFArg)->(co.int) = {}

// The named callable type supplies the polymorphic contract.
identity PolyId(value) = value;

// Rank-2 return: returns an already-polymorphic named callable.
makeIdentity()->(PolyId) = { $=> identity; }
```

A declaration that defines its own generic signature continues to use `@co.dap.generic`. By contrast, an ordinary named function implementing a named `co.polymorphic(...)` callable type inherits that callable contract and does not redeclare its binders with `@co.dap.generic`. Restricted lambdas do not introduce a `co.polymorphic(...)` binder set of their own.

Inline polymorphic type construction is invalid in ordinary type-use or value-expression positions:

```
consume(
  value co.polymorphic({T}, (T)->(T))
)->(); // compiler error: name the polymorphic type first

result := box(
  co.polymorphic({U}, (U)->(U))
); // compiler error: pass the named type object instead
```

The valid type-object form is:

```
PolyId co.type = co.polymorphic({U}, (U)->(U));  
result := box(PolyId);
```

A named function may implement the same callable type directly:

```
identity PolyId(value) = value;
```

Quick Reference

Form	Status	Context
<code>name co.type = co.polymorphic(...);</code>	✓ Allowed	Named polymorphic type definition
<code>identity PolyId(value) = value;</code>	✓ Allowed	Named function implements the callable contract and binders owned by <code>PolyId</code>
polymorphic binder syntax in an ordinary declaration head	✗ Compiler error	Define the binder set in a named <code>co.polymorphic(...)</code> type or use <code>@co.dap.generic</code> for a declaration that owns its own generic signature
inline <code>co.polymorphic(...)</code> in an ordinary parameter/result type	✗ Compiler error	Declare a named polymorphic <code>co.type</code> and use that name
inline <code>co.polymorphic(...)</code> as an ordinary call argument	✗ Compiler error	Pass the named polymorphic type object instead

The rule in one sentence: `co.polymorphic(...)` owns the polymorphic callable contract; an ordinary named function may implement that named contract directly without redeclaring its binders.

Generic declarations that own their own generic parameter set are supported only for structs, classes, ordinary functions, and ordinary methods, and introduce those parameters through `@co.dap.generic`. An ordinary function implementing

a named polymorphic callable type is different: the binder set is owned by the named `co.type`, so the implementation does not redeclare it.

`OperatorOverloadDecl` is deliberately excluded even though an operator implementation has a callable shape. A declaration carrying `@co.dap.operator` must not also carry `@co.dap.generic`. A generic class or struct may own an operator, but the operator itself remains non-generic and is associated with the canonical owner declaration rather than with operator-level type parameters.

The following declaration-head generic forms are invalid:

```
// Cache.fol
_(T) co.module = {}           // compiler error
// operations.unit.fol
_(F(_)) co.unit = {}         // compiler error
Callback(T) co.delegate = (T)->(T); // compiler error
```

A parameterized `co.type` declaration is a separate parameterized-type form and does not use `@co.dap.generic`:

```
// option.unit.fol
_ co.unit = {
  Option(T) co.type =
    co.variants(Some(T), None);
}
```

Generic structs and classes remain file-backed primary declarations. Their names come from filenames:

```
// LinkedList.fol
@co.dap.generic(types=[{name=T}])
_ co.struct = {
  value T;
  next  LinkedList;
  prev  LinkedList;
}
```

A generic struct can be used to achieve type-level-function behavior. Because a struct is a pure data declaration without methods, inheritance, or other object-oriented behavior, its generic application can be modelled as a mapping from type arguments to a specialized pure-data type:

```
Pair : (Type, Type) -> Type
Pair(co.int, co.string) -> specialized Pair type
```

This is a semantic model, not a separate FoLang declaration category. `Pair` remains an ordinary `@co.dap.generic` struct, and its concrete application must be named through `co.type` before use:

```
IntStringPair co.type = Pair(co.int, co.string);
```

Generic structs are therefore not pure type-level functions: they introduce ordinary struct types and participate in the normal struct type system. They merely provide the same useful type-to-type mapping behavior during generic specialization.

```
// linked_list_values.unit.fol
_ co.unit = {
  IntLinkedList co.type = LinkedList(co.int);
  someFun()->()={
    myIntList IntLinkedList;
  }
}
```

```
// Employee.fol
@co.dap.generic(types=[{name=T}, {name=R}])
_ co.class = {
  id T;
  name R;
}
```

```
// employee_types.unit.fol
_ co.unit = {
  EmployeeIntString co.type =
    Employee(co.int, co.string);
  emp EmployeeIntString;
}
```

Generic functions use the same annotation but are declared inside a legal function-owning context such as an ordinary unit, class, or companion unit:

```
//sommGen1.unit.fol
```

```
_ co.unit = {
```

```

@co.dap.generic(types=[{name=T}, {name=R}])
add(a T, b T)->(R) = {
    ...
}
someFun()->()={
    add_int_int := add.withTypes(co.int,co.int);

    //    or

    add_int_int co.function =  add.withTypes(co.int,co.int);

    k := add_int_int(12,10);
}
}

```

Specialization

`@co.dap.specialize` to specialize generics for specific types upfront
 //sommGen2.unit.fol

```

_ co.unit = {

    @co.dap.generic(
        types=[
            {name=T}
        ],
        requires=[
            co.Add(left=T, right=T, result=T)
        ]
    )
    add(a T, b T)->(T) = {
        $=> a + b;
    }
}

```

for the above generic want to specialize for `co.int` //sommGen5.unit.fol

```

_ co.unit = {
    @co.dap.specialize(
        target=add,
        types=[
            {name=T, type=co.int}
        ]
    )
}

```

```

    )
    addInt(a co.int, b co.int)->(co.int) = {
        $=> co.intrinsic.intAdd(a, b);
    }
}

```

FoLang supports partial specialization. The following example specializes only part of a generic declaration's type mapping: `//sommGen7.unit.fol`

```

_ co.unit = {
  @co.dap.generic(
    types=[
      {name=T},
      {name=R}
    ]
  )
  transform(value T)->(R) = {
    ...
  }
}

```

`//sommGen8.unit.fol`

```

_ co.unit = {
  @co.dap.specialize(
    target=transform,
    types=[
      {name=T, type=co.string},
      {name=R}
    ]
  )
  transformString(value co.string)->(R) = {
    ...
  }
}

```

Fields of `@co.dap.specialize`

Attribute	Values
target	Fully qualified generic target name; when qualification is omitted, resolution begins in the current package according to the normal lookup rules.

types	Type-resolution mappings for the specialization.
priority	Optional specialization priority where the specialization rules permit it.
strategy	Specialization strategy, such as <code>intrinsic</code> , where supported.

Effects and Handling Effects

FoLang handles recoverable execution failures as typed **effects** without `try`, `catch`, `throw`, or `finally` statements. Effect declaration and effect handling have deliberately separate owners:

Construct	Owner and placement	Purpose
<code>@co.dap.effects</code>	Callable declaration or definition	Declare effects the callable may emit
<code>@co.dap.onEffect</code>	Immediately before a call expression	Handle effects from that invocation
<code>@co.dap.defer</code>	Inside a callable	Register unconditional completion work

The defining library knows which effects an exported callable may emit, but it does not know the caller's logging, retry, cleanup, resource, or return policy. The definer therefore publishes only `@co.dap.effects`; the caller chooses `@co.dap.onEffect` handlers and resolutions independently at each ordinary call site. An execution-model invocation is the deliberate exception: the caller may register handlers, but its resolution is fixed implicitly to `return_with_error` because effects cannot propagate across that execution boundary.

A non-handler library implementation may still use `@co.dap.onEffect` for calls made inside its own implementation. At those sites the library is itself the caller. It may not attach `@co.dap.onEffect` to an exported callable definition to impose handling on external callers. Effect-handler modules are the explicit exception: `@co.dap.onEffect` is forbidden throughout them.

Effect Channel and Error Result Channel

An emitted effect and a returned `co.error` value use different channels:

Channel	Meaning
Effect channel	Non-local typed failure that must be handled, converted, or propagated
Ordinary result channel	A normal returned value, including a <code>co.error</code> value

Returning a non-empty `co.error` does not emit it as an effect. `return_with_error` explicitly consumes an effect at a call site, converts it to a `co.error` object, and returns it from the enclosing caller through the ordinary result channel.

An error object may carry its category, origin, message, source location, cause, and backend/runtime diagnostics. Native or backend exceptions must be translated into typed FoLang recoverable effects before FoLang resolution begins; the translated type need not have been advertised by `@co.dap.effects`.

Recoverable Error Contract, Mixin, and Effect Emission

Every recoverable effect value is an instance of a class satisfying the standard `co.error` interface. Error classes may add category-specific fields and helper methods. The standard `co.AbstractError` mixin supplies common state and concrete behavior for that interface. Custom error classes normally compose the mixin, declare `co.error` as an interface, and add their own information:

```
@co.dap.oops(
  interfaces=[co.error],
  mixins=[co.AbstractError]
)
_ co.class = {
  // category-specific error fields and methods
}
```

`co.AbstractError` is declared as `_ co.mixin`, not as a class and not as a distinct abstract-class kind. As a mixin it participates only in composition and has no independent class-instance construction semantics. Its exact matching concrete methods may satisfy `co.error` interface slots when the consuming error class composes the mixin. If an explicit source mapping or remapping is required, the consuming class uses `@co.dap.implement(type=co.error)` according to the ordinary method- resolution rules. Interface method declarations themselves do not carry that annotation.

`@co.dap.implement` and `@co.dap.implementation` remain distinct:

Form	Purpose
<code>@co.dap.implement</code>	Satisfy an interface, trait, or mixin method contract
<code>@co.dap.implementation</code>	Bind an authorized bodyless standard declaration to a backend-neutral runtime operation

FoLang source and HIR use one backend-neutral effect-emission operation. Its standard declaration is conceptually:

```
@co.dap.implementation(  
    kind=co.dap.implementationKind.runtime,  
    operation=co.runtime.operation.effect.emit  
)  
emit(error co.error)->();
```

The operation accepts only a `co.error` value. Each backend may implement it using native exceptions, tagged control results, runtime unwinding, task completion, or another mechanism, but it must preserve FoLang matching, handler, resolution, and defer semantics. A backend or native-library failure that is recoverable in FoLang must first be translated into a class instance satisfying `co.error`; a raw backend exception must not enter FoLang handling directly.

Fatal runtime failures are outside this hierarchy. They do not satisfy `co.error`, cannot appear in `@co.dap.effects` or `@co.dap.onEffect`, and cannot be continued, retried, returned as an error, propagated as a FoLang effect, or delivered to an effect handler. They terminate the affected application or runtime execution according to runtime policy.

`@co.dap.effects`: Definer Contract

`@co.dap.effects` is informative public symbol metadata, not a closed or enforced catch-or-declare contract. Its `emits=[...]` list advertises effect types currently known to possibly leave the callable:

```
@co.dap.effects(  
    emits=[  
        co.DatabaseError,
```

```

        co.NetworkError
    ]
)
loadCustomer(id co.int)->(Customer) = {
    ...
}

```

It is valid on ordinary callable definitions, bodyless declarations, interface methods, native declarations, and other ordinary callable contracts. It is invalid on an execution-model declaration because that declaration converts all recoverable effects at its public boundary. It is also invalid on a call, expression, statement, field, or non-callable declaration.

Every `emits` entry must resolve to an accessible class type satisfying `co.error`. Duplicate canonical types are invalid, and a broader declared error class covers its subtypes. The list is an open set of known possible effects, not an exhaustive closed effect row and not a claim that every listed effect occurs on every invocation.

Absence of `@co.dap.effects`, or an empty `emits` list, means that the source explicitly advertises no effects. It does not mean that the callable is effect-free. An undeclared effect may still arise and propagate like an unchecked exception.

The compiler computes the callable's known outgoing-effect metadata from its explicitly declared effects plus statically known unhandled callee effects, minus effects consumed at call sites. When a statically selected concrete handler has a known outgoing effect, the compiler adds the possibility of `co.GenericError`, because that is the value exposed beyond the failed handler pipeline. It serializes this known set in exported `.foIenc` symbols. Unknown runtime effects remain possible even when absent from that metadata.

Because the contract is open, an implementation or override may advertise additional effects not named by its interface or overridden declaration. Effect metadata informs callers and tooling but does not distinguish overloads or make undeclared propagation invalid.

`co.EffectHandler`

The standard library defines the handler-module contract:

```

// co.EffectHandler
_ co.signature = {

```

```
    handle(error co.error)->();  
}
```

Every `handlers=[...]` entry must be an accessible singleton module matching this signature. It is not a constructed class instance or arbitrary function reference. The signature limits only the module's public contract; a handler may use private functions, associated singleton objects, configuration, registries, locks, thread-local state, and other accessible facilities.

`co.EffectHandler` constrains only the structural member signature. The signature itself does not declare handler effects. Each concrete handler implementation may independently advertise effects on its actual `handle` implementation:

```
@co.dap.effects(  
    emits=[co.LogWriteError]  
)  
handle(error co.error)->() = {  
    writeLog(error);  
}
```

Omitting `@co.dap.effects` from that implementation does not prohibit an unexpected effect. It means only that the implementation advertises no effect explicitly. Since `handlers=[...]` contains statically resolved concrete module references, the compiler can use their declared and inferred known effects to include the possibility of `co.GenericError` when analysing the caller.

An effect-handler module must not use `@co.dap.onEffect`, either on its public `handle` implementation or at a call site inside one of its private functions. Handlers are the terminal declarative stage of the current handling pipeline; allowing them to start another local handler pipeline would permit recursive handling chains with no clear semantic boundary. A handler should therefore use ordinary values, variants, explicit checks, or non-effectful operations for problems it can recover from locally. If it nevertheless emits a recoverable effect, the handler-failure rules below apply. `@co.dap.effects` remains valid on the concrete `handle` implementation because it advertises such possible outgoing handler effects without handling them locally.

`@co.dap.onEffect` : Caller Policy

`@co.dap.onEffect` prefixes exactly one call expression. It defines how that specific invocation handles matching effects from its callee:

```

showCustomer(id co.int)->(co.error) = {
    customer Customer =
        @co.dap.onEffect(
            co.DatabaseError={
                handlers=[
                    DBConnectionCloseHandler,
                    LogErrorHandler
                ],
                resolution=return_with_error
            },
            co.NetworkError={
                resolution=propagate
            }
        )
    loadCustomer(id);
}

```

It is invalid on a callable definition or declaration, and it does not create a statement-oriented `try/catch` region. The prefix belongs to the immediately following call AST node. For nested calls, it handles only that decorated call:

```

outer(
    @co.dap.onEffect(
        co.NetworkError={resolution=propagate}
    )
    inner()
);

```

The policy above applies to `inner()`, not to `outer(...)`. A separately placed `@co.dap.onEffect` is required to handle effects emitted by `outer`.

Each top-level field name must resolve to a valid effect type. It need not appear in the callee's advertised `emits` list because that list is open and an unexpected effect may still occur. A call-site entry for a non-advertised type is valid proactive handling, although tooling may identify it as not currently advertised by the callee. Keys normalize to canonical type identity; duplicates are invalid. When multiple entries match, the most specific type wins.

For an ordinary invocation, each effect record contains exactly one singular `resolution=` field. A record may additionally contain an optional non-empty ordered `handlers=[...]` list. When handlers are present, all of them execute

before the resolution. The absence of `handlers` does not imply propagation: a matching ordinary-call record without handlers still applies its declared resolution.

For an invocation of a callable declared with `@co.dap.executionmodel`, an effect record instead contains a required non-empty `handlers=[...]` list and must omit `resolution`. Its resolution is fixed by the language to `return_with_error`. Supplying any explicit resolution, including `return_with_error`, is a compile-time error. The call-site annotation remains useful because it registers application handlers even though the developer cannot change the execution-boundary resolution.

For an ordinary invocation, an effect with no matching `@co.dap.onEffect` entry propagates from the enclosing caller by default. A statically known unmatched effect is included in the caller's computed outgoing-effect metadata, and an unknown runtime effect propagates even when it appears in no `@co.dap.effects` list. For an execution-model invocation, an unmatched effect cannot propagate across the execution boundary; it is converted to the callable's ordinary `co.error`-compatible result without running call-site handlers.

There is no `effects={...}` wrapper, no `resolutions=[...]` list, and no `invoke` resolution.

Execution-Model Call-Site Policy

An execution-model declaration must expose exactly one result position compatible with `co.error`. The first unhandled recoverable effect that reaches its execution boundary is retained as the primary error object and is returned through that ordinary result position. Other ordinary result positions that the failed execution did not produce receive `co.const.none` only when those positions admit none. A refinement/dependent result position makes this failure-conversion shape invalid and must be redesigned with an explicit initialized alternative, commonly a variant.

The caller may attach ordered handlers to a particular submitted invocation. For execution governed by `co.cPCA`, the prefix is placed on the canonical submission call and its policy is bound to the statically selected execution-model target:

```
// appl.fol
SubmitParams  co.type = co.List(co.any);
SubmitResults co.type = co.List(co.any);
```

```

@co.dap.onEffect(
  co.DatabaseError={
    handlers=[
      DBConnectionCloseHandler,
      LogErrorHandler
    ]
  }
)
co.cpa.submit(
  loadCustomerAsync,
  params=SubmitParams{id},
  results=SubmitResults{customer, error}
);

```

The omitted resolution is not default propagation. In this context it denotes the mandatory implicit `return_with_error` policy. These records are invalid:

```

co.DatabaseError={resolution=continue}
co.DatabaseError={resolution=propagate}
co.DatabaseError={resolution=return_with_error}
co.DatabaseError={} // no handler and no caller policy to add

```

The handlers execute synchronously and sequentially in the execution context that receives the effect, before completion is published to the submitting caller. The annotation registers statically resolved singleton modules with the invocation; it does not require the submitting caller's stack to remain active. Unmatched effects receive the same implicit conversion without handlers.

For concurrent or parallel execution, "first error" means the first effect observed by the applicable execution-model runtime. Unless a particular model defines deterministic observation priority, scheduling may affect which of simultaneously occurring effects becomes primary.

Resolution Records

The available resolutions are:

- `continue`
- `retry`
- `return`
- `return_with_error`

- `propagate`

Examples:

```
co.ValidationError={
  resolution=continue
}

co.NetworkError={
  handlers=[LogRetryHandler],
  resolution=retry,
  retry={
    max_attempts=3,
    on_exhausted=propagate
  }
}

co.DatabaseError={
  handlers=[DBConnectionCloseHandler, LogErrorHandler],
  resolution=return_with_error
}
```

Invalid records include:

```
co.DatabaseError={} // missing resolution

co.DatabaseError={
  resolution=[continue, return] // resolution is singular
}

co.DatabaseError={
  resolution=continue,
  resolution=return // duplicate resolution
}

co.DatabaseError={
  handlers=[],
  resolution=propagate // an explicitly supplied handlers list is empty
}

co.DatabaseError={
  resolution=retry // missing retry configuration
}
```

Ordered Handler Execution

Handlers execute synchronously and sequentially in declared order. Every handler receives the same original `co.error`, and each `handle` call must complete before the next begins. The implementation must not implicitly reorder, parallelize, schedule, or detach the list.

For an ordinary invocation, handlers run on the caller thread and execution context receiving the effect. For an execution-model invocation, they run in the execution context that observes the effect before completion is published to the submitting caller. This permits handler modules to use state accessible in the context where handling actually runs. An ordinary function, class object, incompatible module, inaccessible module, duplicate module, or empty list is invalid.

If a handler emits a recoverable effect, the remaining handlers for the original effect are skipped and the original effect's resolution is not applied. Because `@co.dap.onEffect` is forbidden throughout an effect-handler module, the handler cannot start a nested local handler pipeline for the new effect. A known handler effect makes `co.GenericError` part of the caller's computed outgoing-effect metadata; an undeclared handler effect remains valid and follows the same runtime rule without necessarily appearing in advance metadata.

Handler Failure and `co.GenericError`

Let `E0` be the original effect being handled, `H` the handler currently executing, and `E1` a recoverable effect emitted while `H` handles `E0`. The standard library creates one `co.GenericError` class instance that satisfies `co.error` and records:

- the original handled error `E0`;
- the handler failure `E1`;
- the canonical identity of `H`; and
- available causal, source, and runtime diagnostic context.

Creation of this wrapper terminates the current handler pipeline immediately. The remaining handlers are skipped, the resolution selected for `E0` is cancelled, and neither `E0` nor the new `GenericError` is submitted to the same handler list again.

The boundary then determines what happens to the wrapper:

Context	Result of handler failure
---------	---------------------------

Ordinary call	<code>co.GenericError(E0, E1, H)</code> propagates from the enclosing caller to its caller
Execution-model invocation	<code>co.GenericError(E0, E1, H)</code> is returned through the execution model's ordinary error-result position; otherwise unproduced none-admitting results receive <code>co.const.none</code> , while refinement/dependent result positions make that substitution invalid

`co.GenericError` is used only for a recoverable effect emitted by a handler while processing another recoverable effect. It is not created for an ordinary `propagate`, an ordinary `return_with_error`, or the normal implicit `return_with_error` conversion at an execution-model boundary. When handlers complete successfully, those paths preserve the original error object `E0`. A higher ordinary caller may handle the propagated `GenericError` at its own call site according to ordinary rules.

A fatal failure inside a handler is not `E1`, is never wrapped in `co.GenericError`, and terminates the application immediately like every other fatal failure.

Resolution Semantics

Resolution	Consumes effect	Behavior at the decorated call
<code>continue</code>	Yes	Skip the failed call result and continue after the call
<code>retry</code>	Conditionally	Invoke the same callee again until success or exhaustion
<code>return</code>	Yes	Exit the enclosing caller normally
<code>return_with_error</code>	Yes	Convert to <code>co.error</code> and return it from the enclosing caller
<code>propagate</code>	No	Emit the effect from the enclosing caller

`continue` does not mean retry. It resumes after the decorated invocation. If the failed call owes one or more values to its surrounding expression, every missing result position receives the universal `co.const.none` value. An assignment therefore completes with none in the affected destination rather than becoming a compile-time error.

`return` exits the enclosing caller, not the failed callee. It is directly valid for a unit-returning caller. In a non-unit caller, every ordinary result position not otherwise produced by the resolution receives `co.const.none` only if that position admits none. A missing refinement/dependent result makes the resolution invalid.

`return_with_error` exits the enclosing caller. That caller's return signature must contain exactly one position compatible with `co.error`. Additional result positions not otherwise produced receive `co.const.none` only if they admit none. An unproduced refinement/dependent result position, or a missing or ambiguous error-compatible result, is a compile-time error.

`propagate` emits the effect from the enclosing caller. Explicit `propagate` is useful when handlers must run before propagation; without a matching record, propagation occurs automatically without handlers. A known propagated effect is advertised through computed outgoing-effect metadata even if the caller did not spell it in a source annotation.

Retry

`retry` repeats only the decorated invocation, never the complete enclosing caller. It requires exactly one `retry={...}` record:

```
@co.dap.onEffect(  
  co.NetworkError={  
    handlers=[LogRetryHandler],  
    resolution=retry,  
    retry={  
      max_attempts=3,  
      on_exhausted=return_with_error  
    }  
  }  
)  
sendRequest();
```

`max_attempts` is a positive compile-time integer and includes the initial invocation. `max_attempts=3` therefore permits at most three total invocations, not one initial invocation plus three retries.

`on_exhausted` must be exactly one of `continue`, `return`, `return_with_error`, or `propagate`; it cannot be `retry`. It is applied to the final error after the final failed attempt. All ordinary validity rules for the selected exhaustion action still apply.

The caller evaluates the decorated call's receiver and argument expressions once before the first attempt and captures their resulting values. Retries use those same values according to ordinary value/reference parameter semantics; FoLang does not deep-copy mutable arguments. A caller that needs fresh argument evaluation must move that computation into the retried callee.

For each failed attempt, handlers run in declared order before the next attempt or the exhaustion action. A handler failure aborts the retry sequence. Each callee invocation is a distinct invocation: defers registered inside the callee execute when that attempt exits before another attempt begins. Defers registered by the enclosing caller do not execute merely because one call attempt failed or was retried.

Retry repeats observable callee behavior. The developer must therefore ensure that the callable is idempotent, transactional, compensated, or otherwise safe to invoke repeatedly.

Caller-Owned Database Handling

An exported database library declares effects but does not register application handlers:

```
@co.dap.effects(  
  emits=[co.DatabaseError]  
)  
executeDatabaseOperation(command DBCommand)->>() = {  
  ...  
}
```

The application defines its own `DBConnectionCloseHandler` and `LogErrorHandler` modules conforming to `co.EffectHandler`. It may keep a connection pool and current connection in an associated singleton object or another caller-owned

facility. At the call site it chooses both handlers and their order:

```
runDatabaseWork(command DBCommand)->(co.error) = {
  @co.dap.onEffect(
    co.DatabaseError={
      handlers=[
        DBConnectionCloseHandler,
        LogErrorHandler
      ],
      resolution=return_with_error
    }
  )
  executeDatabaseOperation(command);
}
```

When the call emits `co.DatabaseError`, FoLang closes the caller-owned connection, logs the same original error, converts it to `co.error`, and returns it from `runDatabaseWork`. Another caller may choose retry or direct propagation for the same library operation without changing the library.

Static Effect Information and Default Propagation

At each call the compiler combines:

```
callee declared and inferred known effects
+ matching call-site @co.dap.onEffect entries
+ co.GenericError when concrete handlers have known outgoing effects
+ enclosing caller return signature
```

Effects consumed by `continue` or `return`, converted by `return_with_error`, or successfully eliminated by `retry` do not escape. An exhausted retry ending in `propagate`, an explicit `propagate`, and every unmatched callee effect do escape. Statically known escaping effects are included in the caller's computed outgoing metadata. Undeclared effects also escape but cannot be promised to callers in advance. When the enclosing declaration is an execution-model declaration, this analysis describes its internal implementation only: the public execution boundary converts the first escaping recoverable effect to its ordinary error result, so no effect is advertised outward.

At the outermost application/runtime boundary there is no caller to receive an unhandled effect. The runtime reports the effect and terminates the affected execution flow according to runtime policy.

`@co.dap.defer`

`@co.dap.defer` registers a callable for execution when the enclosing function or method actually exits. It performs unconditional completion work whether the exit is successful, ordinary, or effect-driven. It is not an effect handler, does not inspect the completion error, and does not choose among effect resolutions.

`@co.dap.defer` does not implicitly capture arguments through `$->args`. Values required by the deferred callable must be supplied/passed explicitly at registration.

```
someErrorFun(a co.int)->>() = {
    resource Resource = acquireResource();

    @co.dap.defer
    cleanup(resource Resource, originalArgument co.int)->>() = {
        releaseResource(resource);
        logArgument(originalArgument);
    }(resource, a);

    performOperation(resource);
}
```

The explicit arguments are evaluated and captured when execution reaches the defer registration. The deferred callable itself executes later. FoLang never injects a `co.error` object or any other completion-status binding into that callable. Values needed by deferred work must be declared as ordinary parameters and supplied explicitly at registration.

Defer Registration and Ordering

A defer is registered only when execution reaches it. A defer located after a failing operation is not registered when that failure prevents execution from reaching the defer. Completion work intended to cover the complete function must therefore be registered before operations that may terminate the function.

Every registered defer:

- executes exactly once when the enclosing function actually exits;

- executes after normal completion, ordinary return, `return`, `return_with_error`, or `propagate`;
- does not execute merely because an effect was handled with `continue` while the function remains active;
- participates in last-in, first-out ordering with other registered defers; and
- is not guaranteed after forced process termination, unrecoverable runtime shutdown, or a hardware failure that prevents further execution.

The completion order is:

```
1. execute the function body
2. resolve locally handled effects
3. determine that the function will exit
4. execute registered deferred callables in LIFO order
5. incorporate an error produced by a deferred callable
6. complete the ordinary return, return_with_error, or propagation
```

If an error already caused the function to exit, that first error remains the primary error. A later deferred-call error does not replace it and may be attached only as causal/diagnostic information supported by the primary error representation. If no primary error exists and a deferred callable emits an effect, that effect becomes the primary completion effect and propagates from the enclosing function unless it was handled at a call site inside the deferred callable. A statically known deferred-call effect is included in the enclosing function’s computed outgoing-effect metadata; an unknown effect still propagates. Remaining deferred callables continue to execute.

Effect-Handling Summary

Construct	Purpose
<code>@co.dap.effects</code>	Definer-owned open metadata advertising known effects an ordinary callable may emit
<code>@co.dap.onEffect</code>	Caller-owned policy attached to one invocation; execution-model calls permit handlers but use implicit <code>return_with_error</code>
<code>co.EffectHandler</code>	Standard signature requiring <code>handle(error co.error)->()</code>

<code>handlers=[...]</code>	Ordered list of developer-defined handler-module references
<code>co.GenericError</code>	Standard wrapper created only when a handler emits a recoverable effect while processing another recoverable effect
<code>continue</code>	Consume the effect, substitute <code>co.const.none</code> for missing results, and continue
<code>retry</code>	Repeat only the decorated invocation under a bounded retry policy
<code>return</code>	Consume the effect and terminate through the ordinary return channel
<code>return_with_error</code>	Consume and return the current error as an ordinary result
<code>propagate</code>	Forward the effect from the enclosing caller; known effects remain advertised upstream
<code>@co.dap.defer</code>	Execute error-independent completion work in LIFO order on every function exit

FoLang effect handling is therefore explicit, typed, lexically scoped, and declarative. It provides local recovery, early return, propagation, callbacks, and guaranteed completion work without statement-oriented exception syntax.

Reflections

The `co.meta` reflection form shown below is a dynamic-runtime facility and is valid only inside a `dynamicvmrt` capability domain; it does not grant `co.meta` access to ordinary application or packaged code.

```
@co.dap.reflection(enable=true, package="co.meta")

x co.int = 10;
x.reflect().getType();    //co.int
x.reflect().getValue();   //10;
x.reflect().getKind();    // value
```

Dynamic VM Runtime

The `@co.ddap.dynamicruntime` directive enables full access to the `co.meta` package. It is valid **only for source files in a `dynamicvmrt` capability domain**: a standalone `@co.dap.library(type=dynamicvmrt)` project or the project-local `components/dynamicvmrt/` component. In every permitted source file it must obey the category-wide [Directive Placement](#) rule and appear at file top level, never inside the `_ co.component` declaration or any nested declaration/body. Using `@co.ddap.dynamicruntime` in an executable application, packaged code, an application projected library/component, a `native` domain, or any other source context is a compiler error.

Within a valid `dynamicvmrt` domain, the directive enables dynamic class and type loading, monkey patching, runtime reflection, instrumentation, eval-based code execution, and other defined dynamic-runtime/metaprogramming capabilities through `co.meta`. These capabilities remain inside that projected dynamic-runtime boundary and do not automatically escape into ordinary application, packaged, or other library/component code.

1. runtime type creation from strings, streams, files, and other supported dynamic-runtime inputs
2. complete reflection and introspection
3. Runtime code modification add/remove/update methods etc.

When a library is marked `dynamicvmrt` and enables `@co.ddap.dynamicruntime`, the final binary includes the runtime support required to create and manage dynamic types, methods, and objects.

Dynamically created objects may interact with compiled types according to the dynamic-runtime boundary rules; ordinary compiled code does not gain unrestricted reverse access to dynamic-runtime facilities.

Through surface API's they connect to runtime and through runtime apis invoke the dynamic type and the results will be returned are compiled types only.

Runtime can directly provide handle to compiled types to dynamic code running inside it.

Loaders are the dynamic-runtime containers used to manage these runtime-created objects and types.

FoLang provides `BasicLoader`. A user-defined loader extends `co.meta.BaseLoader` using the ordinary loader declaration rules, for example:

```
//MySpecLoader.fol
```

```
@co.dap.extends(co.meta.BaseLoader)
_ co.loader={

}
```

The basic loader provides the operations required to create, update, delete, and manage runtime-created objects.

Loaders form a hierarchy. When a referenced runtime type is not found in the current loader realm, lookup proceeds through the base-loader chain and finally to the compiled-type environment.

Native Code

`@co.dap.native` marks a function or method declaration as a **native implementation declaration**. It does not grant native capability merely because the annotation is present. The annotation is valid only inside a `native` library/component domain when the installation permits the native capability.

A native library/component may use the `co.native` package to express low-level implementation such as assembly or machine-level operations through facilities including `co.native.asm` and `co.native.inline`. The same native domain also

owns foreign-function interoperability: extern declarations, foreign symbols, C/native ABI-compatible types, calling conventions, symbol linkage, pointer/address forms, and permitted FoLang-side marshalling or invocation code.

These facilities intentionally share one capability boundary. A foreign call may produce an ABI value, address, or pointer that native memory, platform, assembly, or runtime code consumes directly, and native code may prepare values for a foreign call without routing them through a second projected API. `ffi` therefore names an interoperability feature area/API family where useful; it is **not** a separate FoLang capability, library, or component kind.

The frontend preserves native and foreign-interoperability declarations and their metadata through the backend interchange contract. The reference backend demonstrates one implementation of that contract. A conforming third-party backend is not required to reproduce the reference backend's internal native-code lowering, ABI lowering, instruction representation, allocation strategy, marshalling implementation, or code-generation mechanism, but the externally observable behavior required by the specification must be preserved.

Native Functions

```
// native.unit.fol
```

```
@co.dap.native
nativeMethod(a co.int, b co.int)->(co.int) ={
    // native implementation
}
```

FFI and ABI

1. `@co.dap.declare`
2. `@co.dap.implementation`

Runtime-Operation Declarations

Backend-neutral runtime-operation marker

The exported standard `co.*` package may declare a callable, property, constructor, type/layout declaration, or another explicitly supported standard declaration without a FoLang body when `@co.dap.implementation` supplies the implementation classification required for that declaration kind. A runtime classification identifies a compiler-owned runtime operation. The marker defines the operation's backend-independent meaning; it does not name a C++ header, C++ function, JVM member, WASM import, linker symbol, or any other backend-specific implementation.

```
// standard-package private source: f0λ.out.Console
_ co.unit = {

    @co.dap.implementation(
        kind = co.dap.implementationKind.runtime,
        operation = co.runtime.operation.out.println
    )
    println(value co.string) -> ();
}
```

The declaration above contributes two linked identities:

```
public FoLang symbol = co.out.println
runtime operation    = co.runtime.operation.out.println
signature            = (co.string) -> co.unit
```

The currently defined standard implementation classification is:

kind	Meaning
<code>co.dap.implementationKind.runtime</code>	Lower through a registered backend/runtime handler identified by <code>operation</code> .

Annotations, Decorators, Pragmas, and Directives

The entries in this language-defined inventory form the current built-in metadata registry used for `@co.*` name recognition. The parser must recognize a language-owned metadata name through this predefined registry before accepting the

metadata application. Field/argument preservation and partial frontend field validation follow [Built-in Metadata Parsing](#). Every entry classified as `DIRECTIVE` follows the category-wide [Directive Placement](#) rule and is file-level only. Every entry classified as `PRAGMA` additionally follows the category-wide [Pragma Placement](#) rule and is valid only in an executable application's `src/app1.fo1`.

Kind		
<code>PRAGMA</code>	"@co.pdap.threadpool", "@co.pdap.schedularpool"	
<code>DIRECTIVE</code>	"@co.ddap.import", "@co.ddap.dynamicruntime", "@co.ddap.use", "@co.ddap.alias", "@co.ddap.d ynamicdispatch", "@co.ddap.o verload"	<code>@co.ddap.overload</code> is different from <code>@co.dap.overload</code> it has takes whether <code>paramtypes</code> or <code>paramandreturntypes</code> as attributevalue of <code>strategy</code>
<code>ANNOTATION</code>	"@co.dap.extend", "@co.dap.t emplate", "@co.dap.macro", "@co.dap.o perator", "@co.dap.annotation", "@co.dap.library", "@co.dap.native", "@co.dap.class", "@co.dap.static", "@co.dap.obj ect", "@co.dap.inline", "@co.dap.ctf e", "@co.dap.friend", "@co.dap.sealed", "@co.dap.extension", "@co.da p.override", "@co.dap.implem ent", "@co.dap.virtual", "@co.dap.abstract", "@co.dap.delegate", "@co.dap.typeclass", "@co.dap .matcher", "@co.dap.constructor", "@co.dap.oops", "@co.dap.ext ends", "@co.dap.hokrlt", "@co.dap.indexer",	//mop => meta object programming

	"@co.dap.generic", "@co.dap.comptime", "@co.dap.typefromvalue", "@co.dap.local", "@co.dap.private","@co.dap.p ublic","@co.dap.compose", "@co.dap.guard","@co.dap.pa ckage","@co.dap.protected", @co.dap.internal","@co.dap.e xport","@co.dap.eager", "@co.dap.lazy", "@co.dap.packed", "@co.dap.declare","@co.dap.i mplementation","@co.dap.sim d", "@co.dap.reflection", "@co.dap.mop","@co.dap.nes ted","@co.dap.inner","@co.da p.final","@co.dap.const","@co. dap.decorator","@co.dap.spec ialize","@co.dap.scope","@co. dap.symbol","@co.dap.with"	
DECORATOR	"@co.dap.before", "@co.dap.after","@co.dap.aro und", "@co.dap.effects", "@co.dap.onEffect", "@co.dap.defer","@co.dap.call able", "@co.dap.executionmodel"	

Built-in Metadata Parsing

Built-in directives, annotations, pragmas, and decorators under `@co.*` use one generic metadata application grammar:

```
annotation = "@", qualified-name,  
            [ "(", [ annotation-argument-list ], ")" ] ;
```

Every named field or attribute in a directive, annotation, pragma, or decorator uses `=`. The same rule applies recursively to records/maps nested inside a metadata application. `:` is not a metadata binder and its use in any metadata field or nested

metadata record is a syntax error.

```
@co.dap.generic(types=[{name=T, variance=covariant}]) // valid
@co.dap.generic(types=[{name=U, bound=co.number}]) // valid nested fields
@co.dap.implementation(
  kind=co.dap.implementationKind.runtime,
  operation=co.runtime.operation.out.println
) // valid backend-neutral runtime-operation marker
```

For `@co.dap.implementation`, `kind` classifies how a bodyless standard declaration is implemented and `operation` identifies the compiler-owned backend-neutral runtime operation. The `operation` value is resolved as a qualified operation symbol and preserved in `.fo1enc`/HIR; it is not target-language source text. The annotation is valid only on a declaration kind for which this specification permits a runtime-operation marker.

The reference intentionally contains no colon-bound `@co.*` metadata example; the normative metadata syntax above rejects such spellings.

This metadata rule does not change ordinary value syntax. Object field initializers and runtime map entries continue to use `:` according to their own grammar:

```
employee := Employee{name: "Rao"};
StringMap co.type = co.Map(co.string, co.string);
map := StringMap{"name": "Rao"};
```

The compiler maintains a predefined built-in metadata registry for language-owned `@co.*` forms. After reading the qualified metadata name, the parser must match the **complete name** against that registry. A registered enabled form is parsed according to the common metadata grammar and its applicable known frontend rules. A registered reserved/future form may be recognized and diagnosed as unsupported according to its registry entry. An `@co.*` metadata name that is not present in the predefined registry is a **parse error**; an unknown language-owned metadata name is never silently accepted.

```
@co.* metadata name
  -> lookup complete name in predefined built-in metadata registry
      -> registered and enabled          -> parse/collect metadata
application
```

-> registered but unsupported	-> unsupported-feature diagnostic
-> not registered	-> parse error

Recognition of the metadata **name** is strict; knowledge of every field is not. Once a built-in form has been recognized, the parser must collect and preserve the complete metadata application, including every supplied positional argument, named argument, field, attribute, and argument expression.

For each collected field or argument:

- when the frontend already has defined knowledge of that field, it may validate the applicable value shape, structural requirements, defaults, or other frontend rules;
- when the frontend has no defined knowledge or semantic handling for that field, the field is still accepted, collected, and preserved as parsed; lack of frontend field knowledge alone is not an error and does not block frontend artifact generation;
- a malformed argument expression or malformed metadata argument-list structure remains a syntax error under the common metadata grammar; and
- later semantic/backend stages may interpret preserved fields according to the applicable feature contract.

Accordingly, an unknown built-in **form name** and an unknown/unhandled **field of a known form** are deliberately different cases:

```
@co.dap.unknownForm(...)
  -> name absent from built-in registry
  -> parse error

@co.dap.generic(knownField=..., backendField=...)
  -> co.dap.generic is registered
  -> collect both fields
  -> validate fields the frontend understands
  -> preserve fields the frontend does not understand
```

User-defined metadata outside `co.*` is limited to annotations and decorators. Their qualified names are not looked up in the built-in registry; they are resolved through the ordinary imported/package symbol table and must resolve to a valid user-defined annotation or decorator declaration. An unresolved custom metadata

name is a name-resolution/compiler error. FoLang provides no user declaration construct for directives or pragmas, so custom directives and custom pragmas are not available.

Directive Placement

Directives are **source-file-level compiler metadata**. Every language-owned metadata form registered in the `DIRECTIVE` category, including every `@co.ddap.*` directive, must occur in the top-level metadata region of a source file. A directive cannot occur inside the body of a component, unit, class, struct, module, function, method, typeclass, instance, extension, matcher, annotation declaration, block, or any other declaration or nested lexical context.

The placement rule is structural and category-wide:

source-file top-level metadata region	-> directive permitted
inside <code>`_ co.component = { ... }`</code>	-> compiler error
inside <code>`_ co.unit = { ... }`</code>	-> compiler error
inside class/struct/module/typeclass/instance/etc. body	-> compiler error
inside function/method/extension/matcher body	-> compiler error
inside ordinary/nested block	-> compiler error

For file-backed declaration sources, directives appear before the file's primary declaration. For `src/component.fo1` and `components/<kind>/component.fo1`, directives therefore appear before the `_ co.component = { ... }` declaration, never inside its body. For the executable entry file `src/app1.fo1`, directives belong to the entry-file metadata preamble before the first non-metadata declaration or executable statement.

A directive's **semantic scope** is defined by the individual directive, but its **syntactic placement** is always file-level. For example, `@co.ddap.import` and `@co.ddap.alias` establish file-local bindings; `@co.ddap.use` establishes file-scoped activation; `@co.ddap.dynamicdispatch` is application-wide but is written only in the application entry-file preamble; and `@co.ddap.dynamicruntime` is valid only in a permitted `dynamicvmrt` capability source while still being written at that source file's top level.

A directive immediately preceding a primary declaration is not an annotation on that declaration. The compiler classifies the metadata name through the built-in registry first; entries classified as `DIRECTIVE` are attached to the current source-file/top-level semantic context rather than to an inner declaration AST node. Encountering a directive after entering a declaration/body context is a compile-time **metadata-placement error**.

For an ordinary package source file, the file has one primary top-level declaration. A directive in that file's metadata preamble may therefore configure or otherwise affect that primary declaration when the directive's own semantic contract says so. This does **not** make the directive a lexical declaration or a member of the primary declaration. The directive remains file-level compiler metadata.

Directives do not introduce names, lexical scopes, or symbol-table entries. The frontend records them separately with the source-file/top-level semantic context and consults them while validating or compiling the primary declaration. By contrast, symbols and symbol tables model name-bearing declarations and lexical visibility and may therefore exist recursively for classes, functions, methods, blocks, nested declarations, and other scoped constructs.

The common `@qualified.name(...)` surface syntax does not weaken this rule. After parsing the common metadata shape, the frontend classifies the built-in name by registry category. A form classified as `DIRECTIVE` or `PRAGMA` is accepted only through the file-preamble path; declaration/member/block metadata positions accept annotations or decorators instead. Thus a directive cannot be smuggled into a nested scope merely because directives and annotations share the same lexical shape.

```
SourceFileContext
├─ directives / pragmas / import metadata    // compiler metadata, not
symbols
├─ primary top-level declaration
│   └─ lexical/semantic contexts
│       └─ symbol tables
│           └─ nested symbol tables as required
```

This restriction applies automatically to future entries added to the language-owned `DIRECTIVE` registry unless the specification explicitly changes the category-wide rule.

Pragma Placement

Pragmas are **executable-application-owned configuration metadata**. Every language-owned pragma whose complete name is registered in the `PRAGMA` metadata category, including every `@co.pdap.*` form, is valid only in the executable application's fixed entry source, `src/app1.fo1`. This is a source-role/metadata-placement rule, not a distinct grammar production.

The following placement invariant applies to the entire pragma category:

executable application: <code>src/app1.fo1</code>	-> pragma permitted
application package source under <code>src/<package>/</code>	-> compiler error
project-local components/ <code><kind>/...</code>	-> compiler error
standalone projected application library	-> compiler error
standalone native library	-> compiler error
standalone dynamicvmrt library	-> compiler error
standalone packaged library	-> compiler error
exported/packaged package contexts	-> compiler error

A component, package, or library may document operational assumptions or recommended settings, but it cannot publish, export, inherit, or impose a pragma on its consumer. The executable application owns final application-wide policy. A pragma found outside `src/app1.fo1` is therefore a compile-time **metadata-placement error** after the built-in metadata name has been recognized.

This restriction applies automatically to future entries added to the language-owned `PRAGMA` registry unless the language specification explicitly changes the category-wide rule.

Misc

Method/Function Dispatch

FoLang uses compile-time static overload resolution and single-dispatch method semantics by default.

Enabling Dynamic Multiple Dispatch

`@co.ddap.dynamicdispatch(...)` is an **executable-application-only** semantic directive. It may be declared only in the top-level metadata preamble of the application rooted at `src/app1.fo1`, in accordance with [Directive Placement](#). It

cannot appear inside any declaration or block. Using it in `src/component.fo1`, in any standalone library, or in any `components/<kind>/component.fo1` is a compiler error. Components are not libraries, but they are equally forbidden from enabling this application-wide semantic mode.

Applicability and Widening

Dynamic multiple dispatch uses the **same applicability, nominal widening, specificity, and ambiguity rules as ordinary static overload resolution**. The difference is only the source of the selection tuple: static overload resolution uses compile-time argument types, while dynamic multiple dispatch uses actual runtime argument types.

Function overload

FoLang permits overloading for the ordinary function categories defined in [Functions and Methods](#); not every function-shaped declaration is overloadable. Result arity participates in the function contract as specified by the overload rules, while inner functions cannot be overloaded.

Note: FoLang treats each function-shaped declaration category as semantically distinct. Categories that share function-like syntax do not become interchangeable with ordinary functions or methods unless the specification explicitly defines that relationship.

Interface/Parent-Class-Specific Method Implementation and Override

FoLang provides interface-specific method implementation through `@co.dap.implement` and mixin/trait-specific overriding through `@co.dap.override`.

Without a type-specific selector, an implementation/override applies to all applicable inherited requirements with the same resolved signature according to the conflict rules.

To target a specific interface, trait, or mixin requirement, supply the applicable type selector: `type=*` for the default/all-target form, `type=SomeInterface` for one target, or `types=[A, B, C]` for multiple targets.

Lifecycle Members: `@@new`, `@@init`, and `::` Invocation

`@@new` and `@@init` are compiler-owned **class lifecycle members**, not ordinary methods. Every `co.class` receives the compiler-provided lifecycle implementations as part of the language-defined class base behavior. Semantically, those inherited implementations behave like protected lifecycle members: they are available to the compiler/runtime and to source contexts that are permitted to reach protected parent lifecycle behavior, but they are not automatically exposed as ordinary source-callable lifecycle APIs.

The `@@` prefix is declaration syntax for customization of this compiler-owned lifecycle family. A source declaration named `@@new` or `@@init` never creates a new lifecycle name.

Normal class construction does not require developer lifecycle customization. The compiler may use the inherited lifecycle machinery internally while processing ordinary class construction according to the composite-construction rules.

Developer lifecycle customization is configured through the existing generic annotation. No separate lifecycle annotation exists:

```
@co.dap.generic(  
    types=[{name=T}, {name=R}],  
    lifecycle=true  
)  
_ co.class = {  
    ...  
}
```

The `lifecycle` field is an optional field of `@co.dap.generic(...)`.

For a **generic class**, the compiler interprets it as follows:

<code>lifecycle field absent</code>	-> inherited compiler lifecycle remains; developer override/overload is forbidden
<code>lifecycle=false</code>	-> inherited compiler lifecycle remains; developer override/overload is forbidden
<code>lifecycle=true</code>	-> inherited compiler lifecycle remains; developer may override or overload <code>@@new</code> / <code>@@init</code>

For other declaration kinds that legally use `@co.dap.generic(...)`, such as generic structs, generic functions, and generic methods, the `lifecycle` field is **not considered**. Its presence does not by itself produce a diagnostic and has no lifecycle effect on those declarations.

The lifecycle customization rules are:

1. source declarations named `@@new` or `@@init` are valid only as members of `co.class`;
2. the enclosing class must be generic through a valid `@co.dap.generic(types=[...])`;
3. that same generic annotation must have `lifecycle=true`;
4. `lifecycle=true` grants permission to **override or overload** the existing compiler-owned lifecycle family; it does not create the lifecycle family and does not by itself expose any lifecycle call;
5. a normal/non-generic class cannot source-declare a lifecycle override or overload;
6. a generic class with `lifecycle` absent or `lifecycle=false` cannot source-declare a lifecycle override or overload;
7. each developer-defined lifecycle override/overload has ordinary FoLang accessibility. A public lifecycle implementation is externally accessible; an implementation carrying any other valid accessibility classifier follows the normal rules of that classifier;
8. ordinary `Type::new(...)` / `object::init(...)` lookup considers the developer-defined lifecycle override/overload candidates for the resolved class. The inherited compiler-provided lifecycle implementation is not automatically exposed as an ordinary source-callable candidate;
9. therefore `::new(...)` or `::init(...)` is valid for an ordinary caller only when a matching developer-defined lifecycle implementation exists and is accessible to that caller;
10. inside a valid lifecycle customization, access to an inherited parent lifecycle implementation is permitted when the ordinary protected/accessibility rules allow it; `this->parent::new(...)` in `@@new` and `this->parent::init(...)` in `@@init` select the primary parent, while `this->classes[Type]` or its `this->parents[Type]` alias explicitly selects a direct class parent by its resolved type

identity; mixin, trait, and interface relationship categories do not participate in lifecycle lookup;

11. lifecycle customization eligibility is independent of project/package/component placement and follows the ordinary placement rules of the enclosing generic class.

Lifecycle invocation uses the dedicated `::` form for source-visible developer lifecycle implementations:

```
Employee::new(...)
employee::init(...)
```

`::` is **not** a second general-purpose method-call operator. It performs lifecycle-member lookup and may name only a lifecycle invocation name defined by the language. In the current language profile those names are `new` and `init`.

Ordinary member lookup and lifecycle lookup therefore remain separate:

```
Employee.new(...)      // ordinary class/static method named new
Employee::new(...)     // developer-defined lifecycle @@new candidate

employee.init(...)     // ordinary instance method named init
employee::init(...)    // developer-defined lifecycle @@init candidate
```

Consequently, FoLang does not reserve or block ordinary methods named `new` or `init`. Ordinary `.` lookup and lifecycle `::` lookup use different semantic channels.

The declaration/invocation mapping is fixed:

Developer lifecycle declaration	Lifecycle invocation	Meaning
<code>@@new(...)</code> override/overload	<code>Type::new(...)</code>	customized allocation / uninitialized-instance lifecycle operation
<code>@@init(...)</code> override/overload	<code>object::init(...)</code>	customized instance-initialization lifecycle operation

A lifecycle call always includes its call parentheses. Bare `Type::new` or `object::init` is not a first-class member reference.

This is a category-wide rule: any future compiler-owned lifecycle declaration added with the declaration spelling `@@name` must use the corresponding invocation spelling `receiver::name(...)` when a developer-defined accessible customization is invoked. Lifecycle declarations are never invoked through ordinary `.` member syntax.

```
// Employee.fol
@co.dap.generic(
  types=[{name=T}, {name=R}],
  lifecycle=true
)
_ co.class = {

  id T;
  name R;

  // Public lifecycle override: externally accessible through
  Employee::new(...).
  @co.dap.class
  @co.dap.public
  @co.dap.override
  @@new(a co.hokrlt, b co.hokrlt)->(co.uninit) = {
    T co.type = a;
    R co.type = b;

    // Valid protected parent-lifecycle access from a lifecycle
    customization.
    this->parent::new();

    $=> co.uninit.instance(Employee, this);
  }

  // Private lifecycle overload/override: accessibility remains private.
  @co.dap.override
  @co.dap.constructor(access=private)
  @@init() = {}

  // Public lifecycle override/overload: externally accessible through
  ::init(...).
  @co.dap.override
  @co.dap.constructor(access=public)
  @@init(id T, name R) = {
    // Valid protected parent-lifecycle access from a lifecycle
    customization.
```

```

        this->parent::init();

        this.id    = id;
        this.name = name;
    }

    getEmployee(id T)->(Employee) = {}
}

a := Employee::new(co.int, co.string); // valid: public developer @@new
b := a::init(1, "Rao");                // valid: public developer
@@init

```

`lifecycle=true` alone does not expose the inherited compiler lifecycle implementations. If the generic class declares no developer lifecycle override/overload, it continues to use ordinary construction rules and has no newly exposed lifecycle API merely because the field is `true`.

A generic class without lifecycle customization permission remains an ordinary generic class:

```

@co.dap.generic(types=[{name=T}])
_ co.class = {
    value T;
}

```

```

// plain_generic_types.unit.fol
_ co.unit = {
    IntPlainGeneric co.type = PlainGeneric(co.int);
    someFun()->()={
        x IntPlainGeneric;
    }
}

```

The same metadata field is harmless on other generic declaration kinds:

```

@co.dap.generic(
    types=[{name=T}],
    lifecycle=true
)
_ co.struct = {
    value T;
}

```

The struct remains an ordinary generic struct; the `lifecycle` field is ignored for lifecycle semantics. The same target-insensitive metadata rule applies to generic functions and generic methods: `lifecycle=true` does not grant lifecycle customization to them and does not cause an error merely because the field is present.

```
_ co.unit = {  
  @co.dap.generic(  
    types=[{name=T}],  
    lifecycle=true  
  )  
  identity(value T)->(T) = {  
    $=> value;  
  }  
}
```

`identity` remains an ordinary generic function. Its `lifecycle` field is not considered. If source attempts to place `@@new` or `@@init` in a struct, function, method, unit, or any other non-class context, the lifecycle declaration itself is rejected because lifecycle customization is class-only.

Anonymous Classes/Types

Anonymous/inline ordinary classes use the compiler-provided class lifecycle machinery internally but cannot opt into developer lifecycle customization because they do not satisfy the generic-class `lifecycle=true` contract. They therefore expose no developer-defined lifecycle API. `// somunit.unit.fol`

```
_ co.unit = {  
  
  someFun ()->()={  
    emp := co.class{};  
  
    empObj := emp;  
  
    empObj1 := co.class{  
      name co.string;  
  
      @co.dap.public  
      doSomething(s co.string)->(co.int)={  
        co.out.println(s);  
      }  
    }  
  }  
}
```

```

        this.name=s;
        $=> 33;
    }
    }.init(); //parameter less init only as co.class doesn't have any
other init

    empObj1.doSomething("abc"); // compiler error: co.class does not
define this method
    }
}
// someInterface.fol
_ co.interface={

    doSomething(s co.string)->(co.int);
}

//someUnit.unit.fol

_ co.unit={

    someFun()->()={

        empObj1 someInterface = co.class{
            name co.string;

            @co.dap.public
            doSomething(s co.string)->(co.int)={
                co.out.println(s);
                this.name=s;
                $=> 33;
            }
        }.init(); //when assigned to interface only default init no
parameters

        empObj1.doSomething("abc"); // successful
        // internally the anonymous class implements the interface
someInterface as empObj1 is now the type someInterface
    }
}

//SomeClass.fol

_ co.class = {

```

```

name co.string;

@co.dap.public
doSomething(s co.string)->(co.int)={
    co.out.println(s);
    this.name=s;
    $=> 33;
}

}

//someUnit.unit.fol

_ co.unit={

    someFun()->()={

        empObj1 SomeClass = co.class{
            name co.string;

            @co.dap.public
            doSomething(s co.string)->(co.int)={
                co.out.println(s);
                this.name=s;
                $=> 33;
            }
        }.init(); //default init always valid

        empObj2 SomeClass = co.class{
            name co.string;

            @co.dap.public
            doSomething(s co.string)->(co.int)={
                co.out.println(s);
                this.name=s;
                $=> 33;
            }
        }.init(name="somename"); //valid as by default class will have
init with all field member implemented as optional named parameters

        empObj1.doSomething("abc"); // successful
        // internally the anonymous class inherits the class SomeClass as
empObj1 is now the type SomeClass

```

```
}  
}
```

FoLang internally creates anonymous class/object values through the ordinary anonymous-class construction and initialization machinery.

Their ordinary construction/use continues to follow the anonymous-class rules independently of the lifecycle facility.

Dependent Type Index Rules

An **index** is an argument to a dependent type, such as the `n` in `Vector(n)`, or a dimension in an array derivation, such as the `n` in `co.int->([N])`. Both positions obey the same rules.

Examples in this section use the following dependent `co.type` family. Because `N` occurs in the array-dimension position of the RHS, the compiler classifies it as a value/index parameter:

```
Buffer(N) co.type =  
  co.dependentType(co.int->([N]));
```

An index is a literal or a name

An index is an integer literal or a name. Arithmetic, function calls, indexing and every other operator are rejected. `// someldxEG1.unit.fol`

```
_ co.unit = {  
  Buffer(N) co.type = co.dependentType(co.int->([N]));  
  
  someFun()->()={  
    @co.dap.const SIZE co.int = 1024;  
  
    v Vector(3);           // ✓ literal  
    v Vector(SIZE);        // ✓ @co.dap.const name  
    buf Buffer(SIZE);      // ✓ named array type with the  
same index rule  
  
    v Vector(n + 1);       // ✗ arithmetic is not permitted    v Vector(computeSize()); // ✗ a call is not permitted in an
```

```

index
    buf Buffer(n * 2);           // ✗ same rule for array sizes
}
}

```

This restriction applies only to the **size** of an array, never to element access. Indexing an array is an ordinary expression and arithmetic is fine.

```

buf Buffer(SIZE);           // dependent type application; size is
restricted
buf[i + 1] = 42;           // access – unrestricted
buf[compute(x)] = 7;       // access – unrestricted

```

What a named index may resolve to

A name used as an index resolves in exactly one of two ways.

A parameter bound by the enclosing type declaration or callable signature. In a dependent type declaration, the declaration head introduces the name and the RHS determines its role. In a callable signature, an ordinary value parameter may be reused as an index, and an otherwise-unbound symbolic index appearing consistently in dependent type positions is bound by that signature according to the dependent family's declared parameter domain. //someEG2.unit.fol

```

_ co.unit = {
  // n is declared here; [n] classifies it as a value/index parameter.
  Vector(n) co.type =
    co.dependentType(co.int->([n]));

  // n is a symbolic dependent index shared by both parameter types.
  dotProduct(a Vector(n), b Vector(n))->(co.int) = {
    // ...
  }

  // n is an ordinary value parameter and is reused in the return type.
  readVector(n co.int->(Vector(n))) = {
    // ...
  }
}

```

A `@co.dap.const` **compile-time constant**. Outside a signature that binds it, a name has nothing to bind to, so it must be a constant the compiler can substitute.

```

_ co.object->(for=VectorClient) = {
  @co.dap.const SIZE co.int = 1024;

  allocate()->>() = {
    buf Buffer(SIZE); // ✓ SIZE substitutes to 1024
    v Vector(SIZE);   // ✓ same rule for dependent types
  }
}

```

Nothing else qualifies. `@co.dap.final` marks an immutable binding, and an immutable value need not be known while compiling, so it cannot be substituted.

```

_ co.object->(for=VectorClient) = {
  @co.dap.final n co.int = readInput();

  invalidIndices()->>() = {
    bad Vector(n);           // ✗ immutable, but not known at compile
time

    m co.int = 10;
    alsoBad Vector(m);       // ✗ an ordinary variable is not an index
  }
}

```

So in a plain variable declaration, where no signature is binding anything, the only legal names are `@co.dap.const` constants.

An index is non-negative

Zero is permitted; a negative index is not.

```

empty Buffer(0);           // ✓ zero-length array

buf Buffer(-1);            // ✗ rejected while parsing
v Vector(-1);             // ✗ rejected while parsing

_ co.object->(for=VectorClient) = {
  @co.dap.const OFFSET co.int = -1;

  invalidOffset()->>() = {
    buf Buffer(OFFSET); // ✗ rejected after substitution
  }
}

```

A negative literal cannot be written at all, because no prefix operator is reachable in an index position. A negative constant is rejected when the compiler substitutes it. Both are compile-time errors.

When two dependent types are equal

Two dependent types are equal when their constructors are the same and their indices are pairwise equal. An index comparison has exactly three cases.

Index form	Compared by
integer literal	value
<code>@co.dap.const</code> name	substituted literal value
parameter	name identity

```
Vector(3)    vs Vector(3)    // equal
Vector(n)    vs Vector(n)    // equal
Vector(n)    vs Vector(m)    // NOT equal – rejected
Vector(SIZE) vs Vector(1024) // equal when @co.dap.const SIZE = 1024
```

Rejecting `Vector(n)` against `Vector(m)` is the point of the feature. It is what lets the compiler catch a size mismatch without the developer writing a single check.

What FoLang deliberately does not do

FoLang does not decide index equality up to arithmetic. `Vector(n+1)` and `Vector(1+n)` are not merely unequal — they cannot be written.

Accepting them would require symbolic reasoning, and there is no partial version of it. Once `n+1 == 1+n` is accepted, the next reasonable request is `2*n == n+n`, and the type checker becomes a theorem prover by accretion. That is the complexity FoLang is built to avoid.

The cost is narrow. Length-arithmetic signatures such as `concat(Vector(n), Vector(m)) -> Vector(n+m)` are out of scope; return a dynamically sized type and check at run time instead. Everything that needs only same-parameter identity still works, and that covers the common cases.

```
multiply(a Matrix(r, n), b Matrix(n, c)) -> (Matrix(r, c))
dotProduct(a Vector(n), b Vector(n))      -> (co.int)
zip(a Vector(n), b Vector(n))             -> (Vector(n))
```

Matrix multiplication, the usual demonstration of dependent types, needs only that the shared `n` matches.

Dependent types are checked, never inferred

Every dependent type application appears explicitly in a written declaration or signature. FoLang never infers an omitted dependent type or an omitted index value. The parameter-role classification performed while resolving a defining `co.type` RHS is not dependent-type inference; it only classifies names already written in that declaration head.

This keeps checking decidable without a constraint solver and avoids Hindley-Milner-style whole-program inference of value indices. Inferring a dependent type at a use site would require inferring the index **value**, not merely an ordinary static type.

Conceptual Prototype Delegation for Non-Class Object Kinds

FoLang uses **prototype delegation** only as a **conceptual member-resolution model** for specifying how language-defined members are available on runtime object kinds that do not use inheritance. It defines the required member-resolution contract without prescribing a physical runtime representation.

The idea is loosely analogous to other mainstream languages which support prototype lookup only in one narrow sense: when a member is not supplied at the most specific level, lookup may be understood as continuing through successively more general language-defined member sources. FoLang does **not** adopt complete prototype object model, mutable prototype chains, `instanceof` semantics, prototype reflection, or prototype-based `is-a` relationships.

Default Backend may provide information of Prototype Object link on reflection but it is a kind of textual form like a label cannot be used for reading, traversing, updating etc.

Built-in Packages

Public path	Responsibility
<code>co</code>	language data types and kinds projected from <code>f$\Phi$$\lambda$.lang</code> ; network declarations such as

	<code>http</code> , <code>tcp</code> , and <code>udp</code> projected from <code>f0λ.net</code> ; and core collections/types such as <code>List</code> , <code>Map</code> , <code>Set</code> , and <code>Comparable</code> projected from <code>f0λ.core</code>
<code>co.sys</code>	file, concurrent, parallel, goto, invoke, bind, call, apply, setTimeout, setInterval, scheduler, cron, event
<code>co.os</code>	signal, cmd, execute, run, env, getenv, setenv, sleep, exit, cwd, chdir, fork, wait, pipe, dup, dup2, close, readfd, writefd, random
<code>co.meta</code>	ast, instrument, transform, augment, reflect, introspect, patch, inject, create, runtime(eval,etc), realm
<code>co.native</code>	load, register, asm, inline, emit, ffi, spawnon[gpu,cpu,npu,apu,fpga,asic,tpu,mki,mcu],arch[x86,x86-64,risc,arm,vliw]
<code>co.in</code>	read, readln
<code>co.out</code>	println, print
<code>co.sys.file</code>	
<code>co.regex</code>	pattern, match, search
<code>co.crypto</code>	rsa, aes, hash, md5, rand, uuid, ssl, tls
<code>co.dap</code>	built-in decorators and annotations, including backend-neutral standard runtime-operation implementation markers
<code>co.ddap</code>	built-in directives
<code>co.pdap</code>	built-in pragmas
<code>co.const</code>	<code>true</code> , <code>false</code> , <code>none</code>
<code>co.encoding</code>	base64Encode, base64Decode, json, yml, bson
<code>co.utils</code>	<code>makeImmutable</code> , <code>makeShared</code> , <code>copyOnWrite</code> , <code>toSnapshot</code> — object behavior policies
<code>co.dynamic</code>	dynamic capabilities
<code>co.runtime</code>	compiler-owned backend-neutral runtime-operation identifiers and semantic contracts; <code>co.runtime.operation.*</code> markers are

	implemented by the selected backend/runtime
<code>co.compiletime</code>	
<code>co.macro</code>	
<code>co.pattern</code>	
<code>co.control</code>	continuation, CPS, full/delimited continuation control, shift/reset, prompt/control, and continuation-oriented control abstractions
<code>co.cPCA</code>	concurrent/parallel/async submission, task/thread execution facilities, future/callback completion, await, pools, channels, events, actors, process/distributed facilities, scheduling, fiber/coroutine facilities, defer, lazy, and related execution APIs
<code>co.hokrlt</code>	
<code>co.operator</code>	"arity", "fixity", "associativity"
<code>co.operator.fixity</code>	"prefix", "infix", "postfix"
<code>co.operator.arity</code>	"unary", "binary"
<code>co.operator.associativity</code>	"left", "right", "none"
<code>co.hw</code>	cpu, memory
<code>co.stex</code>	
<code>co.http</code>	
<code>co.udp</code>	
<code>co.tcp</code>	

Contexts, Symbol Tables, and Symbols

FoLang distinguishes **named-definition contexts** from **lexical symbol-table scopes**.

- A named definition owns a `Context` and therefore participates in `$` / `$-` `>parent` navigation.
- An anonymous or labelled block creates a lexical child `SymbolTable` but no new source-visible Context.
- An ordinary declaration introduces a `SymbolInfo` entry into the active SymbolTable.

- `co.unit` contributes declarations to its package/companion owner and does not create a source-visible context object merely because it is a source grouping.

Context

```
Context {
    ParentId:                string,    // nearest enclosing named-
definition/structural Context
    ParentCtxSymbolTableId: string,    // exact enclosing visibility table
at the definition branch point
    Id:                      string,    // unique Context ID

    ImportedContextIds:      { <alias>: <context-id> },

    Prefix:                  string,
    ContextType_:            string,
    SymbolTables_:           [string], // lexical tables owned by this
Context; at least the root table
    ChildCtxIds:             [string], // direct child named-definition
Context IDs
    ResolutionPolicy:        string,
    OwnerSymbolId:           string    // named definition owning this
Context; empty for structural roots
}
```

`ParentId` is a named-definition/structural ownership relationship. Anonymous and labelled block scopes are not inserted into this Context chain.

`ParentCtxSymbolTableId` records the exact lexical visibility point in the parent Context at which this Context was created. This allows a nested named function declared inside a block to capture declarations visible at that definition point even though the block itself owns no Context.

Symbol Tables

```
SymbolTable {
    Id:                string,    // unique SymbolTable ID
    ParentId:          string,    // previous declaration-order segment in the
same lexical scope
    LexicalParentId: string,    // enclosing lexical-scope table active at
scope entry
    ContextId:         string,    // owning named-definition/structural
```

```
Context
  Prefix:          string,

  SymbolIds:       [ <symbol-id> ],
  SymbolsByName:   { <declaration-key>: [ <symbol-id> ] }
}
```

`ParentId` and `LexicalParentId` are intentionally different:

```
ParentId
  earlier declaration-order segment in the same lexical scope

LexicalParentId
  immediately enclosing lexical scope
```

Multiple anonymous/labelled block `SymbolTables` may therefore share the same `ContextId`.

Symbols

```
SymbolInfo {
  GetSymbolType() -> string
  GetSymbolID() -> string
  GetContextID() -> string
  GetType() -> string
  GetName() -> string
  IsInternal() -> bool
  SetOwnedContextID(string)
  GetSymbolTableID() -> string
  Anchor() -> string
}
```

Common symbol data:

```
SymbolDetails {
  SymbolId_          string
  OwnedContextId     context
  SymbolType_        string
  Name_              string
  IsInternal_        bool
  Type_              string
  SymbolTableId      string
  ResolutionState_   string // "resolved" | "unresolved" |
```

```
"partially_resolved"  
}
```

`OwnedContextId` is nonempty only when the declaration represented by the symbol owns a Context.

`SymbolTableId` identifies the table in which the symbol is defined. `Anchor()` returns the source-visibility anchor used by later resolution/semantic passes.

The source-level contextual model is therefore:

```
named definition  
  -> Context + root lexical SymbolTable  
  -> establishes $  
  
anonymous/labelled block  
  -> child lexical SymbolTable only  
  -> does not establish a new $  
  
ordinary declaration  
  -> SymbolInfo in the active SymbolTable
```

Appendix A - Derived FoLang EBNF

The standalone EBNF below is a derived documentation aid for the syntax defined by this reference. It is not normative and does not add to, narrow, or override the language requirements stated here. If it is incomplete or conflicts with this reference, this reference governs; update the EBNF to match the normative requirements.

```
(*  
  FoLang EBNF Grammar – current alpha profile  
  
  Derived from the normative requirements in:  
    docs/language-ref.md  
    SHA-256:  
f7607e6423f9f91775719f84249ef22440035d33f58cbeaad94f924531ba12f6  
    Bytes: 212158  
    Lines: 5065  
  
  Authority:  
    The language reference is normative. This EBNF is a derived aid only;
```

it may be incomplete and must not be used to infer syntax that the language reference does not define.

Notation:

= defines a production
; ends a production
| alternative
[...] optional
{ ... } zero or more
(...) grouping
"..." terminal text
? ... ? lexical or context-sensitive condition described in prose

Parser/lexer layering:

- Source text is UTF-8.
- U+FEFF is permitted only as the first source code point.
- white-space/comments are discarded between tokens.
- newlines never terminate statements.
- the parser consumes the token stream produced by the lexer.

Filesystem-selected parser roles:

src/appl.fol
-> application-entry-file

src/component.fol
components/application/component.fol
components/native/component.fol
components/dynamicvmrt/component.fol
components/packaged/component.fol
components/operators/component.fol
-> component-surface-file

ordinary package files

<Name>.fol
<Fragment>.unit.fol
<StructName>.comp.unit.fol
-> the matching package-source-file alternative selected from the externally validated filename class

All FoLang source under src/ and components/ uses this common grammar. Filesystem placement supplies the structural role/component kind and

the

semantic validation context. lib/*.folenc is deserialized rather than source-parsed.

Current-alpha rules that are deliberately explicit here:

- A named function block body requires "=" before the body.
- An anonymous function literal does not use "=" between

signature/body.

- Direct declaration/function/block bodies end at "}" and cannot take ";".

- Braced expressions/literals still require the enclosing statement ";".

- Object construction is `Type{field: value, ...}`.

- A type's arrow tail `"T->( ... )"` carries two structural meanings

that

the base and tail shape separate: a derivation applied to `T` or the

result

list of a function type. Generic and parameterized type applications

use

the uniform `Type(...)` postfix form. Type arguments are positional

and

bind in the generic declaration's parameter order:

`co.int->([5])`

derivation

`co.int->(&, meta={type=out})`

derivation,

attributes

`(co.int)->(co.int)`

function type

`co.List(co.string)`

positional type args

`co.Map(co.string, co.int)`

key, value

`co.Array(2, co.int, [2,4])`

bound values

An argument's value may be a type, an integer, or a bracketed list, which is what the `Array` and `Matrix` declarations bind.

- Generic arguments are positional; ``name=value`` remains exclusive to metadata and derivation attributes.

- A generic collection application is first named by a concrete type

alias.

That alias is followed directly by its literal body:

`StringList{"A","B","C"}`

`StringIntMap{"A": 1, "B": 2}`

`IntSet{1,2,3}`

The resolved type determines the entry interpretation.

`builtin-collection-type-name` records the reference's `Builtin`

Collections

registry, which is the closed set of built-in names admissible in

that

prefix position.

- There is no untyped composite literal. Every runtime list, array,

set,

map, struct, or class value names its concrete type and uses ``{ ...`

`}``.

Bare `{ ... }` and `[ ... ]` are not primary expressions. Bracketed lists remain available only where a type production explicitly admits a dependent value such as array dimensions. See "Canonical Object and Collection Construction" in the language reference.

- `.match()` means automatic matcher selection; `.match(expr)` selects an explicit matcher. Bare `.match` is invalid.
- Control-chain vocabulary uses `.then(X)` for one-shot conditional selection, `.loop(block)` for single-condition repeated execution, `.otherwise(condition)` only for another selection condition, and `.default(X)` only for a terminal selection fallback. A `.loop(block)` form has exactly one condition and cannot be followed by `.otherwise(condition)` or `.default(X)`. Element iteration is owned by `.each(...)` itself: its explicit-binding form takes index/key, value, and the per-element action directly, while its single-argument form takes a callable callback.

An `.each(...)` call cannot be followed by `.loop(...)`. There is no conditionless `.otherwise` form and no `.do` control verb in the current alpha profile.

- Built-in precedence follows "Built-In Operator Parse Table" in the language reference.
- The built-in type-relation operators `<:` and `:>` are binary infix relational operators at precedence 350. Relational and equality expressions are non-associative and therefore admit at most one operator at their respective precedence levels.
- A `co.predicateType` declaration has a dedicated named predicate binder inside `co.type.where(name => expression)`. This binder form is not a general lambda-expression and does not relax the collection-only placement rule for `|...| => ... lambdas`.
- The only current-alpha pre-declared operator glyphs are `U` and `n`. They are active binary infix operators at precedence 500, left associative; missing overload implementations fail during resolution, not parsing.
- `reserved-future-operator-fixity` is a diagnostic-recognition production: encountering an explicitly specification-reserved future fixity in the current alpha profile produces an unsupported-feature parser error.

- Examples never enable, disable, or reserve syntax. Active syntax comes from the normative language reference; reserved-future treatment applies only to spellings or forms that reference explicitly identifies as future/reserved.
- All built-in `@co.*` directives, annotations, pragmas and decorators use the single generic annotation application shape, but the complete `@co.*` name must first match the compiler's predefined built-in metadata registry. An unregistered `@co.*` name is a parser error. Once a known form is recognized, every supplied field/argument is collected and preserved; fields the frontend already understands may be validated, while unknown/unhandled fields of that known form do not block parsing or frontend artifact generation.
- Every named metadata field/attribute and every `co.operator` declaration attribute uses `"=`". The same binder applies at every metadata nesting level. The `":"` token remains the entry separator for ordinary object values and runtime map literals; it is never a declarative attribute binder.
- For a class with ordered direct class parents, ``this->parent`` and its ``this->super`` alias select the primary parent. The type-named plural form ``this->parents[SomeParent]`` aliases ``this->classes[SomeParent]``.
- ``classes[Type]``, ``mixins[Type]``, ``traits[Type]``, and ``interfaces[Type]`` are contextual compile-time relationship selectors available through ``this``. In a class method ``this`` denotes the current class/type; in an instance method it denotes the current instance. The key is a complete type name or import-alias-based type name that resolves to a direct entry of the matching ordered `@co.dap.oops` field. Numeric, string, computed, and runtime type-object keys are invalid. The selector categories are not runtime collections.
- Accessible inherited class/mixin fields are grouped by identifier. Matching canonical types merge into one child instance-state slot; mismatching canonical types are a semantic error. Field-level

constant,
 immutable, Shared, locking, and CopyOnWrite policies are rejected on
 class and mixin storage before merging. The broader accessible level
 is
 selected as public > protected > internal, while private fields do
 not
 enter the child-visible merge.

- A co.class contains ordinary mutable per-instance fields. An
 Immutable, Shared, or CopyOnWrite policy may target the complete
 class
 instance graph, but a selected class-owned field cannot be an
 independent policy root. Class-associated constant, immutable,
 global,
 shared, or locked state belongs to a named co.object.
- A co.object is one named singleton declaration and requires
 ->(for=Target) or ->(for=[Target, ...]). An annotation target
 denotes an
 annotation implementation; a homogeneous non-empty class-target list
 denotes an explicitly associated support object. The association is
 non-owning, non-structural, non-transitive, and never becomes a
 managed
 reference edge merely through for=.
- For methods, repeated paths to one canonical declaration
 deduplicate.
 Distinct matching concrete methods within one relationship category
 use
 the first source in that ordered list. Matching concrete methods
 across
 categories require an explicit child resolution. Matching abstract
 obligations may share one implementation, and matching mandatory
 virtual slots may share one override.
- `forall` is a contextual spelling recognized only in the value of a
 co.type declaration. Higher-rank parameters and results use the
 resulting named polymorphic type; functions and anonymous functions
 do
 not introduce forall binders directly.
- A co.type declaration may have a parameterized declaration head,
 as in Option(T) co.type or Vector(n) co.type. Declaration parameters
 are classified from their semantic use in the RHS. A type-position
 use
 makes a type parameter; a dependent index/value use under an
 outermost
 co.dependentType(...) RHS makes a value parameter. co.dependentType
 is
 an RHS type-expression constructor, not a declaration or result

kind.

- In a signature, ``Name co.type;`` is an ordinary abstract type slot. ``T co.associatedType;`` is narrower: `T` must parameterize at least one required `co.type` alias whose matching-module definition instantiates a generic container. The module repeats `co.associatedType` when binding `T`. An unused associated type is invalid.
- Typeclass `shape=(F(_))` metadata and `F(A)/F(B)` contract applications remain a context-bounded higher-kinded exception and are not general type-alias syntax.

Context-sensitive/semantic requirements intentionally remain outside EBNF:

filename canonicalization and package ownership; project layout and reachability; component/project-kind placement; import target uniqueness/mutual exclusion; projected-library capability

restrictions;

interface/signature/typeclass conformance;
instance placement/liveness; overload resolution; dependent-index name resolution; generic-argument binding by declared positional order;
whether a type prefix on a
collection body names a collection type; built-in metadata field

schemas

beyond frontend-known checks; custom annotation/decorator symbol resolution; annotation target/type checks; operator metadata

uniqueness,

ranges and arity/fixity compatibility; operator ownership

normalization,

where an operator belonging to or extending a generic type is associated

with that type's canonical declaration identity rather than with operator-level

generic parameters; rejection of `co.dap.operator + co.dap.generic` on the same

function-shaped declaration; and all type/runtime semantics.

Predicate-type binder scope and immutability, the requirement that its body resolve to `co.bool`, type-object membership, canonical type identity, and the subtype/supertype meaning of `<:` and `>:` are likewise semantic checks outside the context-free grammar.

An omitted initializer ordinarily places that storage location in the universal `co.const.none` state. Refinement-type and concrete dependent-

type

values are non-none exceptions: they cannot store, receive, pass,

return,

or otherwise admit `co.const.none`. Their declarations may omit an initializer, but then denote definitely-uninitialized storage with no

readable language-level value. Flow-sensitive definite-initialization analysis must prove a valid assignment on every reachable path before every value-producing access; otherwise the compiler reports Uninitialized. Uninitialized is the canonical case-sensitive name in the language's named diagnostic registry; messages, locations, and internal numeric identifiers may vary without changing that name. The complete registry is normative in the language reference and groups lexical, parser, project, symbol, type, generic, OOP, metadata, operator, effect, execution-model, usage, and backend rejection families. Diagnostic-name selection is semantic metadata rather than a context-free production. Parser recovery uses UnexpectedToken, ExpectedToken, or fallback InvalidSyntax; once a valid AST construct exists, later phases select the most specific applicable registered diagnostic and suppress derivative cascades. A simple assignment target is not a read. The intrinsic `isNone()` operation is semantically invalid for statically resolved refinement-type and dependent-type values. A construct whose missing-result behavior would substitute `co.const.none` into such a position is invalid, including applicable `continue`, `return`, `return_with_error`, and execution-model failure paths. Aggregate construction is checked per path. Every refinement/dependent struct field must be initialized before the struct value is returned or read. Class `@@new` may yield `co.uninit`, but every successful `@@init` path must initialize each refinement/dependent field before the instance becomes readable. Each `@@init` overload is analysed independently; multiple `@@new` overloads do not merge initialization proofs. For ordinary types that admit `none`, `none` is not a singleton and has no stable observable identity: a runtime may reuse or rematerialize its representation between accesses. Only intrinsic `value.isNone()` reliably observes the state and returns `co.const.true` or `co.const.false`. Equality, inequality, identity, ordering, hashing, and other observations involving `none` are semantically unspecified unless a construct defines explicit `none` behavior. The backend must not expose raw host-memory contents or backend undefined behavior while implementing this indeterminate language-level state. Optional-argument omission remains a separate state

and is not `co.const.none`.

Every `co.class` declaration has class construction semantics. FoLang has no abstract or unconstructable class category. `co.dap.sealed` on a class prevents its selection as a parent but does not prevent

constructing

that class; on a method it prevents overriding but not invocation. A `co.mixin` is a distinct composition declaration kind with no independent class-instance construction operation, not an

unconstructable

class.

A closed `co.variants(...)` result is the recommended, non-mandatory representation for expected domain alternatives such as

Found/NotFound.

Such a declared `None` variant is an ordinary initialized variant value and is distinct from `co.const.none`. No general rule requires a variant destination merely because `resolution=continue` may substitute `co.const.none`; tooling may issue only an advisory diagnostic for

ordinary

`none`-admitting types. Refinement/dependent result positions are the

strict

exception and reject the `none` substitution. `Continue` never

manufactures

a user-declared variant constructor.

Object-association target resolution, homogeneous target-kind

checking,

duplicate-target rejection, singleton lifetime, access rules, and the prohibition against propagating class inheritance or class-instance object policies through a `for=` association are semantic checks outside the context-free grammar. Rejection of field-level policy metadata on class/mixin storage and rejection of a class-owned selected field as

an

independent `makeImmutable/makeShared/copyOnWrite` policy root are

likewise

semantic checks.

Every recoverable effect is an instance of a class satisfying the

standard

`co.error` interface. `co.AbstractError` is the standard

`co.mixin` supplying common error state and concrete behavior; custom error classes normally compose that mixin while declaring `co.error` as an interface. The mixin is not a base class and has no independent class-instance construction semantics. Exact matching composed methods may satisfy interface slots; explicit source mappings use

`co.dap.implement` on the consuming

class. `co.dap.implementation` remains the distinct runtime-operation marker. `co.runtime.operation.effect.emit` is the backend-neutral

emission operation and accepts only `co.error`. Recoverable backend failures must be translated to such class instances. Fatal failures do not

satisfy `co.error` and cannot participate in effect metadata, handling, resolution, or propagation.

Effect declaration and handling use separate placements.

`co.dap.effects`

is legal only on ordinary callable declarations/definitions and requires

one emits list advertising known effects that may leave that callable.

It

is invalid on `co.dap.executionmodel` declarations. Every emits item must be

a class satisfying `co.error`. The list is open and non-exhaustive: absence of `co.dap.effects` advertises no explicit effects but does not prohibit undeclared effects from propagating.

`co.dap.onEffect` is illegal as declaration metadata and is parsed only by

effect-handled-call-expression immediately before one call expression. Each direct qualified-name field of `co.dap.onEffect` must resolve to a valid

effect type; it need not appear in the callee's open advertised set.

Each

ordinary-call record must contain exactly one singular resolution equal to

continue, retry, return, return_with_error, or propagate. The optional handlers field must be a non-empty ordered list of accessible caller-side

module values matching `co.EffectHandler`. For a call whose resolved callee is a `co.dap.executionmodel` declaration, each record instead requires

a non-empty handlers list and forbids resolution; return_with_error is implicit and fixed by the language. Any explicit resolution is invalid.

That signature constrains only `handle(error co.error)->()`; each concrete handle implementation may independently carry `co.dap.effects`, and both its advertised and unexpected effects may escape the handler.

`co.dap.onEffect` is forbidden throughout an effect-handler module, including call sites in private handler functions, so a handler cannot start a nested local effect-handling pipeline.

Their `handle(error co.error)->()` functions execute synchronously in list order before the resolution. Ordinary calls use the receiving caller

context; execution-model calls use the context observing the effect before

completion is published.

If handler H emits recoverable effect E1 while handling original effect

E0, remaining handlers are skipped and E0's selected resolution is cancelled. The standard library creates a `co.GenericError` class instance satisfying `co.error` and wrapping E0, E1, H, and available causal/source/runtime diagnostic context. At an ordinary call this wrapper propagates from the enclosing caller; at an execution-model

boundary it is returned through the ordinary error-result position and otherwise unproduced none-admitting results receive `co.const.none`.

Such a

missing-result path is invalid for refinement/dependent positions. The wrapper is not

reprocessed by the same handler list. `GenericError` is not created for normal propagate or `return_with_error` paths, which preserve E0, and a fatal handler failure terminates immediately without wrapping.

`retry` requires `retry={max_attempts=PositiveCompileTimeInteger, on_exhausted=TerminalResolution}`; `TerminalResolution` is `continue`,

`return`,

`return_with_error`, or `propagate`. Receiver and argument expressions are evaluated once and their values are reused by `retry`. `continue`

substitutes

`co.const.none` for each missing call result. `return` and

`return_with_error`

likewise use `co.const.none` for otherwise unproduced caller result slots;

`return_with_error` is checked against the enclosing caller's error result.

Propagate and unmatched effects leave the caller whether or not they were

explicitly advertised; statically known escaping effects are included in

computed outgoing symbol metadata. Every `co.dap.executionmodel` declaration

must expose exactly one `co.error-compatible` result and forms an effect boundary: its first observed unhandled recoverable effect is converted to

that ordinary result, unmatched effects receive the same conversion without

handlers, and no recoverable effect propagates to the submitting caller.

Known internal effects may remain implementation metadata but are not advertised outward. For concurrent/parallel execution, first observed

may

```

depend on scheduling unless a model defines deterministic priority.
There is no effects wrapper, resolutions list, or invoke resolution in
co.dap.onEffect.
co.dap.defer provides no injected error or completion-status binding;
it
registers explicitly parameterized completion work for LIFO execution
on
every enclosing-function exit after registration is reached.
*)

(* =====
*)

(* 1. Source roots and externally selected file forms *)

(* =====
*)

compilation-unit = package-source-file
                  | application-entry-file
                  | component-surface-file ;

source-filename = companion-unit-filename
                  | ordinary-unit-filename
                  | application-entry-filename
                  | component-surface-filename
                  | ordinary-source-filename ;

companion-unit-filename = filename-identifier, ".comp.unit.fol" ;

ordinary-unit-filename = filename-identifier, ".unit.fol" ;

application-entry-filename = "appl.fol" ;

component-surface-filename = "component.fol" ;

ordinary-source-filename = filename-identifier, ".fol" ;

filename-identifier = identifier-head, { "_", identifier-segment } ;

package-source-file = package-primary-source-file
                     | ordinary-unit-source-file
                     | companion-unit-source-file ;

```

```
package-primary-source-file = file-preamble, primary-declaration ;
```

```
ordinary-unit-source-file = file-preamble, unit-declaration ;
```

```
companion-unit-source-file = file-preamble, unit-declaration ;
```

```
application-entry-file = file-preamble, { entry-item } ;
```

```
component-surface-file = file-preamble, component-declaration ;
```

```
file-preamble = { file-directive } ;
```

(* The file preamble is the only syntactic position for language-owned
DIRECTIVE/PRAGMA metadata. It is parsed before the primary top-level
declaration (or before the first non-metadata entry item in
src/appl.fol).

Directives are recorded on the source-file/top-level semantic context;
they

do not introduce lexical names, scopes, or symbol-table entries. A
directive

may affect the file's primary declaration when its semantic contract
says so,

without becoming a declaration annotation or lexical member.

File directives and pragmas use the generic metadata application shape.
The zero-width category guard consults the predefined built-in metadata
registry. Unknown @co.* names fail during parsing; unknown fields of a
known

form remain collectable/preservable. *)

```
file-directive = import-directive  
                | alias-directive  
                | use-directive  
                | dynamic-runtime-directive  
                | dynamic-dispatch-directive  
                | other-file-metadata-directive ;
```

```
other-file-metadata-directive =  
    annotation, file-directive-category-guard ;
```

```
file-directive-category-guard =  
    ? zero-width parser condition: the just-parsed metadata name is  
    registered
```

as a built-in DIRECTIVE or PRAGMA, is not one of the specifically
guarded

directive forms above, and is permitted in this file context; custom

```

    annotation/decorator applications are not standalone file directives
? ;

entry-item = type-declaration
           | let-function-pattern-clause
           | entry-statement ;

entry-statement = variable-declaration
                | inferred-variable-declaration
                | grouped-variable-declaration
                | multiple-assignment-statement
                | expression-statement
                | empty-statement ;

primary-declaration = struct-declaration
                    | cstruct-declaration
                    | enum-declaration
                    | union-declaration
                    | class-declaration
                    | trait-declaration
                    | mixin-declaration
                    | interface-declaration
                    | signature-declaration
                    | module-declaration
                    | typeclass-declaration
                    | object-declaration
                    | instance-declaration
                    | matcher-instance-declaration
                    | extension-declaration ;

(* =====
*)

(* 2. Directives, annotations, and metadata *)

(* =====
*)

annotations = { declaration-metadata } ;

declaration-metadata = annotation, declaration-metadata-category-guard ;

declaration-metadata-category-guard =
    ? zero-width parser condition: for a built-in co.* metadata name, the

```

```

registry category is ANNOTATION or DECORATOR; DIRECTIVE and PRAGMA
are
    rejected in declaration/member/block annotation positions.
co.dap.onEffect is also rejected here because its only legal parser
path
    is on-effect-call-metadata before a call expression. co.dap.effects
is
    accepted here only when the annotated node is callable; that target
check
    is semantic. A non-co.* metadata name is accepted here as custom
    annotation/decorator syntax and is resolved later through the
ordinary
    symbol table ? ;

(* Generic application shape. Sharing this surface syntax does not grant
placement permission: the surrounding category guard decides whether the
parsed metadata application is legal in the current syntactic position.
The complete qualified name is classified
immediately after it is read:
    - co.* -> must match builtin-metadata-name; otherwise parser error.
    - non-co.* -> syntactically accepted only as a custom
annotation/decorator
    application and later resolved through the ordinary symbol table.
Field names are intentionally NOT closed by this registry. Every field
of a
    recognized form is parsed and preserved; frontend-known fields may
receive
    additional validation, while unknown/unhandled fields remain available
to
    later stages. *)
annotation = "@", qualified-name,
    [ "(", [ annotation-argument-list ], ")" ] ;

builtin-metadata-name-check =
    ? parser validation applied immediately after annotation qualified-
name:
    when the name begins with co.*, the complete name must match
    builtin-metadata-name (or a registered reserved/future entry) and an
    unregistered co.* name is a parse error; a non-co.* name is collected
as
    custom metadata and must later resolve through the symbol table to a
    user-defined annotation or decorator ? ;

builtin-metadata-name = builtin-pragma-name
    | builtin-directive-name

```

```
        | builtin-annotation-name
        | builtin-decorator-name ;

builtin-pragma-name = "co.pdap.threadpool"
                    | "co.pdap.schedularpool" ;

builtin-directive-name = "co.ddap.import"
                        | "co.ddap.dynamicruntime"
                        | "co.ddap.use"
                        | "co.ddap.alias"
                        | "co.ddap.dynamicdispatch"
                        | "co.ddap.overload" ;

builtin-annotation-name = "co.dap.template"
                        | "co.dap.macro"
                        | "co.dap.operator"
                        | "co.dap.annotation"
                        | "co.dap.library"
                        | "co.dap.module"
                        | "co.dap.native"
                        | "co.dap.class"
                        | "co.dap.static"
                        | "co.dap.instance"
                        | "co.dap.object"
                        | "co.dap.inline"
                        | "co.dap.ctfe"
                        | "co.dap.friend"
                        | "co.dap.sealed"
                        | "co.dap.extension"
                        | "co.dap.override"
                        | "co.dap.implement"
                        | "co.dap.virtual"
                        | "co.dap.abstract"
                        | "co.dap.delegate"
                        | "co.dap.dynamicscope"
                        | "co.dap.lexicalscope"
                        | "co.dap.staticscope"
                        | "co.dap.mixedscope"
                        | "co.dap.typeclass"
                        | "co.dap.matcher"
                        | "co.dap.constructor"
                        | "co.dap.oops"
                        | "co.dap.extends"
                        | "co.dap.hokrlt"
                        | "co.dap.indexer"
```

```
| "co.dap.generic"  
| "co.dap.comptime"  
| "co.dap.typefromvalue"  
| "co.dap.local"  
| "co.dap.private"  
| "co.dap.public"  
| "co.dap.compose"  
| "co.dap.guard"  
| "co.dap.package"  
| "co.dap.protected"  
| "co.dap.internal"  
| "co.dap.export"  
| "co.dap.eager"  
| "co.dap.lazy"  
| "co.dap.packed"  
| "co.dap.declare"  
| "co.dap.implementation"  
| "co.dap.simd"  
| "co.dap.reflection"  
| "co.dap.mop"  
| "co.dap.nested"  
| "co.dap.inner"  
| "co.dap.final"  
| "co.dap.const"  
| "co.dap.decorator"  
| "co.dap.specialize"  
| "co.dap.symbol" ;
```

```
builtin-decorator-name = "co.dap.before"  
| "co.dap.after"  
| "co.dap.around"  
| "co.dap.effects"  
| "co.dap.onEffect"  
| "co.dap.defer"  
| "co.dap.callable"  
| "co.dap.executionmodel" ;
```

```
annotation-argument-list = annotation-argument,  
                           { ",", annotation-argument }, [ ",", ] ;
```

```
annotation-argument = [ annotation-key, annotation-binder ], annotation-  
value ;
```

```
annotation-binder = declarative-attribute-binder ;
```

```

declarative-attribute-binder = "=" ;

annotation-key = annotation-key-segment,
                { "-", annotation-key-segment }
                | qualified-name ;

annotation-key-segment = identifier | "for" ;

annotation-value = literal
                  | qualified-name
                  | annotation-declaration-reference
                  | annotation-list
                  | annotation-map
                  | annotation-arrow-pair ;

annotation-list = "[", [ annotation-value,
                          { ",", annotation-value }, [ ",", " ] ], "]" ;

annotation-map = "{", [ annotation-map-entry,
                        { ",", annotation-map-entry }, [ ",", " ] ], "}" ;

annotation-map-entry = annotation-map-key, annotation-binder,
                       annotation-value ;

annotation-map-key = annotation-key | qualified-name ;

annotation-arrow-pair = string-literal, "=>", string-literal ;

annotation-declaration-reference = qualified-name, "(",
                                  [ annotation-reference-type,
                                    { ",", annotation-reference-type } ],
                                  ")",
                                  "->", "(",
                                  [ annotation-reference-type,
                                    { ",", annotation-reference-type } ],
                                  ")" ;

annotation-reference-type = qualified-name ;

(* These productions are contextual views over annotation. They
   deliberately do
       not close the field list: recognized fields may be validated by the
   frontend
       while additional fields of the same known form remain parsed/preserved.
   *)

```

```

import-directive = annotation, import-directive-guard ;

import-directive-guard =
    ? zero-width parser condition: metadata name is exactly co.ddap.import;
    known target fields are package, library, component and as; semantic
    validation requires exactly one of package/library/component, but
    additional unhandled fields remain preserved ? ;

alias-directive = annotation, alias-directive-guard ;

alias-directive-guard =
    ? zero-width parser condition: metadata name is exactly co.ddap.alias ?
;

use-directive = annotation, use-directive-guard ;

use-directive-guard =
    ? zero-width parser condition: metadata name is exactly co.ddap.use ? ;

dynamic-runtime-directive = annotation, dynamic-runtime-directive-guard ;

dynamic-runtime-directive-guard =
    ? zero-width parser condition: metadata name is exactly
    co.ddap.dynamicruntime ? ;

dynamic-dispatch-directive = annotation, dynamic-dispatch-directive-guard ;

dynamic-dispatch-directive-guard =
    ? zero-width parser condition: metadata name is exactly
    co.ddap.dynamicdispatch ? ;

(* =====
*)

(* 3. Names and references *)

(* =====
*)

filename-derived-name = "_" ;

qualified-name = ( identifier | "co" ), { ".", identifier } ;

```

```

declaration-reference = qualified-function-reference | qualified-name ;

qualified-function-reference = qualified-name, "(", [ type-list ], ")",
                                declaration-return-type-clause ;

lifecycle-declaration-name = "@@new" | "@@init" ;

lifecycle-invocation-name = "new" | "init" ;

special-binding = result-binding | recursive-binding ;

this-receiver-expression = "this", this-receiver-context-guard,
                            ordinary-relationship-selector-exclusion-guard ;

relationship-selector-expression =
    "this", this-receiver-context-guard,
    "->", relationship-arrow-guard, relationship-category,
    "[", relationship-type-name, "]",
    relationship-selector-guard,
    compiler-owned-property-call-exclusion-guard ;

relationship-category = "classes"
                        | "mixins"
                        | "traits"
                        | "interfaces" ;

relationship-arrow-guard =
    ? zero-width contextual condition: "->" has relationship-selection
    meaning
    only when its left operand is the hard-reserved this token and its
    right
    token is a language-defined relationship name; value->member is
    invalid,
    while this.member remains ordinary member access ? ;

compiler-owned-this-selector-expression =
    "this", "->", compiler-owned-this-selector-name,
    compiler-owned-this-selector-guard,
    compiler-owned-property-call-exclusion-guard ;

compiler-owned-this-selector-name =
    "object" | "class" | "module" | "kind" | "type" | "struct"
    | "instance" | "callee" | "args" | "params" | "results" |
    "associatedtype"
    | "owner" | "caller" | "fallthrough" | "yield" | "builtins" ;

```

compiler-owned-this-selector-guard =

? zero-width contextual and semantic condition: the selected compiler-owned

property or operation is available in the enclosing executable context;

these names are contextual after this-> rather than globally reserved;

this->args cannot implicitly capture an enclosing invocation's arguments

when the nearest enclosing callable is annotated @co.dap.defer; values

needed by that deferred callable must be passed explicitly at registration

and received through its declared parameters ? ;

compiler-owned-property-call-exclusion-guard =

? zero-width syntactic condition: a compiler-owned this-> attribute is a

property and cannot be followed immediately by call-suffix. Indexing that

belongs to a keyed relationship property is part of its primary form; ordinary member-suffix, lifecycle-call-suffix where otherwise permitted,

and a later call of an ordinary member remain available ? ;

relationship-type-name = qualified-name ;

relationship-selector-guard =

? zero-width contextual and semantic condition:

- the occurrence is inside the applicable class context;

- the selected category maps exactly to the ordered direct

relationship

list with the same name in @co.dap.oops;

- only directly declared relationships are present; transitive relationships are not appended;

- relationship-type-name is a compile-time declaration reference written

as a complete type name or through an applicable import alias;

- the written name resolves to one canonical declaration of the kind required by the selected category and that declaration occurs in

the

corresponding direct relationship list;

- a numeric, string, runtime expression, computed expression, co.type value, unresolved name, ambiguous name, wrong-kind name,

- absent direct relationship, or missing key is invalid;
- classes, mixins, traits, and interfaces are not standalone runtime values or collections;
- only classes selects a class-parent lifecycle branch;
- mixins and traits may qualify a composed implementation branch;
- interfaces produces an interface type/view and ordinary interface dispatch, not an implementation-branch selection;
- whenever a complete `this->classes`, `this->mixins`, `this->traits`, or `this->interfaces` prefix occurs in an applicable class context, this specialized production is selected before ordinary qualified-name/member/index postfix parsing ? ;

parent-selector-expression =

```
"this", this-receiver-context-guard,
( "->", relationship-arrow-guard, ( "parent" | "super" )
| "->", relationship-arrow-guard, "parents", "[", relationship-type-
name, "]" ),
direct-parent-selector-guard,
compiler-owned-property-call-exclusion-guard ;
```

direct-parent-selector-guard =

? zero-width contextual and semantic condition:

- the occurrence is inside the applicable class context;
- direct class parents are selected in their declared classes=[...] order;
- singular `this->parent` and `this->super` select the first direct class parent;
- plural `this->parents[Type]` is identical to `this->classes[Type]`;
- only plural `this->parents` is keyed, and it always requires a

compile-time

relationship-type-name;

- the type name must resolve canonically to a direct class parent;
- `this->parent[Type]`, `this->super[Type]`, numeric keys, string keys,

computed keys, runtime

`co.type` values, and unkeyed `this->parents` are invalid;

- this is a compile-time branch selector, not runtime indexing;
- repeated canonical ancestors use one virtual-base identity;
- whenever a complete `this->parent`, `this->super`, or `this->parents`

prefix occurs in an

applicable class context, this

specialized production is selected before ordinary

qualified-name/member/index postfix parsing ? ;

ordinary-relationship-selector-exclusion-guard =

? zero-width parser lookahead condition: the immediately following

tokens

are none of the complete contextual prefixes "->", "classes", "->",
"mixins",
"->", "traits", "->", "interfaces", "->", "parent", "->", "super", or
"->", "parents"
in a context where relationship-selector-expression or
parent-selector-expression applies ? ;

this-receiver-context-guard =

? zero-width contextual condition: this occurrence denotes a receiver
in

one of the following contexts:

- an instance method of co.class, where it denotes the current instance;
- a co.dap.class method of co.class, where it denotes the current class/type;
- a developer lifecycle declaration admitted by the lifecycle customization guard, where @@new supplies the class/type receiver

and

- @@init supplies the instance receiver;
- an instance or co.dap.class method of a target-bound co.extension, where it denotes respectively an instance of fortype or the fortype class/type; or
- a concrete/default method of co.trait or co.mixin, where it denotes the current composed instance; or
- a function of co.object, where it denotes the current singleton object.

A co.dap.static method and an ordinary free, module, signature,
interface,

typeclass, instance, trait, mixin, or unit function do not acquire a receiver through this production. The separately parsed this =>, this ^=>, this ->>, and this ->| control forms do not require a receiver ? ;

refinement-candidate = "_", refinement-candidate-guard ;

refinement-candidate-guard =

? zero-width contextual condition: this occurrence is inside the
predicate
of the co.refinementType declaration currently being parsed ? ;

recursive-binding = "\$" ;

result-binding = "\$", digit, { digit } ;

```
wildcard = "_" ;

(* =====
*)

(* 4. Type syntax *)

(* =====
*)

type-expression = union-type-expression ;

polymorphic-type-expression = "forall", "(", type-parameter-list, ")", ".",
                             type-expression,
                             forall-context-guard ;

forall-context-guard =
    ? zero-width contextual condition: this occurrence is the value of a
      co.type declaration; every other direct use of forall as a type
      binder is invalid ? ;

type-declaration-value = polymorphic-type-expression | type-expression ;

(* co.dependentType(type-expression) is parsed by the ordinary type-
application
grammar below but has a reserved semantic role when it is the outermost
RHS
of a co.type declaration: it marks that family as value-indexed. Its
head
parameters are still syntactically bare identifiers; semantic resolution
classifies each one as a type or value parameter from its RHS positions.
*)

type-parameter-list = identifier, { ",", identifier } ;

union-type-expression = arrow-type-expression,
                      { "|", arrow-type-expression } ;

arrow-type-expression = type-postfix-expression,
                      [ "->", arrow-type-tail ]
                      | "(", [ function-type-parameter,
                              { ",", function-type-parameter } ],
                      ")", "->", arrow-type-tail ;
```

```

arrow-type-tail = type-derivation
                | parenthesized-type-list
                | type-expression ;

type-postfix-expression = type-atom, { type-argument-list } ;

type-atom = qualified-name
          | "(", type-expression, ")" ;

type-argument-list = "(", [ type-or-value-argument,
                          { ",", type-or-value-argument } ], ")" ;

type-or-value-argument = type-expression | dependent-index | annotation-
list ;

dependent-index = integer-literal | qualified-name ;

type-list = type-expression, { ",", type-expression } ;

(* A parenthesized postfix on a type atom is the uniform type-application
form.
   Symbol resolution classifies its target as a built-in or annotation-
declared
   generic, a parameterized co.type family, a typeclass, or an abstract
type
   parameter. Arguments bind positionally in declaration order, and non-
type
   values use the same argument slots:

       co.Map(co.string, co.int)
       co.Array(2, co.int, [2,4])
       LinkedList(co.int)

   Named bindings are deliberately absent. Parentheses after a generic
   declaration bind that declaration's parameters by position, while an
arrow
   tail may carry the derivation-attribute-list described by "Type
Application
   and Arrow Tails". *)
parenthesized-type-list = "(", [ return-item-list ], ")" ;

type-derivation = "(", derivation-specification, ")" ;

derivation-specification = pointer-specification
                        | array-specification

```

```

| reference-specification
| range-type-specification
| slice-type-specification
| thunk-type-specification
| address-type-specification
| derivation-attribute-list ;

pointer-specification = pointer-stars,
                        [ ",", derivation-attribute-list ] ;

pointer-stars = ? one contiguous symbolic run consisting only of one or
                more "*" characters; its length is the pointer degree ? ;

reference-specification = ( "&" | "&&" | "~" ),
                        [ ",", derivation-attribute-list ] ;

address-type-specification = "@", [ ",", derivation-attribute-list ] ;

thunk-type-specification = "^", [ ",", derivation-attribute-list ] ;

slice-type-specification = "[:]", [ ",", derivation-attribute-list ] ;

range-type-specification = "..", [ ",", derivation-attribute-list ] ;

array-specification = array-dimension-group, { array-dimension-group },
                    [ ",", derivation-attribute-list ] ;

array-dimension-group = "[", array-dimension-content, "]" ;

array-dimension-content = [ array-dimension ], { ",", [ array-dimension ] }
;

array-dimension = "... " | ". " | dependent-index ;

(* Attributes carried by a type derivation. Generic type applications
instead
   use the positional type-argument-list production. *)
derivation-attribute-list = derivation-attribute,
                        { ",", derivation-attribute } ;

derivation-attribute = annotation-key, "=", annotation-value ;

return-type-clause = "->", "(", [ return-item-list ], ")" ;

return-item-list = return-item, { ",", return-item } ;

```

```

return-item = type-expression ;

type-use = qualified-name, { type-argument-list } ;

type-use-argument = type-use
                    | dependent-index
                    | identifier, "=", ( type-use | dependent-index ) ;

declaration-return-type-clause =
    "->", "(", [ declaration-return-item,
                { ",", declaration-return-item } ], ")" ;

declaration-return-item = type-use ;

(* =====
*)

(* 5. Common declaration components *)

(* =====
*)

generic-parameter-clause = "(", generic-parameter,
                           { ",", generic-parameter }, ")" ;

generic-parameter = identifier, [ generic-arity-clause ] ;

generic-arity-clause = "(", generic-arity-slot,
                       { ",", generic-arity-slot }, ")" ;

generic-arity-slot = "_" | identifier ;

kind-options = "->", "(", [ annotation-argument-list ], ")" ;

field-declaration = annotations, identifier, type-use,
                    [ "=", expression ], statement-end ;

embedded-field-declaration = annotations, type-use, statement-end ;

value-specification = annotations, identifier, type-use, statement-end ;

variable-declaration = annotations, typed-variable-declarator,
                        { ",", typed-variable-declarator }, statement-end ;

```

```

typed-variable-declarator = identifier, type-use,
                           [ "=", expression ] ;

inferred-variable-declaration = annotations, inferred-variable-declarator,
                                { ",", inferred-variable-declarator },
                                statement-end ;

inferred-variable-declarator = identifier, definition-operator, expression
;

definition-operator = ( ":@" | "::=" | "?=" ),
                      multi-symbol-infix-operator-boundary-guard ;

(* =====
*)

(* 6. Data and type declarations *)

(* =====
*)

struct-declaration = annotations, filename-derived-name,
                      "co.struct", "=", struct-body ;

surface-struct-declaration = annotations, identifier,
                              "co.struct", "=", struct-body ;

struct-body = "{", { struct-member }, body-close ;

pure-field-declaration = annotations, identifier, type-use,
                          statement-end ;

struct-member = pure-field-declaration | embedded-field-declaration ;

cstruct-declaration = annotations, filename-derived-name,
                      "co.cstruct", "=", cstruct-body ;

surface-cstruct-declaration = annotations, identifier,
                              "co.cstruct", "=", cstruct-body ;

cstruct-body = "{", { pure-field-declaration }, body-close ;

enum-declaration = annotations, filename-derived-name,

```

```

        "co.enum", "=", enum-body ;

enum-body = "{", [ enum-variant,
                    { enum-separator, enum-variant }, [ enum-separator ] ],
            body-close ;

enum-separator = "," ;

enum-variant = annotations, identifier,
               [ "(", enum-state-parameter-list, ")" ],
               [ "=", constant-expression ] ;

enum-state-parameter-list = enum-state-parameter,
                             { ",", enum-state-parameter } ;

enum-state-parameter = identifier, type-expression ;

union-declaration = annotations, filename-derived-name,
                    "co.union", "=", union-body ;

union-body = "{", { pure-field-declaration }, body-close ;

data-declaration = annotations, identifier,
                   [ generic-parameter-clause ], "co.data", "=",
                   data-variant, { "|", data-variant }, statement-end ;

data-variant = qualified-name,
               [ "(", type-list, ")" ] ;

type-declaration = ( polymorphic-type-declaration
                     | simple-type-declaration
                     | refinement-type-declaration
                     | predicate-type-declaration ),
                   type-declaration-context-guard ;

type-declaration-context-guard =
    ? zero-width condition: a named non-UDT type declaration occurs in
      src/appl.fol, a non-operator component surface, a unit, class,
interface,
    module, mixin, signature, function, or nested executable block;
    co.associatedType is governed separately and remains limited to
    signatures and matching modules ? ;

polymorphic-type-declaration = annotations, identifier,
                               [ generic-parameter-clause ], "co.type",

```

```
[ kind-options ],  
[ "=", type-declaration-value ],  
type-family-parameter-role-guard,  
statement-end ;
```

type-family-parameter-role-guard =

? zero-width semantic condition: for a co.type declaration with a
parameterized head, each head parameter is classified from its
resolved

RHS uses. Use in a type position classifies a type parameter. Use in
a

dependent index/value position under an outermost
co.dependentType(...)

RHS classifies a value parameter whose domain is imposed by that
consuming construct. A parameter with incompatible roles, no
resolvable

role, or an outermost co.dependentType(...) RHS with no value
parameter

is InvalidGenericDeclaration. co.dependentType(...) is not a
declaration

kind, variable type, or callable result kind ? ;

simple-type-declaration = annotations, identifier, nonpolymorphic-type-
declaration-kind,

```
[ kind-options ], [ "=", type-expression ],  
statement-end ;
```

nonpolymorphic-type-declaration-kind = "co.newtype"

```
| "co.opaquetype"  
| "co.subtype"  
| "co.supertype"  
| "co.kind" ;
```

refinement-type-declaration =

annotations, identifier, "co.refinementType", "=",
refinement-type-expression, statement-end ;

refinement-type-expression =

"(", type-expression, ")", ".where", "(", expression, ")" ;

predicate-type-declaration =

annotations, identifier, "co.predicateType", "=",
predicate-type-expression, statement-end ;

predicate-type-expression =

```
"co.type", ".where", "(", predicate-type-binder, "=>",  
expression, ")" ;
```

```
predicate-type-binder = identifier ;
```

```
(* =====  
*)
```

```
(* 7. Containers, contracts, instances, and component surfaces *)
```

```
(* =====  
*)
```

```
unit-declaration = annotations, filename-derived-name,  
                    "co.unit", "=", unit-body ;
```

```
unit-body = "{", { unit-member }, body-close ;
```

```
unit-member = function-declaration  
              | data-declaration  
              | type-declaration  
              | function-object-declaration  
              | delegate-declaration  
              | extern-variable-declaration ;
```

```
extern-variable-declaration =  
    "@co.dap.declare", "(", "type", "=", "extern", ")",  
    identifier, type-expression,  
    statement-end ;
```

```
class-declaration = annotations, filename-derived-name,  
                    "co.class", [ kind-options ], "=", class-body,  
                    class-lifecycle-capability-guard ;
```

```
class-lifecycle-capability-guard =  
    ? zero-width semantic condition:  
        - every co.class has compiler-owned inherited lifecycle  
implementations  
        for @@new and @@init as non-public/protected lifecycle machinery;  
        - lifecycle is an optional field of the enclosing class's  
co.dap.generic  
    metadata application, not a separate annotation;  
    - for a generic co.class, lifecycle=true grants source permission to  
    override or overload the existing compiler-owned @@new / @@init  
family;
```

```

        lifecycle absent or lifecycle=false forbids developer lifecycle
        customization but does not remove the inherited compiler lifecycle;
    - if lifecycle is present on a generic struct, generic function, or
generic
    method, that field is accepted but is not considered for lifecycle
    semantics;
    - if any lifecycle-method-declaration occurs in the class body, the
class
    must carry valid co.dap.generic(types=[...], lifecycle=true)
metadata;
    - lifecycle=true does not itself expose ::new(...) or ::init(...);
    - no filesystem/package/component placement condition is imposed by
the
    lifecycle facility itself ? ;

class-body = "{", { class-member }, body-close ;

class-member = class-instance-field-declaration
               | function-declaration
               | type-declaration
               | lifecycle-method-declaration ;

class-instance-field-declaration =
    pure-field-declaration, class-instance-field-policy-guard ;

class-instance-field-policy-guard =
    ? zero-width semantic condition:
    - the declaration is ordinary mutable per-instance storage;
    - co.dap.const and co.dap.final are forbidden on the field;
    - field-level Shared, locking, CopyOnWrite, static/class/global
storage,
    ownership, atomicity, and inline initialization are forbidden;
    - an object-graph policy may later target the complete class
instance,
    never this selected field as an independent class-storage root ? ;

trait-declaration = annotations, filename-derived-name,
                    "co.trait", "=", trait-body ;

trait-body = "{", { trait-member }, body-close ;

trait-member = function-declaration, trait-member-guard ;

trait-member-guard =
    ? zero-width semantic condition:

```



```

interface-body = "{", { function-specification | type-declaration }, body-
close ;

signature-declaration = annotations, filename-derived-name,
                        "co.signature", "=", signature-body ;

signature-body = "{", { signature-member }, body-close ;

signature-member = value-specification
                  | function-specification
                  | signature-type-component
                  | associated-type-requirement
                  | type-declaration ;

signature-type-component = annotations, identifier,
                          "co.type",
                          [ "=", type-expression ], statement-end ;

(* `Name co.type;` is an ordinary abstract type component and does not
   imply genericity. Every co.associatedType component must name a type
   argument used by at least one bodyless co.type component of the same
   signature. A matching module must fulfill that alias by instantiating a
   generic container with the associated type; otherwise the associated-
   type
   declaration is invalid. *)
associated-type-requirement = annotations, identifier,
                             "co.associatedType", statement-end ;

module-declaration = annotations, filename-derived-name,
                    "co.module", [ kind-options ], "=", module-body ;

module-body = "{", { module-member }, body-close ;

module-member = variable-declaration
              | inferred-variable-declaration
              | function-declaration
              | type-declaration
              | associated-type-binding ;

associated-type-binding = annotations, identifier,
                        "co.associatedType", "=", type-expression,
                        statement-end ;

(* Each module co.associatedType binding corresponds by name to a
   requirement in the matched signature. The module must also contain the

```

corresponding required co.type alias, and that alias must instantiate a generic container using the associated type. Missing, unrelated, or unused bindings are compiler errors. Typeclass and instance higher-kinded forms are unaffected. *)

```
component-declaration = annotations, filename-derived-name,  
                           "co.component", "=", component-body,  
                           component-surface-context-guard ;
```

```
component-body = "{", { component-member }, body-close ;
```

```
component-surface-context-guard =
```

```
  ? zero-width structural validation supplied by filesystem role:  
    src/component.fol is either a projected standalone library annotated  
with  
    co.dap.library (application when type is omitted; otherwise  
application,  
    native, or dynamicvmrt) or a packaged standalone component containing  
the  
    co.dap.export package selector, never both; components/application,  
    components/native, and components/dynamicvmrt expose only their  
permitted  
    projected boundary declarations and carry no co.dap.library  
annotation;  
    components/packaged uses the export selector; components/operators  
    contains operator declarations only; the operators component is  
permitted  
    only for an executable application or a projected application library  
? ;
```

```
component-member = import-directive  
                  | surface-struct-declaration  
                  | surface-cstruct-declaration  
                  | type-declaration  
                  | function-declaration  
                  | component-export-selector  
                  | operator-declaration ;
```

```
component-export-selector = annotation, component-export-selector-guard ;
```

```
component-export-selector-guard =
```

```
  ? zero-width structural/context condition: metadata name is exactly  
    co.dap.export and this standalone component-body entry is used only
```

for

packaged src/component.fol or components/packaged/component.fol; the parser collects the complete field payload, while semantic validation checks the packages selector and project/component context ? ;

extension-declaration = annotations, filename-derived-name,
 "co.extension", extension-target-options, "=",
 extension-body ;

extension-target-options =
 "->", "(", "fortype", "=", type-expression, ")" ;

extension-body = "{", { extension-member }, body-close ;

extension-member = field-declaration | function-declaration ;

object-declaration = annotations, filename-derived-name,
 "co.object", object-association-options,
 "=", object-body, object-association-guard ;

object-association-options =
 "->", "(", "for", "=", object-association-targets, ")" ;

object-association-targets =
 qualified-name
 | "[" , qualified-name , { "," , qualified-name } , [","] , "]" ;

object-association-guard =
 ? zero-width semantic condition:
 - the scalar target resolves to exactly one annotation declaration or exactly one co.class declaration;
 - the list form is non-empty, contains only co.class declaration references, contains no duplicate canonical target, and cannot mix annotation and class declaration kinds;
 - the object name follows the ordinary filename-derived primary-name rule and is independent of every target name; no companion filename

or

companion-owner relationship is created;
- the declaration denotes one singleton object whose association is explicit, non-owning, non-structural, and non-transitive;
- association does not promote fields or methods, participate in

class

inheritance/dynamic dispatch, add a managed-reference graph edge,

or

propagate construction, copying, freezing, sharing, locking,
CopyOnWrite behavior, or destruction between either side ? ;

```
object-body = "{", { field-declaration | function-declaration }, body-close  
;
```

```
instance-declaration = annotations, filename-derived-name,  
                        "co.instance", [ kind-options ], "=",  
                        instance-body ;
```

```
instance-body = "{", { function-declaration | variable-declaration }, body-  
close ;
```

```
typeclass-declaration = typeclass-annotations, filename-derived-name,  
                        "co.typeclass", "=", contract-body ;
```

```
typeclass-annotations = { annotation }, typeclass-annotation, { annotation  
} ;
```

```
typeclass-annotation = "@co.dap.typeclass", "(",  
                        typeclass-option, { ",", typeclass-option }, ")" ;
```

```
typeclass-option = typeclass-kind-option  
                  | typeclass-shape-option  
                  | typeclass-aliases-option ;
```

```
typeclass-kind-option = "kind", annotation-binder, identifier ;  
typeclass-shape-option = "shape", annotation-binder, typeclass-shape ;  
typeclass-aliases-option = "aliases", annotation-binder,  
                            "[", typeclass-alias,  
                            { ",", typeclass-alias }, "]" ;  
typeclass-alias = "{", "name", annotation-binder, identifier, ",",  
                  "type", annotation-binder, type-expression, "}" ;
```

```
(* Exactly one typeclass-annotation is required. Kind and shape are  
required;  
   aliases is optional, option order is immaterial, and duplicate options  
are  
   invalid. Its shape is metadata-owned; no parameter clause may occur  
after  
   filename-derived-name. *)
```

```
typeclass-shape = generic-parameter-clause ;
```

```
matcher-instance-declaration = annotations, filename-derived-name,
```

```

        "co.matcher", matcher-options, "=",
        matcher-body ;

matcher-options = "->", "(", "type", annotation-binder, type-expression,
")" ;

matcher-body = "{", { field-declaration | function-declaration }, body-
close ;

contract-body = "{", { function-specification | value-specification },
body-close ;

named-block-declaration = annotations, identifier,
        "co.block", "=", block, body-closure-guard ;

(* =====
*)

(* 8. Functions and callable declarations *)

(* =====
*)

delegate-declaration = annotations, identifier,
        "co.delegate", "=", function-type, statement-end ;

function-object-declaration = annotations, identifier,
        "co.function", "=",
        function-object-binding ;

function-object-binding = expression, statement-end ;

function-declaration = annotations,
        [ receiver-clause, receiver-clause-context-guard ],
        function-name,
        function-parameter-lists,
        [ declaration-return-type-clause ], function-
binding,
        function-shaped-declaration-classification-guard ;

function-parameter-lists = parameter-list
        | nonempty-parameter-list, nonempty-parameter-
list,
        { nonempty-parameter-list } ;

```

```

function-shaped-declaration-classification-guard =
  ? zero-width semantic classification:
    when attached directly to this function-shaped declaration, the
following
    classifying metadata forms select specialized AST kinds:
      co.dap.generic          -> GenericFunctionDecl
      co.dap.decorator        -> DecoratorDecl
      co.dap.extension        -> ExtensionMethodDecl
      co.dap.macro            -> MacroDecl
      co.dap.template         -> TemplateDecl
      co.dap.native           -> NativeFunctionDecl
      co.dap.executionmodel   -> ExecutionModelFunctionDecl
      co.dap.operator         -> OperatorOverloadDecl
      co.dap.indexer          -> IndexerDecl
    these classifying forms are mutually exclusive on one function-shaped
declaration except for the deliberate co.dap.operator +
co.dap.extension
    composition. In that combination the declaration remains an
    OperatorOverloadDecl and co.dap.extension supplies the existing
target/owner;
    it does not create a second AST declaration kind. co.dap.operator and
    co.dap.generic are incompatible on the same declaration, with or
without
    co.dap.extension: an operator declaration never introduces operator-
level
    generic parameters. A generic enclosing class, struct, or extension
target
    does not make the operator declaration generic; operator ownership is
associated with the canonical owner declaration identity. Every other
combination of two classifying forms is a compiler error. When none
is
    present the declaration is an ordinary FunctionDecl; non-classifying
metadata does not change its AST declaration kind. An
    ExecutionModelFunctionDecl must have exactly one co.error-compatible
result and must not carry co.dap.effects ? ;

function-name = identifier ;

receiver-clause = "(", ( type-use
                        | identifier, type-use ), ")" ;

receiver-clause-context-guard =
  ? zero-width condition: an explicit receiver occurs only on a function
declared directly in a validated <StructName>.comp.unit.fol

```

```

companion;
    ordinary units and nested class, module, signature, interface,
function,
    or executable-block contexts do not inherit receiver permission ? ;

parameter-list = "(", [ parameter,
                        { ",", parameter } ], ")" ;

nonempty-parameter-list = "(", typed-parameter,
                           { ",", typed-parameter }, ")" ;

parameter = typed-parameter | untyped-template-parameter ;

typed-parameter = [ "..." ], [ "~" ], identifier, [ "?" ],
                  type-use, [ "=", expression ] ;

untyped-template-parameter = [ "..." ], [ "~" ], identifier, [ "?" ],
                              [ "=", expression ],
                              template-parameter-context-guard ;

template-parameter-context-guard =
    ? zero-width condition: the enclosing function-shaped declaration
carries
    built-in @co.dap.template metadata; ordinary functions and every
other
    callable category require an explicit parameter type ? ;

function-binding = function-definition
                  | function-delegation
                  | function-type-value-binding
                  | function-alias-binding
                  | statement-end ;

function-definition = "=", block, body-closure-guard ;

function-delegation = "=>>", expression,
                     { "=>>", expression }, statement-end ;

function-type-value-binding = "=", type-expression, statement-end ;

function-alias-binding = "=", non-block-expression, statement-end ;

function-specification = annotations, function-name,
                        parameter-list, { parameter-list },
                        [ declaration-return-type-clause ], statement-end

```

```

;

function-type = "(", [ function-type-parameter,
                      { ",", function-type-parameter } ], ")",
               return-type-clause ;

function-type-parameter = type-expression
                        | identifier, type-expression ;

anonymous-function-expression = parameter-list, declaration-return-type-
clause, block,
                               anonymous-function-binding-context-guard,
                               anonymous-function-generic-context-guard ;

anonymous-function-binding-context-guard =
    ? zero-width condition: the anonymous-function expression is the root
value
    of a typed, inferred, let, field or co.function binding initializer;
    an optional postfix invocation may immediately follow it, but the
literal
    cannot be a standalone statement, argument, return value or nested
operand ? ;

anonymous-function-generic-context-guard =
    ? zero-width condition: the anonymous function declares no generic
names;
    any generic names in its signature belong to an enclosing
    @co.dap.generic struct, class, function or method ? ;

lambda-expression = "|", [ lambda-parameter,
                          { ",", lambda-parameter } ], "|", "=>",
                      ( expression | block ) ;

lambda-parameter = identifier, [ type-use ] ;

local-function-declaration = annotations, function-name, parameter-list,
                              { parameter-list }, declaration-return-type-
clause,
                              function-definition,
                              local-function-nongeneric-guard ;

local-function-nongeneric-guard =
    ? zero-width condition: an inner/local function must not carry
    co.dap.generic and cannot introduce generic parameters; it may use

```

types

already visible from an enclosing generic class or function ? ;

```
(* =====  
*)
```

```
(* 9. Function-pattern clauses and patterns *)
```

```
(* =====  
*)
```

```
let-function-pattern-clause = annotations, "let", identifier,  
                                pattern-parameter-list,  
                                [ where-clause ], "=", pattern-result ;
```

```
pattern-parameter-list = "(", [ pattern,  
                                { ",", pattern } ], ")" ;
```

```
where-clause = ".where", "(", expression, ")" ;
```

```
pattern-result = block, body-closure-guard  
                | non-block-expression, statement-end ;
```

```
pattern = wildcard  
        | literal-pattern  
        | binding-pattern  
        | constructor-pattern  
        | record-pattern  
        | tuple-pattern  
        | qualified-name ;
```

```
literal-pattern = literal  
                | ( "+" | "-" ),  
                  ( integer-literal | floating-literal ) ;
```

```
binding-pattern = identifier ;
```

```
constructor-pattern = qualified-name, "(", constructor-pattern-item,  
                                { ",", constructor-pattern-item }, ")",  
                                enum-state-pattern-guard ;
```

```
constructor-pattern-item = [ identifier, "=" ], pattern ;
```

```
enum-state-pattern-guard =
```

```

    ? zero-width semantic condition: when qualified-name resolves to a
      parameterized co.enum state, every item has identifier "=", every
      declared state parameter occurs exactly once, and no unknown or
duplicate
      label occurs; when it resolves to a zero-parameter enum state the
bare
      qualified-name form is required instead; co.variants and co.data
      constructors retain their separately specified positional patterns ?
;

record-pattern = qualified-name, "{", [ record-pattern-field,
                                         { ",", record-pattern-field } ],
"}" ;

record-pattern-field = identifier, [ ":", pattern ] ;

tuple-pattern = "(", pattern, ",", pattern,
                { ",", pattern }, ")" ;

match-case = ".case", "(", match-case-body, ")" ;

match-case-body = pattern, [ ":", expression ], "=>",
                    ( expression | block ) ;

match-default = ".default", "(", ( expression | block ), ")" ;

(* =====
*)

(* 10. Statements, blocks, and termination *)

(* =====
*)

block = "{", { block-item }, [ block-tail-expression ], "}" ;

block-item = statement ;

block-tail-expression = expression ;

statement = named-block-declaration
            | type-declaration
            | variable-declaration
            | inferred-variable-declaration

```

```

| grouped-variable-declaration
| let-value-declaration
| local-function-declaration
| multiple-assignment-statement
| enclosing-callable-return-statement
| return-statement
| break-statement
| continue-statement
| lock-statement
| labeled-block
| labeled-loop-statement
| expression-statement
| block-statement
| empty-statement ;

```

```

grouped-variable-declaration = "(", typed-variable-declarator,
                                { ",", typed-variable-declarator }, ")",
                                statement-end ;

```

```

let-value-declaration = "let", identifier, [ type-expression ], "=",
                        expression, statement-end ;

```

```

multiple-assignment-statement = assignment-target, ",", assignment-target,
                                { ",", assignment-target }, "=",
                                expression-list, statement-end ;

```

```

assignment-target = postfix-expression
                  | tuple-assignment-target ;

```

```

tuple-assignment-target = "(", assignment-target, ",", assignment-target,
                          { ",", assignment-target }, ")" ;

```

```

return-statement = "this", "=>", [ expression-list ], statement-end ;

```

```

enclosing-callable-return-statement =
    "this", "^=>", [ expression-list ], statement-end,
    enclosing-callable-return-guard ;

```

```

enclosing-callable-return-guard =
    ? zero-width contextual and semantic condition:
        - the occurrence is inside an anonymous block supplied as a call
argument;
        - no named function, method, anonymous function, function expression,
or
        lambda boundary occurs between the statement and that argument

```

```

block;
    - the statement terminates the anonymous block and its nearest
lexically
    enclosing callable, producing values for that callable;
    - the values satisfy that callable's declared result positions;
    - a direct callable-body occurrence, bare or named block occurrence,
      function-expression occurrence, and every ^^=> or multi-level
spelling
    is invalid ? ;

expression-statement = non-block-expression, statement-end ;

break-statement =
    "this", "->|", [ label-reference ], statement-end,
    break-target-guard ;

break-target-guard =
    ? zero-width semantic condition:
        - without label-reference, this ->| targets the ordinary nearest
          breakable control region;
        - with label-reference, the reference resolves only to an active
enclosing
        labeled-block or labeled-loop-statement;
        - resolution is lexical and selects the innermost enclosing matching
label;
        - the target is exited structurally; no arbitrary goto/address jump
exists ? ;

continue-statement =
    "this", "->>", [ label-reference ], statement-end,
    continue-target-guard ;

continue-target-guard =
    ? zero-width semantic condition:
        - without label-reference, this ->> targets the ordinary nearest
loop;
        - with label-reference, the reference must resolve to an active
enclosing
        labeled-loop-statement;
        - a plain labeled-block is not a valid continue target;
        - resolution is lexical and selects the innermost enclosing matching
label ? ;

labeled-block =
    label-declaration, ":", block, body-closure-guard ;

```

```

labeled-loop-statement =
    label-declaration, ":", expression-statement,
    labeled-loop-statement-guard ;

labeled-loop-statement-guard =
    ? zero-width semantic condition: the labeled expression-statement is a
      current-profile loop form whose outer control operation is
    .loop(...);
    labeling an arbitrary expression statement does not turn it into a
    loop ? ;

label-declaration = label-identifier ;

label-reference = label-identifier ;

empty-statement = ";" ;

expression-list = expression, { ",", expression } ;

statement-end = ";" ;

body-close = "}", body-closure-guard ;

body-closure-guard =
    ? zero-width condition: the next significant token is not ";", or there
    is no next token ? ;

block-statement = block, body-closure-guard ;

lock-statement = "lock", "(", expression, ")", block,
    body-closure-guard ;

non-block-expression = expression, non-block-expression-guard ;

non-block-expression-guard =
    ? zero-width condition: the complete source span is not admissible as
    an
    unparenthesized block production; when both block and another braced
    expression are possible, the block reading has priority ? ;

(* =====
*)

```



```
equality-expression = relational-expression,  
    [ equality-operator, relational-expression ] ;  
  
equality-operator = ( "==" | "!=" ),  
    multi-symbol-infix-operator-boundary-guard ;  
  
relational-expression = range-expression,  
    [ relational-operator, range-expression ] ;  
  
relational-operator = "<" | ">" | multi-symbol-relational-operator ;  
  
multi-symbol-relational-operator = ( "<=" | ">=" | "<:" | ">:" ),  
    multi-symbol-infix-operator-boundary-guard ;  
  
range-expression = additive-expression,  
    [ range-operator, [ additive-expression ] ]  
    | range-operator, additive-expression ;  
  
range-operator = ( ".." | "<.." | "..<" | "<..<" ),  
    multi-symbol-range-operator-boundary-guard ;  
  
additive-expression = predeclared-glyph-expression,  
    { additive-operator, predeclared-glyph-expression } ;  
  
additive-operator = "+" | "-" ;  
  
predeclared-glyph-expression = multiplicative-expression,  
    { predeclared-operator-glyph,  
    multiplicative-expression } ;  
  
multiplicative-expression = unary-expression,  
    { multiplicative-operator, unary-expression } ;  
  
multiplicative-operator = "*" | "/" | "%" ;  
  
unary-expression = { prefix-operator }, power-expression ;  
  
prefix-operator = "+" | "-" | "!" ;  
  
power-expression = ( effect-handled-call-expression | postfix-expression ),  
    [ power-operator, unary-expression ] ;  
  
effect-handled-call-expression = on-effect-call-metadata,  
    postfix-expression,
```

```

effect-handled-call-context-guard ;

on-effect-call-metadata = annotation, on-effect-call-metadata-guard ;

on-effect-call-metadata-guard =
    ? zero-width parser condition: the complete metadata name just parsed
    is
        exactly co.dap.onEffect; co.dap.effects and every other metadata name
    are
        invalid in this expression-prefix position ? ;

effect-handled-call-context-guard =
    ? zero-width semantic condition: the decorated postfix-expression's
    outermost operation is an ordinary or lifecycle invocation. The
    metadata
        belongs to that one call AST node, not to calls nested in its
    receiver or
        argument expressions. Each effect key must resolve to a class
    satisfying
        co.error and need not be present in the callee's open advertised
        emits set. For an ordinary callee every record requires exactly one
        resolution and may contain a non-empty handlers list. For a callee
        classified as ExecutionModelFunctionDecl, or a canonical submission
    call
        statically bound to such a target, every record requires a non-empty
        handlers list, forbids an explicit resolution, and receives implicit
        return_with_error. Retry captures the evaluated receiver and argument
        values once and reinvokes only an ordinary decorated call node ? ;

power-operator = "**", multi-symbol-infix-operator-boundary-guard ;

postfix-expression = primary-expression,
                    { postfix-suffix | postfix-operator } ;

postfix-operator = "!" ;

postfix-suffix = call-suffix
                | index-suffix
                | member-suffix
                | lifecycle-call-suffix
                | match-suffix ;

call-suffix = "(", [ argument-list ], ")", enum-state-call-guard ;

enum-state-call-guard =

```

? zero-width semantic condition: when the call target resolves to a parameterized co.enum state, every argument has identifier "=", every declared state parameter occurs exactly once, and no unknown or duplicate label occurs; a zero-parameter enum state is a bare value and cannot take a call suffix; ordinary functions and co.variants or co.data state functions retain their own argument rules ? ;

argument-list = argument, { ",", argument } ;

argument = ([identifier, "="], expression)
 | block
 | lambda-expression
 | wildcard ;

index-suffix = "[", [expression-list], "]" ;

member-suffix = ".", (member-identifier | "for") ;

(* Lifecycle members are invoked through a dedicated call form rather than ordinary member lookup. Every class has compiler-owned inherited @@new and @@init lifecycle machinery, but those inherited implementations are not automatically source-callable through `::`. A generic class with co.dap.generic(..., lifecycle=true) may source-define lifecycle overrides or overloads. Only matching developer-defined lifecycle candidates participate in ordinary source `::` lookup, and each candidate retains its declared accessibility. The lifecycle field is ignored for lifecycle semantics on generic structs/functions/methods. The call parentheses are part of the production so lifecycle members cannot be selected as first-class ordinary member values.

Current invocation names map to declaration names as follows:

```

new    -> @@new
init   -> @@init

```

Future language-defined lifecycle names extend both the declaration-name and invocation-name sets; every lifecycle invocation still uses `::`. Semantic validation checks receiver kind, developer customization, accessibility, overload resolution, and construction state. *)

lifecycle-call-suffix = lifecycle-invocation-marker, lifecycle-invocation-name,
 "(" , [argument-list] , ")" ,
 lifecycle-call-context-guard ;

```

lifecycle-call-context-guard =
    ? zero-width semantic condition checked after receiver/type resolution:
        - lifecycle invocation performs lookup only in source-defined
lifecycle
    override/overload candidates admitted for the resolved class;
    - such candidates can exist only on a generic co.class carrying valid
      co.dap.generic(types=[...], lifecycle=true) metadata;
    - the inherited compiler-provided lifecycle implementation is not an
      automatically exposed ordinary-source `::` candidate;
    - overload resolution selects a matching developer-defined lifecycle
      signature and ordinary accessibility rules must permit the caller
to
    reach that candidate;
    - no matching accessible candidate is a compile-time error;
    - ordinary methods named new/init remain unrelated and continue to
use
    ordinary '.' member lookup ? ;

(* value.to(TargetType) is therefore parsed by the existing member-suffix
plus
    call-suffix path; explicit conversion/cast syntax introduces no
dedicated
    cast production. Target-type validity is resolved semantically. *)

member-identifier =
    ? an identifier token other than "match"; "match" is routed through
      match-suffix so matcher-chain syntax has one parser path ? ;

(* No unprefixed composite literal exists. Lists, arrays, sets, maps,
records,
    and objects all use the explicit Type{...} construction production.
Blocks occur
    only in grammar positions that explicitly request a block; a bare block
is not
    a general primary expression. *)
primary-expression = literal
    | special-binding
    | relationship-selector-expression
    | parent-selector-expression
    | compiler-owned-this-selector-expression
    | this-receiver-expression
    | refinement-candidate
    | qualified-name
    | grouped-expression

```

```

| tuple-expression
| composite-construction
| anonymous-class-expression
| anonymous-function-expression
| let-expression
| comprehension-expression ;

```

```
grouped-expression = "(" , expression , ")" ;
```

```
tuple-expression = "(" , expression , "," , expression ,
                    { "," , expression } , ")" ;
```

```
composite-construction = type-postfix-expression , "{",
                        [ construction-element ,
                          { "," , construction-element } , [ "," ] ] , "}",
                        composite-construction-guard ;
```

```
construction-element = expression , [ ":" , expression ] ;
```

```
composite-construction-guard =
```

```
    ? zero-width semantic condition: the prefix resolves to a constructible
      concrete type; list, array and set values admit element expressions,
```

```
maps
```

```
    admit key:value entries, and object/record values admit
```

```
identifier:value
```

```
    field initializers; "=" is forbidden as an entry binder because it
belongs
```

```
    to metadata records, not typed JSON-like runtime representations; a
generic
```

```
    collection declaration itself must first be specialized and named by
a
```

```
    co.type alias ? ;
```

```
(* A built-in generic collection application must first be named by a
concrete
```

```
    co.type alias. Every constructed value then uses Type{...}:
```

```

StringList co.type = co.List(co.string)
StringIntMap co.type = co.Map(co.string, co.int)
IntSet co.type = co.Set(co.int)
StringList{"A","B","C"}
StringIntMap{"A": 1}
IntSet{1,2,3}

```

```
Type[...] remains indexing and Type(...) remains invocation. An
```

```

unspecialized
  generic collection symbol is not a value constructor. *)

(* The reference's Builtin Collections registry. These names identify
generic
  declarations and cannot directly stand in a value-construction prefix.
Their
  specialized co.type aliases carry the applicable body form. *)
builtin-collection-type-name = "co.List"
                               | "co.Set"
                               | "co.Map"
                               | "co.Tree"
                               | "co.Trie"
                               | "co.Array"
                               | "co.Tuple" ;

anonymous-class-expression = "co.class", "{",
                              { class-member }, "}" ;

let-expression = "let", "(", "{", let-binding,
                 { ",", let-binding }, "}", ")",
                 ".in", "(", "{", expression, "}", ")" ;

let-binding = ( identifier | special-binding ), "=", expression ;

comprehension-expression = "for", "(", comprehension-binding, ")",
                           ".yield", "(", expression-list, ")" ;

comprehension-binding = pattern, "<-", expression ;

match-suffix = ".match", "(", [ expression ], ")",
               match-case, { match-case }, [ match-default ] ;

multi-symbol-infix-operator-boundary-guard =
  ? zero-width condition: a multi-symbol infix operator has an explicit
  boundary on both operand-facing sides; a boundary is whitespace, a
  comment, or an applicable delimiter, checked before separators are
  discarded ? ;

multi-symbol-range-operator-boundary-guard =
  ? zero-width condition: a multi-symbol range operator has an explicit
  boundary on every side for which an operand is present ? ;

extended-operator-expression =
  ? expression containing a registered project-local custom operator,

```

parsed by precedence climbing from its declared alpha-supported fixity, precedence, associativity, and arity; U and n are not handled by this extension path because they are language-owned operators in the ordinary precedence grammar; explicitly reserved future fixities remain unsupported; every registered multi-symbol operator satisfies the operand-facing boundary rule for its fixity ? ;

```
(* =====  
*)
```

```
(* 12. Informative control-chain shapes (not parser entry paths) *)
```

```
(* =====  
*)
```

```
(* Control vocabulary in the current alpha profile:
```

- .then(X) conditionally evaluates X once; X may be a block or value.
- .then(...) may participate in .otherwise(condition) selection

chains.

- .loop(X) is a single-condition repetition form and cannot participate in

- .otherwise(condition) chains or take a .default(X) fallback.

- .otherwise(condition) always introduces another selection condition; there

- is no conditionless .otherwise form.

- .default(X) is the optional terminal fallback of a selection chain; it is

- not part of loop syntax.

- .each(...) is itself element iteration and cannot be followed by .loop.

The explicit-binding form is .each(index-or-key, value, action), where

action may be any expression (including a block or anonymous function) or

a lambda-expression. The single-argument form accepts a callable callback;

semantic analysis requires it to match the receiver's iteration tuple.

These informative productions document recognizable shapes; ordinary parsing

still proceeds through the general postfix/member/call grammar above. *)

```

informative-condition-chain =
    "(", expression, ")", ".then", "(", block, ")",
    { ".otherwise", "(", expression, ")", ".then", "(", block, ")" },
    [ ".default", "(", block, ")" ] ;

informative-loop-chain =
    "(", expression, ")", ".loop", "(", block, ")" ;

informative-labeled-loop =
    label-declaration, ":", "(", expression, ")", ".loop", "(", block, ")"
;

informative-ternary-chain =
    "(", expression, ")", ".then", "(", expression, ")",
    { ".otherwise", "(", expression, ")", ".then", "(", expression, ")" },
    ".default", "(", expression, ")" ;

informative-each-chain =
    postfix-expression, ".each", "(",
    ( ( identifier | "_" ), ",", identifier, ",",
      ( expression | lambda-expression )
    | ( expression | lambda-expression ) ), ")" ;

informative-contains-chain =
    postfix-expression, ".contains", "(", expression, ")",
    ".then", "(", block, ")",
    [ ".default", "(", block, ")" ] ;

informative-pipeline-chain =
    postfix-expression,
    { ( ".filter" | ".map" | ".reduce" | ".forEach"
      | ".sortBy" | ".groupBy" | ".fold" ),
      "(", argument-list, ")" } ;

(* =====
*)

(* 13. Literals and lexical grammar *)

(* =====
*)

literal = builtin-literal ;

```

```
builtin-literal = integer-literal
                  | floating-literal
                  | string-literal
                  | character-literal
                  | boolean-literal
                  | none-literal ;

integer-literal = ( binary-integer-literal
                  | octal-integer-literal
                  | decimal-integer-literal
                  | hexadecimal-integer-literal ),
                  [ integer-suffix ] ;

binary-integer-literal = ( "0b" | "0B" ), binary-digit-sequence ;

octal-integer-literal = "0", [ octal-digit-sequence ] ;

decimal-integer-literal = nonzero-digit, { decimal-digit } ;

hexadecimal-integer-literal = hexadecimal-prefix,
                             hexadecimal-digit-sequence ;

hexadecimal-prefix = "0x" | "0X" ;

binary-digit-sequence = binary-digit, { binary-digit } ;

octal-digit-sequence = octal-digit, { octal-digit } ;

decimal-digit-sequence = decimal-digit, { decimal-digit } ;

hexadecimal-digit-sequence = hexadecimal-digit, { hexadecimal-digit } ;

integer-suffix = unsigned-suffix,
                [ long-suffix | long-long-suffix | size-suffix ]
                | long-suffix, [ unsigned-suffix ]
                | long-long-suffix, [ unsigned-suffix ]
                | size-suffix, [ unsigned-suffix ] ;

unsigned-suffix = "u" | "U" ;

long-suffix = "l" | "L" ;

long-long-suffix = "ll" | "LL" ;

size-suffix = "z" | "Z" ;
```

```

floating-literal = decimal-floating-literal
                  | hexadecimal-floating-literal ;

decimal-floating-literal = fractional-constant,
                          [ exponent-part ],
                          [ floating-point-suffix ]
                  | decimal-digit-sequence,
                    exponent-part,
                    [ floating-point-suffix ] ;

fractional-constant = decimal-digit-sequence, ".",
                      decimal-digit-sequence ;

hexadecimal-floating-literal = hexadecimal-prefix,
                              ( hexadecimal-fractional-constant
                                | hexadecimal-digit-sequence ),
                              binary-exponent-part,
                              [ floating-point-suffix ] ;

hexadecimal-fractional-constant = hexadecimal-digit-sequence, ".",
                                 hexadecimal-digit-sequence ;

exponent-part = ( "e" | "E" ), [ sign ], decimal-digit-sequence ;

binary-exponent-part = ( "p" | "P" ), [ sign ], decimal-digit-sequence ;

sign = "+" | "-" ;

floating-point-suffix = "f" | "F" | "l" | "L"
                      | "f16" | "F16"
                      | "f32" | "F32"
                      | "f64" | "F64"
                      | "f128" | "F128"
                      | "bf16" | "BF16" ;

character-literal = single-quote, alpha-basic-c-character, single-quote ;

alpha-basic-c-character =
    ? any translation character except apostrophe, backslash,
      carriage return, or line feed ? ;

string-literal = double-quote, { alpha-basic-s-character }, double-quote ;

alpha-basic-s-character =

```

? any translation character except double quote, backslash, carriage return, or line feed ? ;

double-quote = ? Unicode scalar value U+0022 ? ;

single-quote = ? Unicode scalar value U+0027 ? ;

backslash = ? Unicode scalar value U+005C ? ;

boolean-literal = "co.const.true" | "co.const.false" ;

none-literal = "co.const.none" ;

byte-order-mark =

? U+FEFF, permitted only as the first code point of a source file ? ;

(* Structured control labels use a dedicated apostrophe-prefixed lexical form.

The same token shape is used in declarations and references:

'outer: declaration prefix

'outer reference

A character literal remains distinct because it has a closing apostrophe,

e.g. 'c'. Labels are not ordinary identifiers and are not operators. *)

label-identifier =

single-quote, identifier, label-identifier-guard ;

label-identifier-guard =

? a zero-width assertion that the next character is not single-quote;
this prevents a complete character literal such as 'c' from being
tokenized as a label identifier plus a trailing apostrophe ? ;

identifier = identifier-head, { "_", identifier-segment },
 identifier-trailing-guard ;

identifier-trailing-guard =

? a zero-width assertion that the next character is not "_" ? ;

identifier-head = ascii-letter, { ascii-alphanumeric } ;

identifier-segment = ascii-alphanumeric, { ascii-alphanumeric } ;

ascii-alphanumeric = ascii-letter | decimal-digit ;

ascii-letter = "A" | "B" | "C" | "D" | "E" | "F" | "G" | "H"

```

| "I" | "J" | "K" | "L" | "M" | "N" | "O" | "P"
| "Q" | "R" | "S" | "T" | "U" | "V" | "W" | "X"
| "Y" | "Z"
| "a" | "b" | "c" | "d" | "e" | "f" | "g" | "h"
| "i" | "j" | "k" | "l" | "m" | "n" | "o" | "p"
| "q" | "r" | "s" | "t" | "u" | "v" | "w" | "x"
| "y" | "z" ;

```

```
binary-digit = "0" | "1" ;
```

```
octal-digit = "0" | "1" | "2" | "3" | "4" | "5" | "6" | "7" ;
```

```
hexadecimal-digit = decimal-digit
| "a" | "b" | "c" | "d" | "e" | "f"
| "A" | "B" | "C" | "D" | "E" | "F" ;
```

```
digit = decimal-digit ;
```

```
decimal-digit = "0" | nonzero-digit ;
```

```
nonzero-digit = "1" | "2" | "3" | "4" | "5" | "6" | "7" | "8" | "9" ;
```

```
hard-reserved-word = "co" | "let" | "this" | "for" | "f0λ" ;
```

```
contextual-keyword = "forall" ;
```

```
(* `::`, `->|`, and `^=>` are active structural markers consumed
respectively
by lifecycle-call-suffix, break-statement, and
enclosing-callable-return-statement. They are not ordinary infix
operators
and are therefore intentionally absent from reserved-operator. *)
lifecycle-invocation-marker = "::" ;
```

```
reserved-operator = "->>" | "<->" | "`" | backslash | "#" ;
```

```
predeclared-operator-glyph = "U" | "n" ;
```

```
reserved-future-operator =
? a complete documented reserved/future symbolic spelling that is not a
current built-in operator and not a predeclared-operator-glyph; the
lexer
recognizes the complete spelling and the parser reports unsupported ?
;
```

```
token = label-identifier
      | identifier
      | keyword-token
      | literal
      | delimiter-token
      | predeclared-operator-glyph
      | reserved-operator
      | symbolic-token
      | reserved-future-operator ;
```

```
keyword-token = hard-reserved-word ;
```

```
delimiter-token = "(" | ")" | "{" | "}" | "[" | "]"
                 | "," | ";" | double-quote | single-quote ;
```

```
symbolic-token = ? the complete maximal contiguous run of one or more
symbol
```

characters after comments, literals, and closed

```
composite
```

spellings are recognized; the run is preserved whole for
contextual classification and is never split as a
fallback ? ;

```
token-separator = white-space ;
```

```
line-comment = "//", { ? any Unicode scalar value except CR or LF ? } ;
```

```
block-comment = "/*", { block-comment-character }, "*/" ;
```

```
block-comment-character = ? any Unicode scalar value that does not begin
the
```

two-character sequence */ ? ;

```
line-break = "\r\n" | "\n" | "\r" ;
```

```
horizontal-white-space = " " | "\t" | "\f" ;
```

```
white-space = horizontal-white-space | line-break | line-comment | block-
comment ;
```

```
(* =====
*)
```

```
(* 14. Operator declarations inside components/operators/component.fol *)
```

```

(* =====
*)

(* The operator component uses the same component-surface-file parser root
as
    every other component.fol. Filesystem context restricts co.operator to
    components/operators/component.fol and forbids all non-operator members
    there. The scanner still recognizes each declaration head as one maximal
    symbolic run while this component builds the owning
ProjectOperatorTable. *)

operator-declaration = operator-symbol, "co.operator", "=",
                        operator-body, statement-end,
                        operator-declaration-context-guard ;

operator-declaration-context-guard =
    ? zero-width structural condition: declaration occurs only inside the
    _ co.component body of components/operators/component.fol; that
    component body contains only operator declarations and its project
kind
    is either executable application or projected application library ? ;

operator-body = "{", operator-property,
                { ",", operator-property }, "}" ;

operator-property = "fixity", declarative-attribute-binder, operator-fixity
                  | "precedence", declarative-attribute-binder,
                    ( "0" | decimal-integer-literal )
                  | "associativity", declarative-attribute-binder,
                    operator-associativity
                  | "arity", declarative-attribute-binder, operator-arity
                  | "commutative", declarative-attribute-binder,
                    boolean-literal
                  | "idempotent", declarative-attribute-binder,
                    boolean-literal
                  | "identity", declarative-attribute-binder,
                    operator-identity-value
                  | "foldable", declarative-attribute-binder, boolean-
literal
                  | "vectorizable", declarative-attribute-binder,
                    boolean-literal
                  | "distributes_over", declarative-attribute-binder,
                    operator-symbol-list
                  | "desugar", declarative-attribute-binder, string-literal

```

```

;

operator-fixity = "co.operator.fixity.infix"
                  | "co.operator.fixity.prefix"
                  | "co.operator.fixity.postfix"
                  | reserved-future-operator-fixity ;

reserved-future-operator-fixity =
    "co.operator.fixity.circumfix"
  | "co.operator.fixity.postcircumfix"
  | "co.operator.fixity.precircumfix"
  | "co.operator.fixity.mixfix"
  | "co.operator.fixity.ternary"
  | "co.operator.fixity.distfix" ;

operator-associativity = "co.operator.associativity.left"
                        | "co.operator.associativity.right"
                        | "co.operator.associativity.none" ;

operator-arity = "co.operator.arity.unary"
                 | "co.operator.arity.binary"
                 | "co.operator.arity.ternary"
                 | decimal-integer-literal ;

operator-identity-value = literal ;

operator-symbol-list = "[", [ operator-symbol-reference,
                              { ",", operator-symbol-reference } ], "]" ;

operator-symbol-reference = character-literal | string-literal ;

operator-symbol = ? a maximal run of one or more symbol characters, where a
                    symbol character is any character that is not an ASCII
                    letter, digit, underscore, whitespace, or one of the
                    delimiters ( ) { } [ ] , ; " ' ; the run must not be a
                    language-owned or hard-reserved symbol and must not
contain
                    // or /* ? ;

```